SECTION TWO: INTRODUCTION TO PROBLEM SOLVING

© 2023

CHAPTER TWELVE
Overview of Core Concepts of Computing

Overview

This chapter provides sufficient introductory knowledge on computing. It sheds light on the importance of computers in computing. Key concepts and definitions of terms were discussed. The chapter discusses the basic concept of computing and its operation. The input, processing, output, storage and communication operations were discussed in great detail. In addition, various types of computing were discussed.

Objectives

This chapter provides students with a basic knowledge of computers. By the end of the chapter, students should be able to:

a. clearly describe what computing is
b. list and give an explanation of the types of computers.
c. understand the components of computers
d. list and explain generations of computers.
e. explain clearly the computing system.

12.1 What is Computing?

Executing an algorithm is what computing is. More precisely, it is when there exists a modelling association between a gadget and the algorithm with its architecture, which is when computing takes place.

Computing improves mental ability—computers can make use of machines to make realistic any mental idea. The availability of automatic computers has significantly influenced how people approach problem solving, as well as the types of problems they feel capable of addressing. Among all technologies of recent times, computing has had the greatest impact on the trajectory of human history, which is now integrated into nearly all aspects of daily life.

Therefore, the term "computing" is defined as any purposeful activity that requires, benefits from, or results in the use of computers.

There are two explanations to why everybody should have the knowledge of computing:

1. Computing is the driving force behind all current cutting-edge and influential technologies, arts, sciences, law, medicine and so on, and will continue to be in the years to come.
2. Learning about computing reveals profound perceptions and raises queries about the nature of our brains, our civilization, and our environment.

For computing to take place, computer(s) is vital. No computing can be carried out without a computer. Computers come in many different forms, and countless computers are embedded in the devices we rely on in our daily lives. One can describe a computer system as consisting of the hardware and software components, along with the user who operates it and the context in which it is utilized.

The computer is quickly evolving into this 21st century's all-purpose device. Early computers were too large and expensive for the average person to own. As a result, they were restricted to a few scientific institutes and laboratories. Only computer engineers would be able to program them. The primary uses were for doing difficult computations in engineering and science.

The computer is no longer only used in laboratories. Computers and computing in general are now present in practically everything we use. The use of computers is spreading quickly. Its use cuts across many fields. Some examples of the fields that rely on computing include transportation, manufacturing, communication, engineering, the military, scientific research, space exploration, financial institutions, air travel, social sciences, humanities, and the extractive industries, among many others.

Computers first produced could only process numerical data, but as technology advanced, they could also process multimedia data, such as text, audio, and video, in addition to quantitative data. ICT, which refers to the merging of computer technology and communication technology, has led to the development of information and communication technologies, which have transformed virtually every aspect of human activity, including science, engineering, commerce, industry, intercontinental trade, transportation, philosophy, tourism, teaching, and research.

In modern times, being literate does not just mean having the ability to read and write but also encompasses computer literacy. As globalization has reduced the barriers of distance between nations and peoples, largely due to advances in ICT, having knowledge of the computer has become increasingly important.

12.1.1 The Computing System

The computing system is comprised of three key components: the user, the computer system, and the context in which the computer is utilized.

12.1.1.1 The Computer System

The hardware and software make up the computer system.

12.1.1.1.1 Hardware

Computers are furnished with both internal and external accessories, which are the physical components of the computer system. This includes casing, central processing unit, random access memory, hard disk, power pack, motherboard, keyboard, mouse, monitor, graphics card, and so on, which are the touchable parts that make up the entire computer system.

12.1.1.1.2 Software

Computer software comprises programs, procedures, code, and associated data that offer directions to computer hardware, guiding it on how to execute specific tasks. Two primary classifications of software exist:

A. System software: System software is in charge of managing, integrating, and controlling the various hardware components of a computer system. This enables other software and users of the system to interact with it as a cohesive unit without needing to worry about low-level details such as transferring data from memory to disk or displaying text on a screen. Typically, system software includes an operating system, essential utilities (such as disk formatting tools, text editors, and file managers), user authentication systems, display managers, administration tools, networking software, and device control programs.

Operating System

An Operating System (OS) is a set of software programs that controls and manages the functioning of a computer's hardware components. Its main tasks include loading and running application programs, managing input/output operations with peripheral devices, and providing a user-friendly interface for the user to interact with the computer. Some popular examples of operating systems include Microsoft Windows, Mac OS X, and Linux.

B. Application software Application software is intended to carry out particular tasks beyond the mere operation of the computer system. This type of software can be a standalone program such as an image viewer, a set of programs working together to achieve a goal like a spreadsheet or text processing, word processing, a group of associated but independent programs with a shared data format or common user interfaces such as Microsoft Office, or even a software system like a database management system (MySQL), that provides a service to various independent applications.

A symbiotic link exists between hardware and software; without software, hardware is severely limited and cannot function at all. They are both dependent on each other to reach their full potential, and they require one another to function.

Table 12.1: System Software Vs. Application Software

	System Software	Application Software

Usage	It empowers the computer to function	Empowers users to work effectively on the computer
Need	It is compulsory	It is optional, depending on the needs
Number of Software	Each computer needs one system software	Each computer can have more than one application
Dependency	It is Independent. This means that it can be used without application software.	It is dependent on the System software. It cannot operate without the system software.
Function	It provides an environment in which the applications run	Provides an environment that enables users to accomplish some specific tasks.

Table 12.2**:** Hardware Vs. Software

SN	**Software**	**Hardware**
1	This is a set of program instructions that tells a computer system what it should do	This contains the physical parts of the computer system.
2	Cannot be executed without the hardware	Hardware cannot execute any task without the software
3	Cannot be touched	Can be touched

4	Debugging is done when there is a problem	Repairs are carried out when there is a fault.
5	If the problem is not solved, the software is reinstalled	If the problem is not rectified, the hardware is replaced.

12.1.1.2 Computer Users

Users of computers are the various types of employees who use them. There are both seasoned users and newcomers. Computer engineers, computer programmers, and computer operators may be added to the list of expert users.

12.1.1.3 The Computing Environment

The computing system consists of the user, the computer system, and the environment in which it is used. This environment includes the building that houses the computer, users, and furniture, as well as additional equipment such as voltage stabilizers, UPS, fans, and air conditioners. Figure 1 provides a schematic diagram of the computing system.

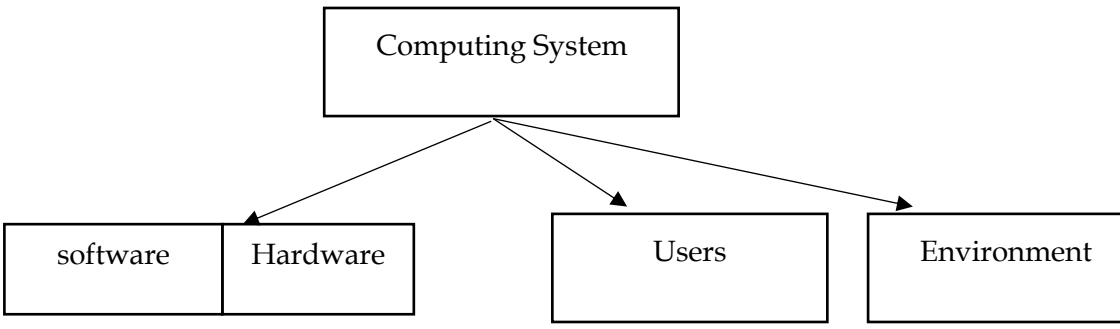


Figure 12.1: A Schematic diagram of the computing system.

12.2 The Basic of Operations/Concepts of Computing

In computing, a basic operation can be described as any activity that is done to complete an assignment. Five main operations exist in computing, and these include input, processing, output, storage and communication operations.

These operations are carried out by some functional units in the computer. These units include the input, storage and output units as well as the central processing unit (CPU), which houses the arithmetic logic unit (ALU) together with the control unit (CU).

12.2.1 Input operation

Input operations are those operations that are initiated by the input unit. This unit is made up of several devices which carry out tasks that are input-related. These devices include the keyboard, mouse, scanner, card reader or microphone. Users use them to give instructions to the computer for processing. The input unit creates an avenue for a user to give instructions to the computer; it is a link between the user and the computer.

Examples of some devices that carry out input operations are shown in Figure. 1.2



Figure 12.2: Input Devices

12.2.2 Processing Operation

Processing operations are carried out by the Central Processing Unit (CPU). As soon as the computer receives an input, it stores it in memory from where it can be accessed by the CPU for processing. The CPU is regarded as the brain of the computer, and this is because it does all the data processing and decision-making in

the computer. There are some components that do the work in the CPU; these include the Arithmetic and Logical Unit (ALU), the Control Unit and the Bank of Registers.

12.2.3 Output operation

Output operations are those operations that are carried out by the output unit of the computer. As soon as the data that were inputted into the computer has been processed by the CPU, the CPU directs the results that have been gotten to the output unit to be received by the user as information. The output unit is a collection of devices that gives information about what has been processed to the user. Some output devices include a headset, projector, speakers, monitor, printers, plotters, etc.

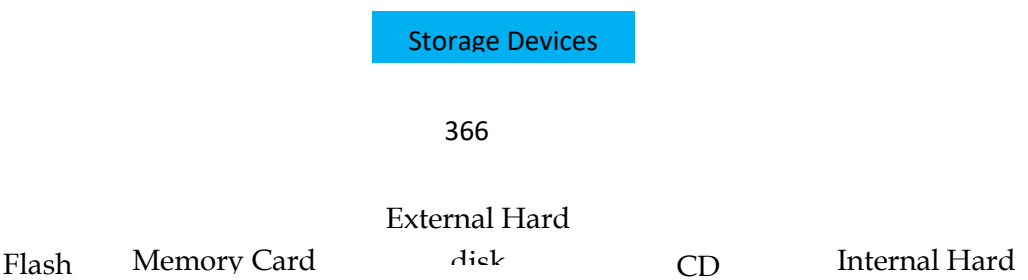


Figure 12.3: Output Devices

12.2.4 Storage Operation

Storage operations are operations that are carried out by the computer after the input, processing and output operations are completed. It allows a computer to keep its data permanently or temporarily. The devices used to carry out storage operations are known as storage devices; these storage devices allow users to keep all kinds of information like videos, audio files, documents, pictures and raw data for future use.

Some devices that are used for storage include:



Flash Memory Card External Hard disk CD Internal Hard



Figure 12.4: Storage Devices

12.2.5 Communication Operations

Communication operations are those operations that enable two or more devices to interact with each other. This interaction includes the transfer of data, instructions, and information. Some communication operations involve cables and wires, while others are done through wireless means. A communications system is made up of computers and computing devices. For communication to take place successfully, the following components are needed:

i) The sender – this is the device that initiates the communication by sending a request to transmit data, instructions, or information. E.g. PC, Camera, Handset or Phone, which is also known as the source.

ii) The message – is the data/ information to be sent or transmitted, e.g. Pictures, Voice notes, Video clips, e.t.c

iii) The receiver – is the recipient of the message, e.g. Handsets, Phones, PCs, tablets, Cameras, e.t.c

iv) A communications channel/ device- this is the device that links the sending device to the receiving device via a communications channel (WiFi or LAN).

v) The protocol – Is the set of rules that govern the communication between the sender and the recipient

12.3 Types of Computing Devices

Existing computer systems come in a wide variety. This section lists a few of the more popular varieties that are currently offered.

12.3.1 Supercomputer

A supercomputer refers to the most powerful computer that is currently available. These devices are specifically designed to process massive amounts of information at a high speed. Supercomputers are particularly useful for scientific research, most especially in the field of nuclear research, where researchers need to accurately predict what will occur during each millisecond of a nuclear chain reaction. These devices are designed to handle tasks that involve processing vast amounts of data. As an example of their power, a micro-computer would take 45 hours to complete a simulation program for an air pollution management project with over 500,000 variables, whereas a supercomputer would only need 30 minutes to accomplish the same task.

12.3.2 Mainframe

Mainframe computers are currently the largest type of computers in common use. They are specifically designed to handle enormous amounts of input, output, and storage. These computers are typically used by large organizations such as power companies, telecommunication companies, and financial institutions. Users typically access mainframe computers through terminals, which consist of a keyboard and a video display or monitor. The mainframe computer is usually located in a dedicated computer room. Major manufacturers of mainframe computers include IBM, Burroughs, and Univac.

12.3.3 Mid-range

A midrange computer falls in between a general-purpose personal computer and a full-size mainframe computer in terms of power and capability. It is more efficient than a personal computer but less powerful than a mainframe computer. The term "midrange computers" is used in a general sense to refer to a computer that falls somewhere in between a personal computer and a mainframe computer in terms of processing power, storage capacity, and other capabilities. When there are few to a few dozen client systems, a midrange computer is deployed as a network server.

Several CPUs, a sizable amount of RAM, and sizable hard drives are precise components of such computers. Additionally, they frequently have connectors for connecting to more business peripherals, such as big data storage systems and technology that supports complex networking. The use of midrange computers, however, is not just restricted to corporations; some individuals create or buy intermediate systems when high-end desktop computers are inadequate, as would be the case with sophisticated real-time picture processing or online gaming.

12.3.4 Personal Computer (PC)

Desktop personal computers are those that are used on a desk and are referred to as Personal Computers (PC). There are a lot of varieties available in the personal computer categories, which are discussed next.

12.3.4.1 Desk and Tower Units

The first and most prevalent type of PC is Desktop and Tower units. Most desktops are just large enough to be carried around; they are yet tiny enough just to fit onto a desk.

12.3.4.2 Laptops

Laptops are small type computers that have been made available. Since the keyboard is mounted flat on the system unit, the screen is utilized to conceal the system. These can be used in automobiles while travelling and, when closed after use, can be carried around like a box.

12.3.5 Pocket PCs

A pocket PC is a mobile device or a portable computer which can be held in the hand or fits in a pocket. They come in various sizes and shapes, ranging from small smartphones to larger tablet computers. These devices are designed to be highly portable and allow users to perform many of the same tasks as traditional desktop or laptop computers. Examples of mobile devices include smartphones, e-readers, and tablets.

12.3.5.1 Electronic Organizers

Organizer is an electronic gadget that users use to manage their schedule and save data like names, addresses, telephone numbers, and meeting dates.

12.3.5.2 Palmtop Computers

The palmtop computer is significantly more compact. Despite their smaller size, mobile devices such as smartphones, tablets, and e-readers contain many of the same components as desktop and laptop computers. These components may include a processor, memory, storage, a display, and various input/output devices such as cameras, microphones, and speakers. However, due to their smaller size, these components are designed to be more compact and energy-efficient.

12.3.5.3 Pen Computers

Pen computers are input tool that enables users of pen-based computers, including personal digital assistants, to write on it or point at a specific pad on the screen (PDAS).

12.3.5.4 Personal Digital Assistant (PDA)

Personal Digital Assistants (PDA) are compact handheld computers with touch screens that are operated with a stylus. A virtual keyboard that displays on the touchpad is used to enter data (such as text). PDAs were initially used as personal organizers, but their usage has now expanded to include features such as mobile phone capabilities, internet connectivity, GPS navigation, multimedia support, and more. Additionally, the concept of PDAs has evolved to become smartphones, which are much more advanced and powerful handheld devices with similar functionalities. Today, many PDAs come equipped with simple database, word processing, and spreadsheet features.

12.3.7 Micro-Computers (Workstations)

The terms "microcomputers" and "personal computers" are often used interchangeably to refer to small computers that are designed for individual use. These types of computers are found in homes, offices, and classrooms and are less expensive and less powerful than mainframe computers or supercomputers. Microcomputers or personal computers are capable of running a variety of software programs for tasks such as word processing, spreadsheet management, and internet browsing

The employment of microcomputers in watches, clocks, and cameras is a common practice. In several fields today, PCs pose a major threat to mainframes and minicomputers. Currently, personal computers (PCs) are significantly more capable than mainframes that existed only a few years ago, and the industry is continuously introducing smaller and faster models every year because of market competition.

12.4 Other Types of Computing

12.4.1 Cloud Computing

Cloud computing is a technology that is made up of a network of remote servers that are hosted on the Internet to manage data instead of making use of a local server or a personal computer. Some common examples of cloud computing technologies that are used today include Google Drive, Dropbox, iCloud, Amazon S3, Microsoft One Drive, etc. These cloud computing services are available on the Internet and enable users to transfer very large files and store information. They give users access to their data whenever they need it and from wherever they are at that particular time for as long as they have access to internet services.

12.4.2 Grid Computing

Grid computing is a technology that uses a combination of several computer resources that are in diverse geographical locations in order to accomplish a particular assignment. The types of assignments that require Grid computing are those that are usually very difficult for one machine to handle alone. Therefore, many machines on a given network are put together under a given protocol to operate as a single virtual supercomputer in order to achieve these difficult tasks. This is achieved by having all unused resources on multiple computers pooled

together and made available for that single task. Some areas where grid computing is applied include:

12.4.2.1 Scientific Research

Grid computing is greatly applied in scientific research to simulate the behaviour of a nuclear explosion. It can also be used to model the human genome or to analyze massive amounts of data generated from particle accelerators.

12.4.2.2 High-performance computing

Grid computing can be used to solve difficult and complex scientific problems.

12.4.2.3 Data Analytics and Business Intelligence

Grid computing can be used in the business world to process large volumes of data, and this can be done in real-time. For instance, a company can use grid computing to analyze the sales that have been made, customer behaviour, or market trends in order to make good business decisions.

12.4.2.4 Healthcare

Grid computing can be applied in the healthcare sector to store and analyze huge amounts of patient data and information. This can be used to create specific medication for a particular category of patients, improve medical research and also help with disease outbreak detection and management.

12.4.2.5 Weather Forecasting

In forecasting, Grid computing can be used to process the huge amounts of data that are usually generated by weather sensors and satellites in real-time to make more accurate weather predictions.

Grid computing can also be applied in the media and entertainment industry to process and render huge amounts of digital media, such as movies, games, and special effects.

12.4.2.6 Government and Defense

In government and defence, grid computing can be applied by government agencies and defence organizations to process and analyze huge amounts of data, such as satellite images, to support decision-making and operations.

12.4.3 Distributed Computing

Distributed computing is a system whereby software components are divided among several computer devices located in different places. Although the software components may be spread across several computers in different locations, they are usually run as one system. This is usually done to improve efficiency and performance. Examples of distributed computing include telephone and cellular networks

12.4.4 Cluster Computing

Cluster computing can be described as a technology that enables several computers to be connected on a network and operate like an individual entity. Each connected device is known as a node. Cluster computing can be applied in solving very complex assignments by creating better speed of computation and increased data integrity. Some example applications of cluster computing can be seen in web applications in the area of Database and web server management, Search Engine, Security and email. It can also be implemented to forecast the occurrence of earthquakes or tornadoes.

12.4.5 Personal Computing

Personal computing involves the use of a computer system that is stationed where the input or output of data is done. Personal computing can be applied in accounting, desktop publishing, spreadsheet and database management and word-processing applications. Individuals can also use it for listening to music, playing games and surfing the Internet.

12.4.6 Time-Sharing Computing

This is a computing method whereby many users with diverse programs interact almost at the same time with the central processing unit (CPU) of a large computer.

Some examples are seen in Multics, UNIX, Windows NT server, Linux, TOPS-20 (DEC) and Windows 2000 server.

12.4.7 Client-Server Computing

This computing method works with requests from clients and responses from the server with the needed information. The server and the client will need to communicate with a particular protocol to enable free interaction. Note that a server will be able to take only a few number of requests from clients at a time. Therefore, it makes use of a system-based technique to prioritise requests.

Summary

i. Computing is defined as any purposeful activity that requires, benefits from, or results in the use of computers.
ii. The computing system is comprised of three key components: the user, the computer system, and the context in which the computer is utilised.
iii. The hardware and software make up the computer system.
iv. Two primary classifications of software exist: the system software and the application software.
v. Computing operations are carried out by the input, storage and output units as well as the central processing unit (CPU), which houses the arithmetic logic unit (ALU) together with the control unit (CU).
vi. Various types of computing exist, and several types of computing devices exist.
vii. Some examples of computing devices include Supercomputers, Mainframe, Midrange, and Personal Computers.
viii. some pocket PCs include Electronic Organizers, Palmtops and Pen computers
ix. Some other types of computing include Cloud Computing, Grid Computing, Distributed Computing, Cluster Computing, Personal Computing, Time-Sharing Computing, Client-Server Computing

Exercises

1. What is a computer?

2. Mention the types of computers that exist.

3. Mention and explain briefly the generations of computer

4. Give a definition to the following:

   i. System software

   ii. Application software

   iii. Hardware

   iv. RAM

   v. Analog computer

5. Mention the 5 basic operations in computing

6. Explain any 3 of the computing operations

7. Define the following terms:

   i. Cloud Computing

   ii. Personal computing

   iii. Distributed Computing

   iv. Client Server Computing

   v. Cluster Computing

8. What is grid computing?

9. List 5 areas of its application.

CHAPTER THIRTEEN
SIX STEPS OF PROBLEM SOLVING AND METHODS OF SOLVING COMPUTING PROBLEMS

Overview

Irrespective of the area of study, computing science is all about solving real-world problems or perhaps even problems from the abstract world with computers. We need to have a standard systematic approach to solving problems. This chapter provides sufficient knowledge about Steps of Problem Solving and Methods of Solving Computing Problems. At the end of this chapter are test items designed to stimulate learning and possibly redirect the attention of learners to grey areas for proper understanding.

Objectives

By the end of this chapter, students should be able to:

1. list and explain the six steps of problem solving
2. understand the methods of solving computing problems.
3. apply the problem-solving paradigm to routine elementary problems.

13.1 Steps in Problem Solving

Computer-based problem solving is a systematic process of designing, implementing and using programming tools during the problem-solving stage. This approach makes the computer system to be more intuitive with human logic than machine logic. The final output of this process is software tools which are dedicated to solving the problem under consideration. Software is just a collection of computer programs, and programs are a set of instructions which guide a computer's hardware. These instructions need to be well specified to solve the problem. After its creation, the

software should be error-free and well-documented. Software development is the process of creating software which satisfies the end user's requirements and needs.

To solve a problem using a computer, the following six steps must be followed:

13.1.1 Identify the Problem

The first step to solving any computer-based problem is to identify (define) the problem that needs to be solved. In order to recognise that a problem exists, you must have some "knowledge" of the situation. Once the right problem has been identified, the next step is understanding the problem.

13.1.2 Understand the Problem

The second step to solving any problem is to make sure that you understand the problem that you are trying to solve. Solving a problem requires a clear understanding of the problem. Understanding the problem is the process of analysing a problem and decomposing an overall system into smaller parts to identify possible inputs, processes and outputs associated with the problem. This leads to the next step.

13.1.3 Identify Alternative Ways to the Problem

The next step after one has understood the problem is to identify possible solutions to the problem. There is always more than one approach to solving a problem, and in some cases, simultaneous solutions may be required. As with the previous steps, time must be taken to develop plenty of innovative and creative ideas. At the end of this step, you can be sure you will get the best solution if you have tried all possible solutions and generated every conceivable option.

13.1.4 Select the Best Way to Solve the Problem from the List of Alternatives

With a list of possible solutions obtained in the last step (step 3), it is time to select the best individual or best combination of solutions to be put into action and to eliminate the problem at hand. The process of choosing the best solution is a matter of ranking all the possible solutions against one another and defining each option as "pluses and minuses." Criteria for the selection of the best individual or best combination of solutions are

1. The next step is to go through and eliminate less desirable or unreasonable solutions.
2. Order the remaining solutions in order of preference.
3. Evaluate the remaining solutions in terms of their advantages and disadvantages.

13.1.5 List Instructions that enable you to Solve the Problem using Selected Solution

In this step, you will carefully outline the instructions that enable you to solve the problem using the selected solution in step 5. You are to list the action plan to implement the selected solution and then implement the solution.

13.1.6 Evaluate the Solution

After the implementation of the solution, the next step is to evaluate how effective the solution is. If you are not pleased with the outcome, return to your alternative solutions to select a new solution or revise the existing solution and repeat the remaining steps.

13.2 Methods of Solving Problems

Solving computer problems requires the adoption of an approach, method or strategy. The approach chosen will determine the process involved in solving the problem and the resources required to achieve the solution. The choice of a problem-solving strategy depends on the nature of the problem, the knowledge and skill of the analyst and the resources available for solving the problem. Computer scientists use two main methods for solving problems. These are algorithms and heuristics.

13.2.1 Algorithm

An algorithm is a sequence of steps for solving a well-defined problem. It is also a step-by-step procedure used to solve a clear, properly outlined and precise problem. An algorithm consists of a list of instructions that carry out prescribed and specific instructions in an orderly manner. Mathematical and computing algorithms are usually procedures for solving the same problem repetitively.

There is a widespread application of algorithms in virtually every area of computing and information technology. Algorithms are used to specify data processing tasks and operations of automated systems. Algorithms may be as simple as the one used for sorting numbers or may complex tasks like social media content analysis or automated flight control operations.

An algorithm usually begins with an initial input followed by a sequence of instructions for the operation or computation to be performed. An output, which is usually more data, is produced at the end of the execution of the instructions for the computation process.

There are two commonly used notations for expressing algorithms, namely, natural languages and pseudocode. Natural language representation involves presenting the algorithm using simple English language sentences. This makes the specification easy to understand by people who are not computer science professionals. A major drawback of this method is the ambiguity that may result from instructions represented in natural language and the difficulty in expressing complex instructions in natural language.

13.2.1.1 Purpose of Algorithms

The following are the reasons why computer scientists create algorithms.

1. To Aid Scalability

Algorithms enable application developers to have a full grasp of what scalability entails. Developers use algorithms to break a complex problem into small and manageable chunks, which are better understood and can be analysed in a few steps quickly.

2. To Achieve Performance

Complex real-world problems are difficult to break down. However, if a complex problem can be broken down into smaller chunks consisting of a few steps each, then the problem is realistic and the solution feasible.

3. To Simplify Program Writing

Program codes can be written faster and more easily once an algorithm is developed and analysed to confirm the feasibility of the proposed solution.

13.2.1.2 Characteristics of Algorithms

The following are characteristics of algorithms.

1. Input: An algorithm must have one or more input values. The input may be any value apart from 0.
2. Output: An algorithm should produce one or more outcomes after execution.
3. Unambiguity: A good algorithm should have clear, well-articulated and unambiguous instructions. Each instruction should have only one interpretation and should not produce conflicting results.
4. Finiteness: An algorithm must have a countable instruction. That is, the instructions in an algorithm cannot be unlimited.
5. Effectiveness: An algorithm should contain sufficient instructions needed to perform the desired task. That is, the number of instructions in an algorithm should be adequate to carry out the process for which the algorithm is designed.
6. Language independence: The creation of an algorithm should not be based on any programming language. This means that the sequence of instructions in the algorithm should be implementable in any language and produce the same results irrespective of the language of implementation.

13.2.1.3 Types of Algorithms

Algorithms are designed to perform different tasks. The nature of tasks an algorithm executes determines the category to which the algorithm belongs. Algorithms can be broadly categorised as follows:

1. **Search engine algorithm:** This algorithm searches databases and produces results of relevant web pages using strings of keywords and operators.
2. **Encryption algorithm.** This security algorithm uses a secret key to transform data into unintelligible form, which unauthorised persons cannot read. Authorised persons can access the transformed data and convert it to intelligible form using a key and an opposite (decryption) algorithm. The process is called decryption. An encryption algorithm may be symmetric, such as the Data Encryption Standard, or Asymmetric, such as the Rivest, Shamir and Adelman algorithm. Symmetric algorithms use the same key for encryption and decryption, while asymmetric algorithms use different keys for encryption and decryption of data.
3. **Greedy algorithm:** The purpose of this algorithm is to solve optimisation problems. It works by finding a locally optimal solution with the hope that such a solution can be adopted as a globally optimal solution.
4. **Recursive algorithm:** This algorithm executes the same set of instructions repetitively until a terminating condition is reached. A small value is used to execute the function whenever it is called.
5. **Backtracking algorithm:** This algorithm uses an incremental approach to address a complex problem by solving one slice of the problem at a time.
6. **Divide-and-conquer algorithm:** Solves a problem in two steps. The first step breaks a complex problem into simple subproblems. The second step solves

each of the subproblems and integrates the solutions to form the solution to the original problem.

7. **Dynamic programming algorithm:** Divides a complex problem into subproblems. It solves each of the subproblems and stores the results. These results are applied to similar problems in the future.
8. **Brute-force algorithm:** This algorithm explores all possible ways to solve a problem. The goal is to find one or more suitable solutions to the problem.
9. **Sorting algorithm:** This is used to rearrange the contents of data structures such as a queue or stack. The arrangement is done using a comparison operator, which determines whether the data is stored in ascending or descending order.
10. **Hashing algorithm:** This algorithm takes data of varying sizes as input and produces a message of the same size irrespective of the sizes of individual input.
11. **Randomised algorithm:** This algorithm uses random elements to minimise run times and time-based complexities.

13.2.1.4 Advantages of Algorithms

1. Algorithms provide automated and faster knowledge creation than traditional problem-solving techniques.
2. Algorithms provide faster manipulation of databases quickly and minimise delays caused by technology and humans.
3. Algorithms can minimise transportation issues. They can identify traffic congestion and suggest alternative travel times and routes.
4. Self-driving cars could dramatically reduce the number of road accidents and improve the quality of life.

5. Algorithms can be used for targeted delivery of news, services and advertisement.
6. More evidence-based social science experiments use algorithms to collect data from social media and click trails.
7. Improved and more proactive police work, targeting areas where crime can be prevented.
8. Algorithms minimise the difficulty associated with decision-making, purchasing, transportation and other human endeavours.

Example 2.1

Write an algorithm to calculate the area of a rectangle.

Step 1: Start

Step 2: Input the Length and Breadth

Step 3: Multiply the Length by the Breadth

Step 4: Produce the value of Area as output

Step 5: Stop

Example 2. 2

Write an algorithm to find the greater of two numbers

Step 1: Start

Step 2: Input Num1 and Num2

Step 3: If Num1 > Num2 then

Step 4: Output Num1 is greater than Num2

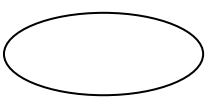
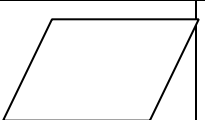
Step 5: Otherwise, output Num2 is greater than Num1

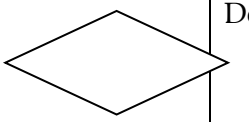
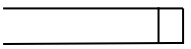
Step 6: Stop

13.2.1.5 Flowchart

A flowchart is a diagrammatic representation of an algorithm. Flowcharts are used to express algorithms in pictorial form. Flowcharts are drawn using a set of basic symbols. Each symbol has a specific meaning and represents a unique operation.

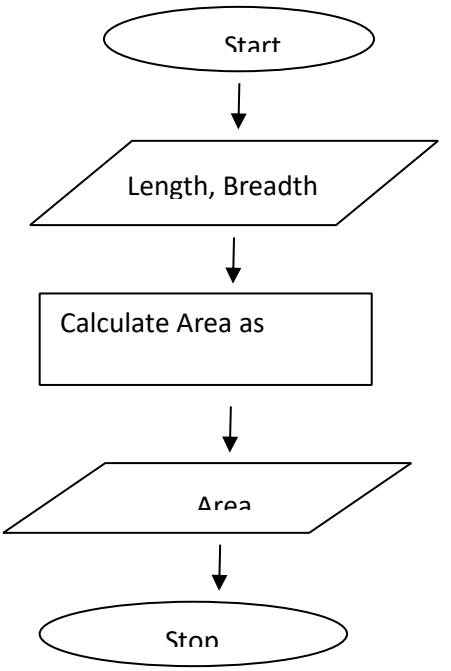
Table 13.1: Basic flowchart symbols, their meaning and purpose.

Symbol	Meaning	Purpose
	Terminator	To indicate start and stop, that is, the beginning and end of a sequence of operations.
	Input and output	Used to represent data input and output or read and

		write operation from and to a file or device.
	Processing	To represent various data manipulation activities. This may be an arithmetic operation, sorting, insertion or deletion.
	Decision	Used to evaluate a condition and make a decision based on the outcome of the evaluation.
	Datastore	Represents a data storage operation or location where data is stored
	Data flow	Represents flow of operation.

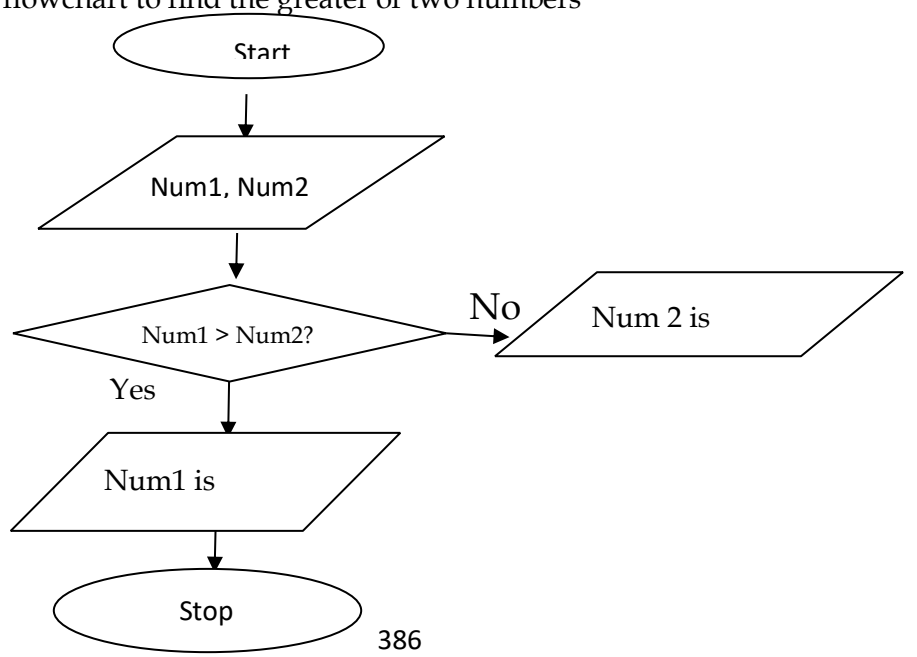
Example 2.3

Draw a flowchart to calculate the area of rectangle.



Example 2.4

Draw a flowchart to find the greater of two numbers



13.2.2 Heuristics

Heuristics are educated guesses or mental shortcuts for solving problems quickly. The results obtained are usually good enough and useful, given time constraints. Investors and financial professionals use a heuristic approach to speed up analysis and investment decisions.

However, heuristics can lead to poor decision-making based on a limited data set, but the speed of decisions can sometimes make up for the disadvantages.

Behavioural economists consider heuristics as one limitation of human beings to behave like rational actors. Availability, anchoring, confirmation bias, and the hot hand fallacy are some examples of heuristics people use in their economic lives.

13.2.2.1 Differences Between Heuristic and Algorithm

An algorithm is a set of instructions used to solve a well-defined problem. They are formalised and can be expressed as a formula. Algorithms are reproducible. That is, an algorithm will always provide the same output given the same input.

A heuristic is an educated guess or gut feeling. Rather than following a set of rules or instructions, a heuristic is a mental shortcut. Moreover, it often produces sub-optimal and even irrational outcomes that may differ even when given the same input.

13.2.2.2 History of Heuristics in Computer Science

Heuristics have played a critical role in the development of theories, technologies and other advances in computer science.

Ancient Greek mathematicians and philosophers used heuristic techniques to solve mathematical and logical problems.

The first and most formal application of heuristics in computer science occurred with the development of artificial intelligence (AI) research in the 1950s and 1960s.

Alan Turing proposed the use of heuristic search to solve difficult problems in AI and computer science (Turing, 1950).

Prior to this, George Pólya's landmark work titled "How to Solve It" in 1945 and republished in 2014 focused on a set of heuristics for solving mathematical problems (Pólya, 2014). This work motivated many computer scientists and researchers to explore heuristics as a problem-solving technique.

Over the years, heuristic techniques have evolved and diversified to address various complex problems in computer science. Some of the most notable developments in the history of heuristics include:

1. **Game Theory**: In the 1950s and 1960s, researchers like Claude Shannon and Arthur Samuel developed early heuristics to explore optimal game strategies like chess and checkers. Their work paved the way for more advanced heuristic techniques used in game theory today.
2. **Optimisation**: In the 1970s, researchers began developing met heuristic optimisation algorithms, such as genetic and simulated annealing, to find near-optimal solutions to complex optimisation problems.
3. **Machine Learning**: The 1980s and 1990s witnessed significant advancements in machine learning techniques, such as decision trees and neural networks, which rely on heuristic methods to learn from data and make predictions.
4. **Human-Computer Interaction**: Heuristic evaluation, a method for identifying usability issues in user interfaces, was introduced by Jakob Nielsen in the 1990s, highlighting the application of heuristics in human-computer interaction.

5. Computer scientists will continue to use heuristic techniques to develop novel methods to solve increasingly complex problems.

13.2.2.3 Relevance of Heuristics to Problem Solving

Heuristics are versatile and adaptable to many areas of computer science. The following are some of the reasons why heuristics are considered an important problem-solving technique.

1. **Efficiency**: Heuristics can significantly reduce the time and resources needed to solve problems, especially when the search space is enormous or the problem is too complex.
2. **Simplicity**: Heuristic methods are often easier to implement and understand than complicated algorithms, making them more accessible to a broader audience.
3. **Flexibility**: Heuristics can be applied to various problem domains, often with minor modifications, making them versatile problem-solving tools.
4. **Robustness**: In situations where the available data is incomplete, uncertain, or noisy, heuristics can still provide helpful solutions, making them applicable to real-world problems.

13.2.2.4 Applications of Heuristics in Software Engineering

Heuristics are widely used by application developers and software engineers to create highly efficient, maintainable, reusable and user-friendly software. The following are specific applications of heuristics to software engineering.

13.2.2.5 Design Patterns

Design patterns are reusable solutions to common software design problems. They are heuristics that software developers can apply when addressing specific design challenges.

Developers use these well-established patterns to create more maintainable and scalable software while avoiding common pitfalls.

Some widely used design patterns include:

1. **Singleton**: Assures a class has just one instance and offers a universal access point to that instance.
2. **Factory Method**: Establishes a framework for instantiating objects in a superclass, allowing subclasses to select the objects they want to use.
3. **Observer**: Establishes a one-to-many relationship between objects so that all of its dependents are automatically informed and updated when one changes.

13.2.2.6 Agile Development

Scrum and Kanban are popular Agile development methodologies used for the efficient management of software projects. These methodologies apply heuristics to streamline development, improve collaboration, and deliver high-quality software.

Some critical heuristics in agile development include:

1. **Iterative Development**: Agile methodologies promote short development cycles, allowing teams to gather feedback continuously, identify issues, and adjust their plans accordingly.
2. **Time-boxing**: Setting strict time limits for tasks helps teams maintain focus and prevents them from getting bogged down in unnecessary details.
3. **Daily Stand-ups**: Regular short meetings enable team members to share updates, identify roadblocks, and ensure everyone is on the same page.
4. **Prioritisation**: Agile methodologies prioritise tasks based on their value to the end user, ensuring that the most critical features are delivered first.

13.2.2.7 Code Smells and Refactoring

Code smells are signs of potential problems in the source code, which are caused by poor design approaches and bad programming practices. Heuristics can identify code smells and guide developers to refactor (rewrite and reorganise) their code to make it more maintainable, readable, and of good quality.

Some examples of code smells and corresponding refactoring techniques include:

1. **Long Method**: A method that is too long and difficult to understand can be broken down into smaller, more manageable processes.
2. **Duplicate Code**: Identical or similar code in multiple locations can be consolidated into a single reusable method or class.
3. **Feature Envy**: When a method accesses data from another class more than its data, moving the technique to that class may be more appropriate.
4. Software engineers can produce more effective, maintainable, and user-friendly software that satisfies the requirements of their customers and end users by utilising heuristics in the development process.

Summary

In this chapter, you learnt the various steps involved in problem-solving processing. The steps are sequential and are six in number: identify the problem, understand the problem, identify alternative ways to the problem, select the best way to solve the problem from the list of alternatives, list instructions that enable you to solve the problem using the selected solution and evaluate the solution. The chapter is rounded up with a sufficient explanation of the methods of solving computing problems.

Exercises

1. What are the 6 steps to problem-solving in computer?
2. Outline various steps that will be needed to get a problem solved

3. State a reason why each of the problem-solving steps is important in developing the best solution for a problem
4. Define an algorithm
5. Do algorithms serve any useful purpose? Justify your answer.
6. List and explain any 5 types of algorithms.
7. Write an algorithm to determine whether the area of a spherical shape is greater than its volume.
8. Write an algorithm to find the largest among any 3 input values.
9. Write an algorithm to rearrange a set of values in (a) ascending order and descending order.

10. Define heuristics.
11. Does heuristics always produce a perfect solution? Justify your answer.
12. Identify and explain the relevance of heuristics to problem-solving in computer science.
13. What are the areas of applications of heuristics to computer science problems?

CHAPTER FOURTEEN
PROGRAMMING TOOLS FOR PROBLEM-SOLVING

Overview

This chapter discusses the programming tools used for problem-solving in computer science and provides adequate knowledge on how to use them. These tools include the flowchart, pseudocode, decision table, decision tree, algorithms, and data flow diagrams. In the discussion, students will understand the importance of these programming tools in solving problems during programming.

A flowchart is a diagram that showcases a workflow, algorithm, or process used in designing, analysing, documenting, or handling a process or program in various fields. Pseudocodes are simpler to develop and debug when compared to algorithms. Algorithms are commands that aid computers in executing calculations and some other problem-solving functions. It is not the entire program or code; it is a simple logic to a problem represented as an informal description in the form of a flowchart or pseudocode. Data flow diagram (DFD) pictorially exhibits data flow via a sub-system or system. A decision tree is a decision-support hierarchical model that uses a tree-like model of decisions and their possible consequences. Decision tables are a concise visual illustration for showcasing which actions are to be performed depending on predetermined conditions. They are algorithms whose output is a set of actions. Data flow diagrams visually represent systems and processes that would be hard to describe in words. At the end of the chapter, the students will be able to solve problems using all the programming tools.

Objectives

This chapter focuses on providing students with basic knowledge of programming tools for problem solving. Specifically, students should be able to:

i) understand the basic concepts of flowcharts,

ii) apply basic symbols and rules to create flowcharts,

iii) differentiate among the common types of flowcharts and where they apply,

iv) undertake solving simple flowcharting problems,

v) demonstrate basic skills in writing pseudocode for simple problems,

vi) write algorithms to solve problems,

vii) learn how to create data flow diagrams and decision trees, and

viii) represent systems and processes while solving problems using data flow diagrams.

14.1 Flowcharting

Flowcharting is a way of illustrating data flows and workflows in a system using schematics, diagrams, and symbols. It is a well-organised way of illustrating a flowchart for algorithms. It is an indispensable tool that assists system analysts in problem solving. It clearly visually shows the components of a system. A flowchart is a diagram that illustrates the steps, sequences, and decisions of a process or workflow. It is a powerful tool that can be used in multiple fields for planning, visualising, documenting, and improving processes. The industrial engineers Frank and Lillian Gilbreth first introduced this tool to the American Society of Mechanical Engineers (ASME) in 1921. Since then, flowcharts have been refined and standardised to optimise processes across various industries.

A flowchart is basically a pictorial or diagrammatic representation of an algorithm using standard symbols. **A flowchart** is a graphical representation that explains the sequence of operations to be performed to solve a problem under consideration. Various **standard symbols** are used to express different operations during flowcharting. All symbols are connected among themselves to show the flow of information and processing.

Flowcharts showcase algorithms graphically. Programmers usually use them for program planning and problem solving. It uses symbols connected among them to show the flow of information as well as processing.

Flowcharts can be helpful in a variety of scenarios, including when you need to document a process, visualise complex ideas or processes, organise your team and assign tasks, make and justify decisions, identify and prevent bottlenecks, standardise processes, and track the progress of a project.

Some guidelines and best practices in flowcharting that are very important are: all requirements should be listed out in a logical order; the flowchart should be clear, neat and easy to follow; the usual direction of the flow of a procedure or system is from left to right or top to bottom; only one flow line should come out from a process symbol while only one flow line should enter a decision symbol; write within standard symbols briefly; use connector symbols to minimise the number of flow lines if you are working on a complex flowchart.

A flowchart is a graphical representation of an algorithm, and it should follow some rules while creating a flowchart. Rule 1 is that the flowchart opening statement must be the "**start**" keyword. Rule 2, flowchart ending statement must be "**end**" keyword. Rule 3: All symbols in the flowchart must be connected with an arrow line. Rule 4: The decision symbol in the flowchart is associated with the arrow line.

There are four basic categories of flowcharts:

i) Document flowcharts show the flow of documents from one business unit to another.

ii) A data flowchart allows you to see the overall data flow in a system.

iii) Program flowcharts show the program's control in a system. They are also one of the essential tools in programming.

iv) System flowcharts are the diagram type that shows you the flow of data and how decisions can affect the events surrounding it.

14.1.1 System Flowchart

A **system flowchart** exhibits physical control of the path taken by data in a system as well as the decisions made at different levels. Some symbols are put together to show data flow by exhibiting what happens to data and its' trajectory. Flowcharts

indicate data flow while equally showcasing how decisions can control events. It manifests data flow via major components like programs, storage media, data entry and processes.

A system flowchart is mainly used for the physical modelling of the system, showing inputs, outputs, and processes. Analysts frequently use them to illustrate the whole system graphically. There are multiple symbols used in a system flowchart. All symbols are unique and represent a different process, input, output, or media item. A system flowchart begins and ends with an oval symbol. This is also called the **terminator** and indicates the start and end of the processes mentioned.

System flowcharts are helpful for showing data pathways for programming processes, but they can also be used for some other processes. One of the primary uses of system flowcharts is in management processes. Processes like inventory control and supply chain management require assistance from such flowcharts to understand the movement of goods from one point to another.

Like other types of flowcharts, system flowcharts consist of start/end terminals, processes, and decisions, all connected by arrows showing the flow and how data moves in the flow.

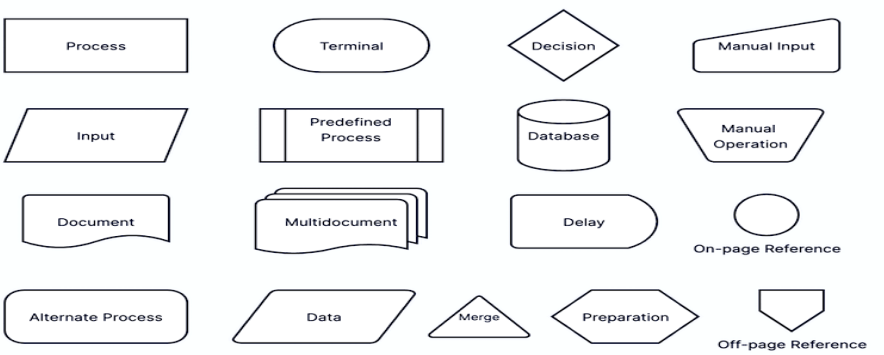


Figure 14.1: System flowchart symbols

Example 3.1: A Car's Cruise Control

Let us look at a car's cruise control. Cruise control allows the car to stay at the desired speed that the driver sets and works by adding or cutting the fuel according to the car's speed. To keep the car at a constant speed, a major component called a speed sensor that records the car's speed will be there. If the car is too slow, it speeds up by adding more fuel, while if the car is too fast, it can be slowed down by cutting some fuel.

A simple flowchart for this system is shown in Figure 3.2.

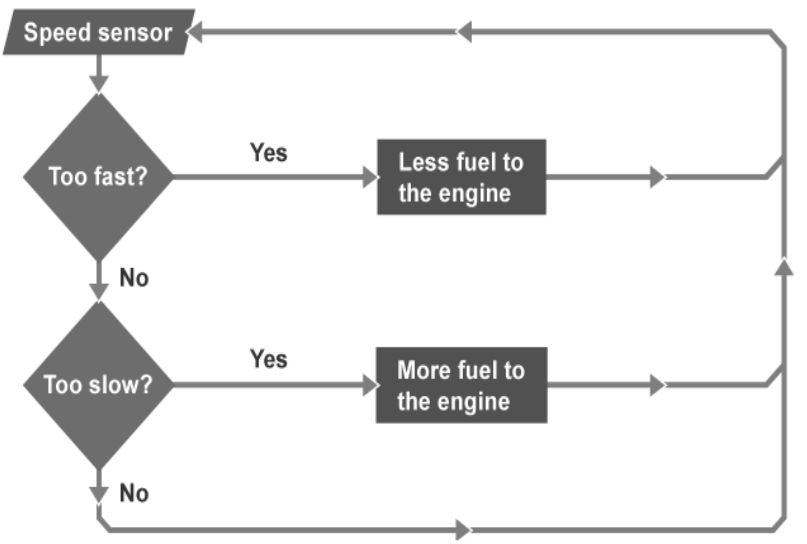


Figure 14.2: Car cruise control system flowchart

Example 3.2: Library Management System

In a digital library management system, it is easier for librarians to manage borrowed books by allowing people to borrow and return books. The system starts with the user logging into their account. The system will then verify the user, allowing them

to access all the library's services, such as changing passwords, borrowing, and returning if they pass all the verification steps.

The library management system's basic flowchart is shown in Figure 3.3.

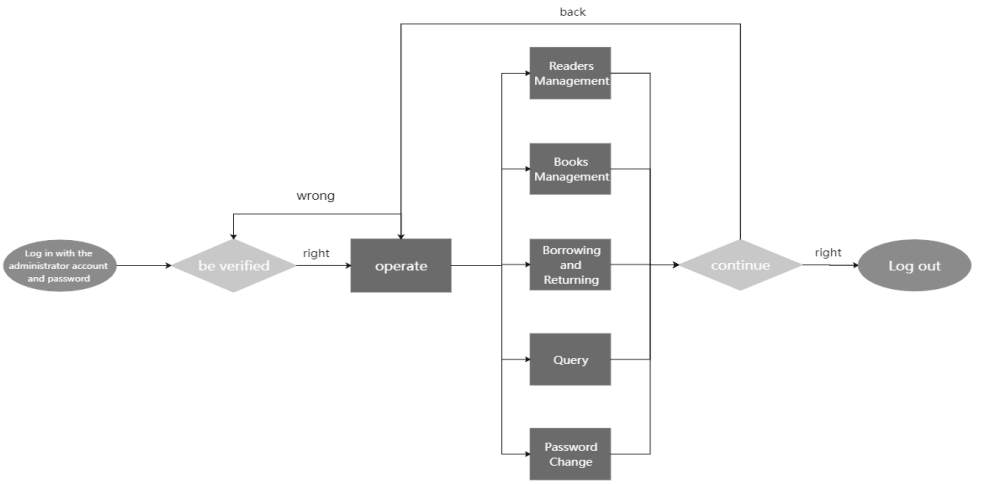


Figure 14.3: Library management system basic flowchart

Example 3.3: Profit/Loss Calculating System

In business, if the production cost is higher than the income, it is a loss. If the income is higher than the cost, it means you have a profit. The system is simple. It will compare the cost and the income to determine your profit or loss. The system flowchart of this system is depicted in Figure 3.4.

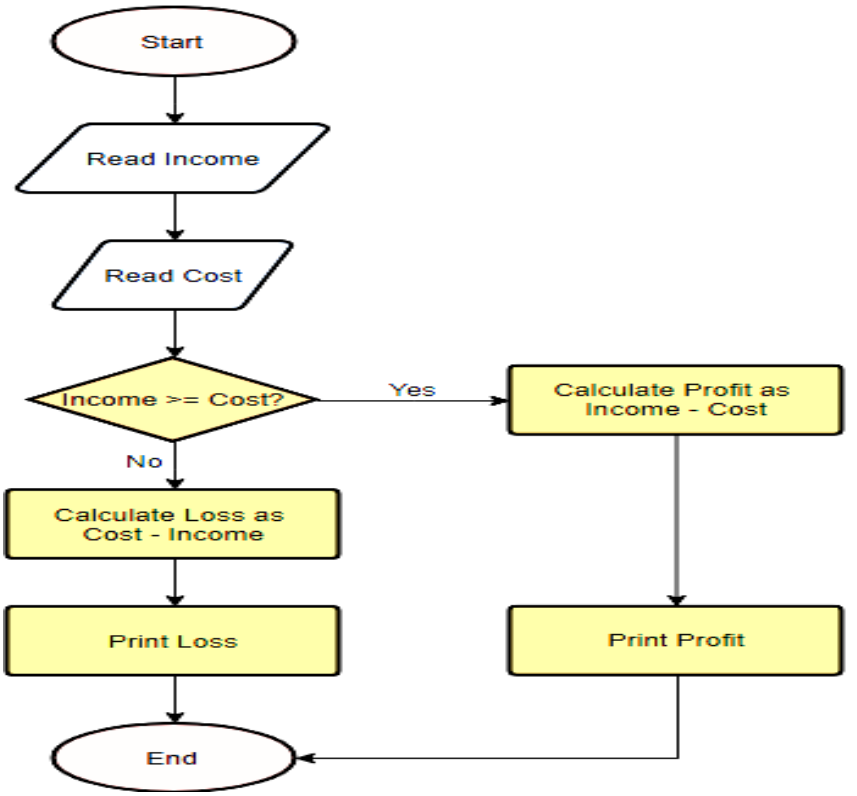


Figure 14.4**: Profit/Loss Calculating System**

14.1.2 Program Flowchart

A program flowchart shows control over a given program in a given system. It is an important tool for coding. It shows how data flows while writing an algorithm.

To express different operations in the flowchart, various standard symbols are used. All symbols are connected among themselves to show the flow of information and processing. Different symbols, as prescribed by the American National Standard Institute (ANSI), which are frequently required while drawing flowcharts, are shown in Figure 14.5.

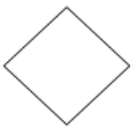
Flowchart Symbol	Symbol Name	Description
	Terminal (Start or Stop)	Terminals (Oval shapes) are used to represent start and stop of the flowchart.
	Flow Lines or Arrow	Flow lines are used to connect symbols used in flowchart and indicate direction of flow.
	Input / Output	Parallelograms are used to read input data and output or display information
	Process	Rectangles are generally used to represent process. For example, Arithmetic operations, Data movement etc.
	Decision	Diamond shapes are generally used to check any condition or take decision for which there are two answers, they are, yes (true) or no (false).
	Connector	It is used connect or join flow lines.
	Annotation	It is used to provide additional information about another flowchart symbol in the form of comments or remarks.

Figure 14.5: Program Flowchart Symbols



On-page and off-page

Some guidelines or points to remember while developing flowcharts are:

i) Flowchart can have only one start and one stop symbol,

ii) On-page connectors are referenced using numbers,

iii) Off-page connectors are referenced using alphabets,

iv) General flow of processes is top to bottom or left to right,

v) Arrows should not cross each other.

Example 3.4: Draw a flowchart to input two numbers from the user and display the larger of two numbers

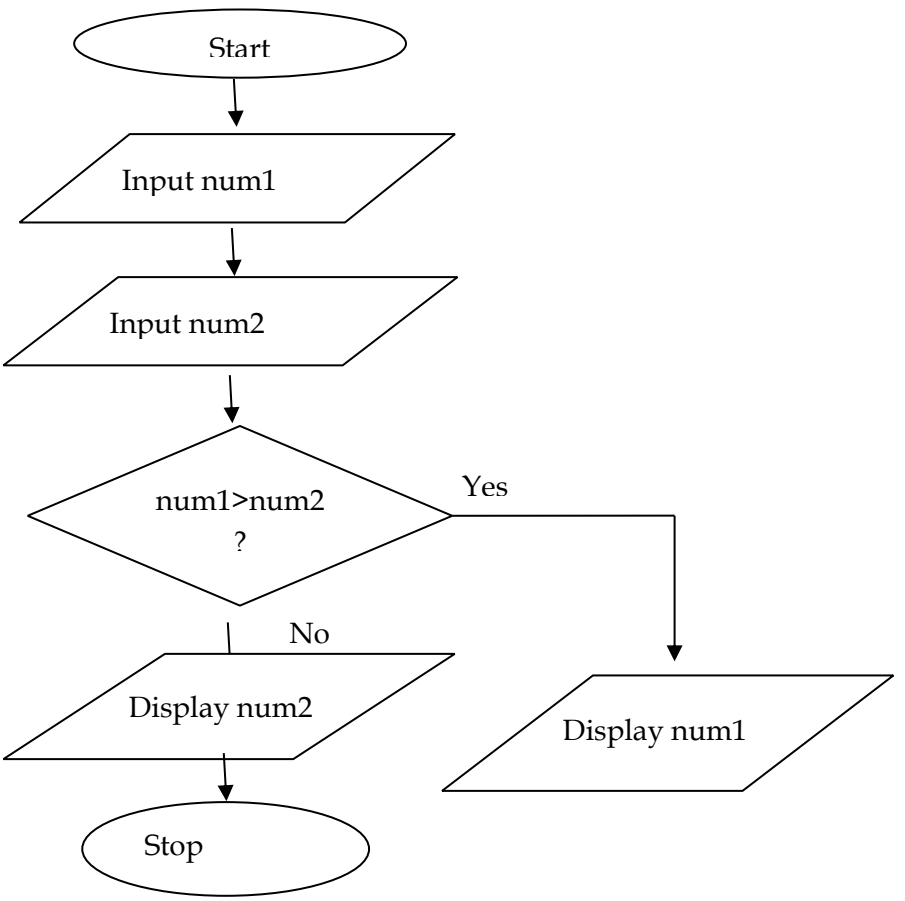


Figure 14.6: Flowchart to input two numbers and display the larger of two numbers

Example 3.5: Draw a flowchart to calculate the average of two numbers.

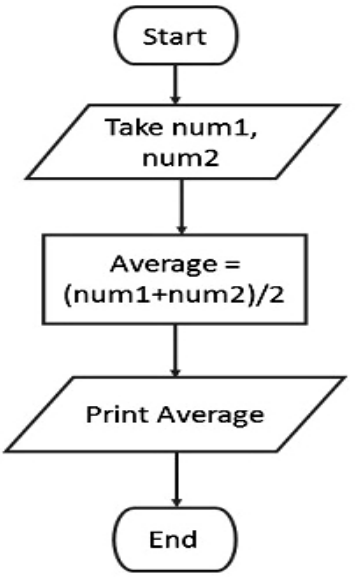


Figure 14.7: Flowchart to calculate the average of two numbers

Example 3.6: Draw a flowchart to calculate simple interest

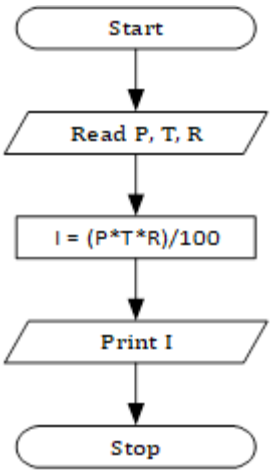


Figure 14.8: Flowchart to calculate simple interest

14.1.2.1 Advantages of Program Flowchart

Drawing a flowchart while solving any problem has the following advantages:

1. Flowcharts are a better way of communicating the logic of the system.
2. Flowcharts act as a guide for blueprints during program design.
3. Flowcharts help in the debugging and testing of processes.
4. With the help of flowcharts, programs can be easily analysed.
5. It provides better documentation.
6. Flowcharts are very useful during the program development phase (coding).
7. Flowcharts serve as a good program documentation tool needed for various purposes.
8. Maintenance of operating programs becomes easy with the help of flowcharts.
9. Easy to trace errors in the software.

10. Easy to understand.
11. The flowchart can be reused for inconvenience in the future.
12. It helps to provide correct logic.

14.1.2.2 Disadvantages of Program Flowchart

The disadvantages are:

i. It is difficult to draw flowcharts for large and complex programs. For complicated logic, flowchart becomes complex and clumsy.
ii. There is no standard to determine the amount of detail.
iii. It is difficult to reproduce the flowcharts.
iv. It is very difficult to modify the flowchart. (If change is required in the logic, then the flowchart needs to be redrawn and requires a lot of time).
v. Making a flowchart is costly.
vi. Some developers think that it is a waste of time.
vii. It makes software processes slow.
viii. If changes are made to the software, then the flowchart must be redrawn, and this requires a lot of time.

14.2 Pseudocode

Pseudocodes constitute the notion of the basic operating principles, workflows and data flows of the program that are similar to the programming language used. A **pseudocode** is a step-by-step description of an algorithm. It does not use any programming language in its representation. Rather, it makes use of simple English language text as it is geared towards human understanding rather than machine reading. Pseudocode is the **intermediate state between an idea and its implementation (code)** in a high-level language.

Pseudocode is an informal method of developing an algorithm. Thus, computer programmers use simple informal language to write a pseudocode. It does not have

any specific syntax that could be followed. The pseudocode is a text-based design tool. Basically, pseudocode represents an algorithm to solve a problem in natural language and mathematical notations. It is written in plain English and in short phrases to represent the functionalities that the specific lines of code would do. Since there is no strict syntax to follow in pseudocode writing, they are relatively difficult to debug.

Pseudocode is an important part of designing an algorithm; it helps the programmer in planning the solution to the problem as well as the reader in understanding the approach to the problem. It is an intermediate state between algorithm and program that supports the transition of the algorithm into the program.



Figure 14.9: Pseudocode is an intermediate state between algorithm and program

Reasons for using pseudocode are for better readability, ease of code construction, good middle point between flowchart and code, acts as a starting point for documentation, easier bug detection and fixing, and programming constructs always written in uppercase such as SEQUENCE, CASE, WHILE, REPEAT-UNTIL, FOR, and IF-THEN-ELSE. These constructs, otherwise called keywords, are used to describe the control flow of the algorithm.

The five important rules that can guide a student while writing the pseudocode are:

1. Write **one** statement per line.
2. Initial keywords should be represented in **a capital case** (**READ**, **WRITE**, **IF**, **WHILE**, **UNTIL**).
3. The indentation of pseudocode should be similar to the actual program to show hierarchy.

4. Ending the multiline structure is necessary.
5. Keep statements in simple language (English).

Before writing the pseudocode of any algorithm, the following points must be kept in mind:

a) Organise the sequence of tasks and write the pseudocode accordingly.

b) Establish the main goal or the aim first.

c) Use standard programming structures such as **if-else**, **for**, **while**, and **cases** the way we use them in programming. Indent the statements if-else, for, while loops as they are indented in a program; it helps to comprehend the decision control and execution mechanism. It primarily also improves readability.

d) Use appropriate naming conventions.

e) Reserved commands or keywords must be represented in **capital letters. Example:** if you are writing IF…ELSE statements, then make sure IF and ELSE are in capital letters.

f) Check whether all the sections of a pseudocode are complete, finite, and clear to understand and comprehend. Also, explain everything that is going to happen in the actual code.

g) Do not write the pseudocode in a programming language. The pseudocode must be simple and easy to understand by minimising the use of technical terms.

We can proceed to express the same logic applied in solving the flowchart problems from the preceding examples as pseudocode.

Example 3.7: Write a pseudocode to input two numbers from the user and display the numbers.

Step 1: START

Step 2: READ NUM1

Step 3: READ NUM2

Step 4: IF NUM1 > NUM2

Step 5: DISPLAY NUM1

Step 6: ELSE

Step 7: DISPLAY NUM2

Step 8: ENDIF

Step 9: STOP

In pseudocode, the term "**READ**" is commonly used to represent the action of obtaining input, whether from a user or some other source. The source of input can vary, and it's not necessarily limited to user input.

It is important to note that Pseudocodes do not follow strict rules, and different sources may use slightly different conventions. However, here is a list of the most common keywords and their associated meanings in pseudocodes:

- **START/BEGIN:** Indicates the beginning of the program or a specific block of code.
- **END/STOP:** Marks the end of the program or a specific block of code.
- **READ/INPUT:** Represents the action of obtaining input, either from a user or some other source.
- **DISPLAY/PRINT/OUTPUT:** Represents the action of outputting information, either to the user or some other destination.
- **SET/ASSIGN:** Assign value to a variable. (e.g. SET Num1 to 5)
- **IF/ELSE IF/ELSE:** Conditional statements for decision-making.

- **WHILE/ENDWHILE:** Looping structure that continues while a condition is true.
- **FOR/TO/ENDFOR:** Looping structure for a specific range of values.
- **DO/UNTIL:** Looping structure that continues until a condition is true.
- **FUNCTION/PROCEDURE:** Defines a function or procedure.
- **CALL:** Invokes a function or procedure.
- **RETURN:** Indicates the end of a function or procedure and returns control to the calling program.
- **DECLARE/DIM/VAR:** Declares a variable.
- **ARRAY**: Represents an array or a collection of variables.
- **EXIT:** Exits a loop or terminates the program.
- **INCREMENT/DECREMENT:** Increases or decreases the value of a variable.

Now, let's go on to solve more examples.

Example 6.8: Write a pseudocode to calculate the average of two numbers.

Step 1: START

Step 2: READ NUM1

Step 3: READ NUM2

Step 4: AVG = (NUM1 + NUM2) / 2

Step 5: PRINT AVG

Step 6: STOP

Example 3.9: Write a pseudocode for a program that calculates simple interest.

Step 1: START

Step 2: READ P, T, R

Step 3: I = P*T*R/100

Step 4: PRINT I

Step 5: STOP

Finally, we will implement the procedure for solving quadratic equations (using the quadratic formula) in pseudocode.

Example 3.10: Write a pseudocode for a program that calculates the roots of a quadratic equation

Step 1: START

Step 2: READ a, b, c

Step 3: DECLARE d, e

Step 4: SET d to ($b^2 - 4ac$**)**

Step 5: e= $\sqrt{d}$

Step 5: Calculate roots:

root1 = (-b + e) / (2a)

root2 = (-b - e) / (2a)

Step 6: PRINT root1, root2

Step 7: STOP

14.2.1 Advantages of Pseudocodes

Some advantages of pseudocodes are:

i) Improves the readability of any approach as it is one of the best approaches to start the implementation of an algorithm.

ii) Acts as a bridge between the program and the algorithm or flowchart. Also works as a rough documentation, so the program of one developer can be

understood easily when a pseudo code is written out. In industries, the approach of documentation is essential. And that is where a pseudocode proves vital.

iii) The main goal of a pseudocode is to explain what exactly each line of a program should do, hence making the code construction phase easier for the programmer.

iv) It allows the designer to focus on the main logic without being distracted by programming language syntax.

v) It is easier to write actual code using pseudocode.

vi) As it is language-independent, it can be translated to any computer language code.

vii) It allows the designer to express logic in plain natural language.

viii) Pseudocodes are concise, more readable, and easier to modify.

ix) Pseudocode cannot be compiled and executed, so its correctness cannot be verified by using computers.

14.2.2 Disadvantages of Pseudocodes

Pseudocode has the following disadvantages:

i. There are no accepted standards for writing pseudocodes, and designers use their style while writing pseudocodes.

ii. Pseudocode cannot be compiled and executed, so its correctness cannot be verified by using computers

14.3 Decision Table

Decision tables are a concise visual illustration for showcasing which actions are to be performed depending on predetermined conditions. They are algorithms that the output is a set of actions. The information communicated in decision tables could also be showcased as decision trees or in a programming paradigm as a sequence of **if-then-else** and **switch-case** statements. Each decision matches a **variable**, **relation**

or **predicate** whose possible values are shown among the condition alternatives. Each action is an operation to be performed, and the entries depict whether the action is to be carried out for the set of condition alternatives the entry matches.

For convenience, many decision tables are included in their condition alternatives or symbols. This can be a hyphen or blank, although using a blank is discouraged as it may merely show that the decision table has not been completed. Decision tables help to reveal situations under which certain input factors are irrelevant to the actions to be taken and allow these input tests to be skipped while streamlining decision-making procedures.

Aside from the basic four-quadrant structure, decision tables vary widely in the way the condition alternatives and action entries are represented. Some decision tables use simple true/false values to represent the alternatives to a condition (similar to **if-then-else**), other tables may use numbered alternatives (similar to **switch-case**), and some tables even use fuzzy logic or probabilistic representations for condition alternatives. In a similar way, action entries can simply represent whether an action is to be performed (by checking the actions to be performed) or, in more advanced decision tables, the sequencing of actions to be performed.

A decision table is considered *balanced* or *complete* if it includes every possible combination of input variables. In other words, balanced decision tables prescribe an action in every situation where the input variables are given.

Decision table offers a handy and compact way to signify complex business logic. In a decision table, business logic is well categorised into conditions, actions (decisions) and rules for indicating the different components that comprise the business logic. A typical example of a decision tree is shown in Figure 14.10.

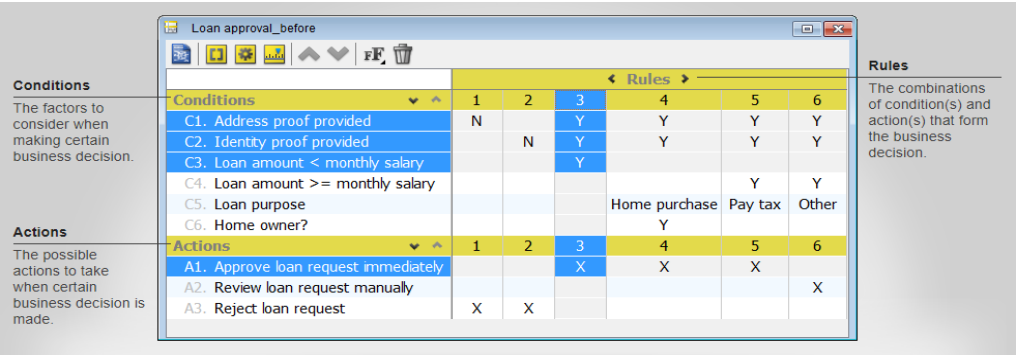


Figure 14.10:Typical Decision Table

Decision table is an avenue for decision-making that consists of studying several conditions and the way they relate. People make use of decision tables to represent and discover business logic, which eventually guarantees better business opportunities.

The use of **decision table software** offers a wide variety of advantages over traditional paper methods, as it makes editing easier. Modern decision table tools can equally provide added value by supporting features like conditions/actions/rules highlighting, table layout and reporting. This software makes sharing, discussion and management of work very much easier.

14.3.1 How to construct Decision Trees for different Scenarios/Use Cases

To aid you in the world of problem solving with decision tables, we will be looking at a few examples. We will start with decision trees with **two conditions**, which have only **two alternatives** and **two actions.**

Example 3.11: You are planning a vacation for a day, and your decision depends on both your budget and the availability of your siblings. You want to decide whether to go to the beach or not based on these factors.

Conditions:

1. High Budget: This condition has two alternatives:

- Yes
- No

2. Siblings Availability: This condition also has two alternatives:

- Yes
- No

Actions:

1. Go on the vacation: If the conditions are favourable (high budget and siblings are available), you will go on the vacation.
2. Stay Home: If the conditions are not favourable, you will stay home.

Now, let us create a decision table to visualise the decision-making process:

Table 14.1**:** Decision table to visualise the decision-making process

Conditions	Budget	Yes	Yes	No	No
	Siblings Available	Yes	No	Yes	No
Actions	Go on Vacation	Yes	No	No	No
	Stay	No	Yes	Yes	Yes

Note: The 'Yes' and 'No' statements can be replaced with 'True' or 'False'. It is also important to note at this point that the number of columns for the varying conditions and corresponding actions is determined by the formula: **No. of Columns = No. of Alternatives to Condition One * No. of Alternatives to Condition Two** Budget has

two alternatives: 'Yes' or 'No', and sibling availability also has two alternatives: 'Yes' or 'No'. Hence, we have **2*2 = 4 columns.**

In this decision table, you can quickly determine your decision based on the current budget and the availability of your siblings. If your budget is high and your siblings are available, you'll decide to go on vacation; otherwise, you'll stay home. The decision table helps simplify the decision-making process for this simple scenario.

Now, we'll proceed with decision trees with **two conditions**, which have only **two alternatives** and **three corresponding choices of actions**.

Example 3.12: You are deciding which mode of transportation to take to work based on two factors: the day of the week (weekday or weekend) and your work schedule.

Conditions:

1. Day of the Week: This condition has two alternatives:
   - Weekday
   - Weekend
2. Work Schedule: This condition also has two alternatives:
   - Morning Shift
   - Evening Shift

Actions:

1. Bicycle: If it's a weekday and you have a morning shift, you'll take your bicycle to work.
2. Car: If it's a weekday and you have an evening shift, or if it's a weekend, regardless of your work schedule, you'll take your car to work.
3. Walk: If you have a morning shift and a weekend, you'll choose to walk to work.

Now, let's create a decision table for this scenario. Recall the formula for determining the number of columns that hold the variable: **No. of columns = no. of alternatives to condition one * no. of alternatives to condition two**; therefore, our table will have **2*2 = 2^2= 4 Columns**

Table 14.2**:** Decision table to decide Mode of Transportation to Work

Conditions	Day Of Week	Weekday	Weekday	Weekend	Weekend
	Shift	Morning	Evening	Morning	Evening
Actions	Bicycle	Yes	No	No	No
	Car	No	Yes	Yes	Yes
	Walk	No	No	Yes	No

In this decision table, you can easily determine your mode of transportation to work based on the day of the week and your work schedule. It provides a structured way to make your decision based on the given conditions and actions.

Now, let's move on to a decision table with **three conditions** and still limit the **actions to three:**

Example 3.13: You are deciding which destination to choose for your weekend trip based on three factors: budget, distance, and weather.

Conditions:

1. Budget: This condition has two alternatives:
   - Low
   - Medium
2. Distance: This condition also has two alternatives:
   - Short
   - Long
3. Weather: This condition has two alternatives:
   - Sunny
   - Rainy

Note: We have three (3) conditions with two alternatives each; hence, our decision tree will have **$2*2*2 = 2^3 = 8$ columns.**

Actions:

1. **Beach:** If you have a high budget, prefer a short distance, and the weather is sunny, you'll choose the beach as your destination.
2. **Mountains:** If your budget is low, you prefer a short distance, and the weather is sunny, you'll choose the mountains.
3. **City:** For all other combinations of budget, distance, and weather, you'll choose the city as your destination.

Now, let's create our decision table for this example.

Table 14.3**:** Decision table to decide the destination for a weekend trip

Conditions	Budget	Low	Low	Low	Low	High	High	High	High
	Distance	Short	Short	Long	Long	Short	Short	Long	Long
	Weather	Sunny	Rainy	Sunny	Rainy	Sunny	Rainy	Sunny	Rainy
Actions	Beach	No	No	No	No	Yes	No	No	No
	Mountain	Yes	No	No	No	No	No	No	No
	City	No	Yes	Yes	Yes	No	Yes	Yes	Yes

In Table 14.3, you can determine your weekend trip destination based on your budget, preferred distance, and the weather forecast. It helps you make the right choice based on the given conditions and available options.

Example 3.14

The limited-entry decision table is the simplest to describe. The condition alternatives are simple Boolean values, and the action entries are checkmarks, representing which of the actions in a given column are to be performed.

The balanced decision table is an example in which a technical support company writes a decision table to enable technical support employees to efficiently diagnose printer problems based on symptoms their clients described to them over the phone.

Table 14.4**:** Printer troubleshooting

		Rules							
Conditions	Printer is printing	No	No	No	No	Yes	Yes	Yes	Yes
	Flashing of red light	Yes	Yes	No	No	Yes	Yes	No	No
	The computer recognises the printer.	No	Yes	No	Yes	No	Yes	No	Yes
Actions	Check power cable			✓					–
	Check the printer signal cable.	✓		✓					–
	Ascertain printer software status.	✓		✓		✓		✓	–
	Check and replace ink.	✓	✓				✓		–

	Check for possible paper jam.		✓		✓				–

As a new technical support employee who has been provided with the above decision table by your employer/company, here is a list of troubleshooting actions that you are expected to perform for every combination of symptoms as described by a client.

Column 1: If the client reports that the printer does not print and is not being recognised by their computer, yet it keeps flashing red light, then you are expected to check the printer's signal and the printer's software status and check/replace the ink.

Column 2: If the client should report that the printer is not printing despite being recognised by the computer while flashing a red light, then you are expected to check/replace the printer's ink and check for possible paper jam.

Column 3: Should the client report that the printer does not print, flashes a red light, or is recognised by their computer, you are to ascertain the status of the printer's software while also ensuring that the printer's power cable and signal cables are connected.

Column 4: For a case where the client reports that although their computer is recognising their printer, it does not print nor flash a red light, your best course of action will be to check for a possible paper jam.

You can go on to generate the implied course of action for the remaining columns as a suggested exercise.

This decision table can be used to train an expert system that diagnoses a printer according to the symptoms reported by a client.

The scheme above only indicates a simple example. It may not essentially correspond to exact reality. However, it reveals how decision tables can conform to a number of conditions with a wide range of possibilities.

14.4 Decision Tree

This is a decision-support hierarchical model which applies a tree-like model of decisions with accompanying consequences, with chance event outcomes, utility and resource costs. It is a method of displaying algorithms that only have conditional control statements. Decision trees are normally applied in operations research. In decision analysis, it helps to recognise a strategy most likely to reach a goal. A decision tree is a popular tool in machine learning.

A decision tree is a flowchart-like structure in which each internal node represents a "test" on an attribute (e.g. whether a coin flip comes up heads or tails), eachbranch represents the outcome of the test, and each leaf node represents a class label (decision taken after computing all attributes). The paths from root to leaf represent classification rules. In decision analysis, a decision tree and the closely related influence diagram are used as a visual and analytical decision support tool, where the expected values (or expected utility) of competing alternatives are calculated.

A decision tree consists of three types of nodes, namely:

i. Decision nodes – typically represented by squares.
ii. Chance nodes – typically represented by circles.
iii. End nodes – typically represented by triangles.

Decision trees are commonly used in operations research and operations management. If, in practice, decisions have to be taken online with no recall under incomplete knowledge, a decision tree should be paralleled by a probability model as the best choice model or online selection model algorithm. Another use of decision trees is as a descriptive means for calculating conditional probabilities.

Drawn from left to right, a decision tree has only burst nodes (splitting paths) but no sink nodes (converging paths). When used manually, they can grow very big and are often hard to draw by hand. Traditionally, decision trees have been created manually, although specialised software is employed.

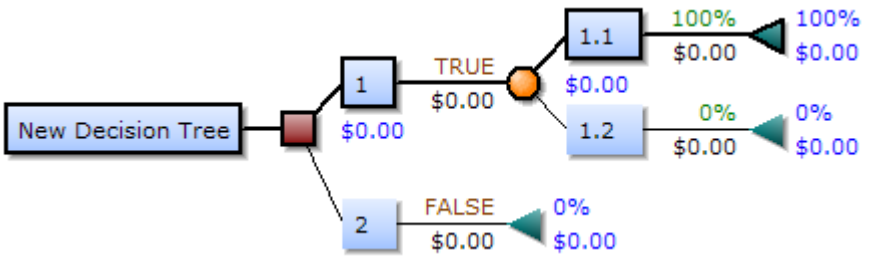


Figure 14.11: Decision-tree elements

While decision trees can be used to solve complex decision problems, we will focus on designing simple decision trees for slightly modified versions of our examples on decision tables in the earlier sections of this chapter.

Example 3.15: Imagine you're planning your daily shuttle to work. Design a decision tree to help you decide what time to arrive at work based on two factors: your chosen mode of transportation and whether or not there is heavy traffic. Consider the decisions you need to make, such as selecting your mode of transportation (car, bicycle, foot) and assessing the traffic conditions (heavy traffic or no heavy traffic). Estimate the likelihood of arriving on time or being delayed. Your decision tree should visually represent the potential outcomes and guide you in making informed choices for your daily shuttle.

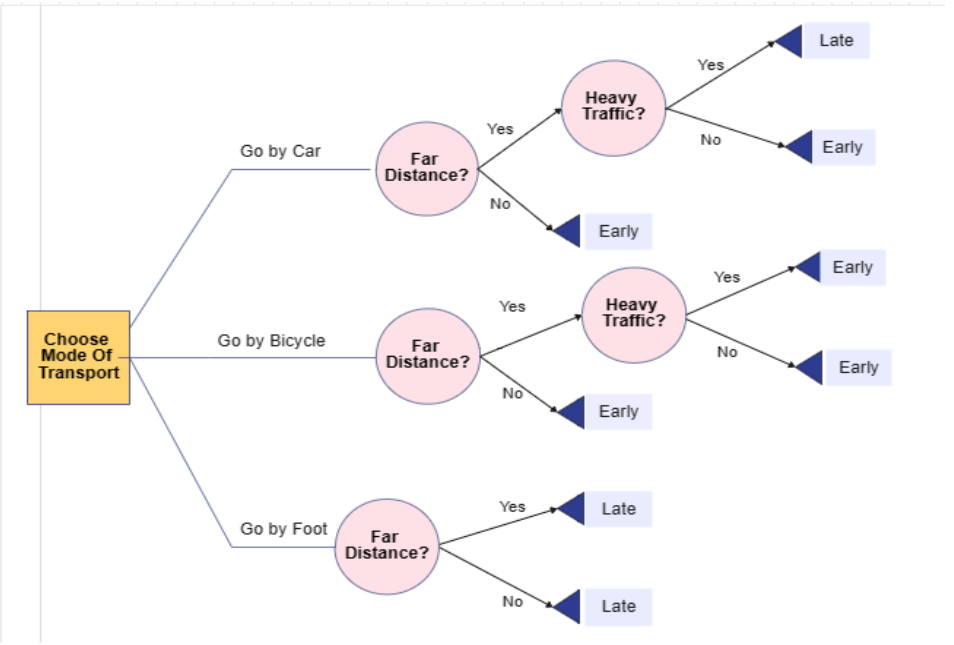


Figure 14.12: Decision tree to decide arrival time at work

From Figure 14.12 and as mentioned earlier, the rectangle is a decision node. It represents a decision you made. The circles are chance nodes, and they depict multiple uncertain outcomes, and the triangles are end nodes or endpoint nodes that indicate an outcome.

From the decision diagram, you will arrive late to work if you ever decide to go on foot. You arrive early in a car only when there isn't heavy traffic, while on a bicycle, you are always early regardless of the magnitude of traffic.

Example 3.16: Design a decision tree for expected value analysis within the context of a software development firm. The decision tree should serve as a visual representation of key decision points and potential outcomes in the software development process. Consider various scenarios, probabilities, and payoffs associated with decisions related to project selection. Provide a hypothetical cost for each decision taken at the decision node, assign hypothetical probabilities/ payoffs

for each uncertain outcome at the chance nodes, and, hence, calculate the expected values.

Do not be put off by the seeming complexity of this problem nor by similar problems you might encounter during your study or work career as a problem solver in the computing field. This question is asking you to create a simple picture (a decision tree) that shows important choices and possible results when developing software. Imagine you are deciding on projects and want to understand the costs and benefits. The decision tree should help you see different scenarios, like which projects might succeed or fail and to what extent. You will need to guess how likely each scenario is and what it could cost or earn. Then, calculate the average (expected) value, i.e., the average possible profit of each choice considering the guesses. The goal is to have a clear view of the risks and rewards in your software development decisions.

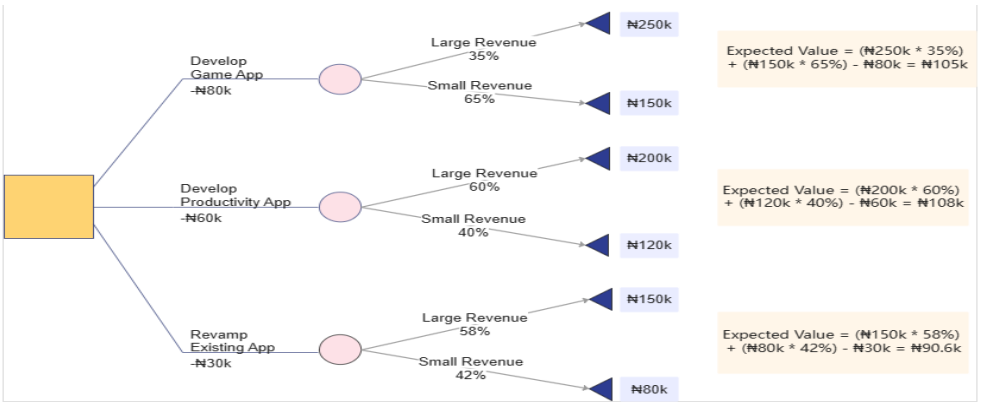


Figure 14.13: Decision tree for expected value analysis within the context of a software development firm

The decision tree depicted in Figure 14.13, to a large extent, is all that is required of us. Now, let's go through the steps that were taken to arrive at this:

Identify Decision Points: Identify key decision points in the software development process where choices need to be made. In this case, these are the decisions related to project selection.

Define Possible Scenarios: For each decision point, identify possible scenarios or alternatives. In this case, the firm could choose to build a game app at a cost of ₦80,000.00, or a productivity app at a cost of ₦60,000.00 or revamp an existing app at a cost of ₦30,000.00.

Decision Nodes and Terminal Nodes: Represent decision nodes (where decisions are made), chance nodes and terminal nodes (endpoints representing outcomes) in the decision tree. These are represented by rectangles, circles, and triangles, respectively.

Connect Nodes: Connect nodes with branches to represent the flow of decisions and outcomes.

Determine Probability/Payoffs: Define the payoffs or outcomes associated with each scenario. In this context of a software development firm, payoffs are represented by financial gain (it could be any other metrics like project success, customer satisfaction, etc.). The various percentages in small and large revenues show this.

Calculate Expected Values: Using the probabilities and/or payoffs to calculate the expected value at each decision node. This involves multiplying the probability of each scenario by its associated payoff and summing these values.

Analyse the Tree: Analyse the decision tree to identify the optimal decision path(s) that maximises the expected value. From our decision tree, developing a productivity app yields the maximum payoff.

Creating an effective decision tree requires careful consideration of choices you might have to make, uncertainties, probabilities, and potential outcomes, as it serves as a structured framework for decision-making in complex situations.

14.5 Algorithm

In computer programming terms, an algorithm is a set of well-defined instructions to solve a particular problem. It takes a set of input(s) and produces the desired output. An algorithm is a step-by-step procedure that defines a set of instructions that must be carried out in a specific order to produce the desired result. Algorithms are generally developed independent of underlying languages, which means that an algorithm can be implemented in more than one programming language. Certainty, fineness, effectiveness, and language independence are some of the characteristics of an algorithm. The scalability and performance of an algorithm are the primary factors that contribute to its importance.

Qualities of a good algorithm are:

i. Input and output should be defined precisely.
ii. Each step in the algorithm should be unambiguous.
iii. Algorithms should be the most effective among many ways of solving a problem.
iv. An algorithm should not include computer codes. Instead, the algorithm should be written in such a way that it can be used in different programming languages.

A problem can be defined as a real-world problem or real-world instance problem for which you need to develop a program or set of instructions. An algorithm is a set of instructions or a step-by-step process that is designed for a solution to a problem. After designing an algorithm, the algorithm is given the necessary and desired inputs. The input will be passed to the processing unit, producing the desired output. The outcome or result of the program is referred to as the output. This is shown in Figure 14.14.

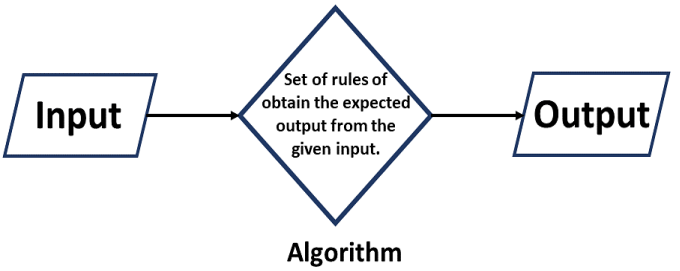


Figure 14.14: Characteristics of an algorithm

An algorithm has the following characteristics:

i) Input: An algorithm requires some input values. An algorithm can be given a value other than 0 as input.
ii) Output: At the end of an algorithm, you will have one or more outcomes.
iii) Unambiguity: A perfect algorithm is defined as unambiguous, which means that its instructions should be clear and straightforward.
iv) Finiteness: An algorithm must be finite. Finiteness in this context means that the algorithm should have a limited number of instructions, i.e., the instructions should be countable.
v) Effectiveness: Because each instruction in an algorithm affects the overall process, it should be adequate.
vi) Language independence: An algorithm must be language-independent, which means that its instructions can be implemented in any language and produce the same results.

Algorithms are required because they aid in your understanding of scalability (when you have a sizable real-world problem, you must break it down into small steps to analyse it quickly) and performance. The real world is challenging to break down into

smaller steps. If a problem can be easily divided into smaller steps, it indicates that the problem is feasible.

The factors to consider when designing an algorithm:

i. Modularity: This feature was perfectly designed for the algorithm if you are given a problem and break it down into small-small modules or small-small steps, which is a basic definition of an algorithm.
ii. Correctness: This is a situation where the given inputs produce the desired output, indicating that the algorithm was designed correctly.
iii. Maintainability: It means that the algorithm should be designed in a straightforward, structured way so that when you redefine the algorithm, no significant changes are made to the algorithm.
iv. Functionality: It considers various logical steps to solve a real-world problem.
v. Robustness: This is an algorithm's ability to define the problem clearly.
vi. User-friendly: If the algorithm is difficult to understand, the designer will not explain it to the programmer.
vii. Simplicity: If an algorithm is simple, it is easy to understand.
viii. Extensibility: Your algorithm should be extensible if another algorithm designer or programmer wants to use it.

There are two factors in which the algorithm is fundamental:

i. Theoretical significance: When you are given a real-world problem, you must break it down into smaller modules. To deconstruct the problem, you must first understand all the theoretical aspects.
ii. Practical significance: Theory cannot be completed without practical application. As a result, the significance of algorithms can be considered both theoretically and pragmatically.

There are no well-defined standards for writing algorithms. It is, however, a problem that is resource-dependent. Algorithms are never written with a specific programming language in mind. Basic code constructs, such as loops like **DO**, **FOR**, and **WHILE**, are common constructs shared by all programming languages flow control, such as **IF-ELSE**. Algorithms are typically written in a step-by-step fashion, but this is not always the case. Algorithm writing is a process that occurs after the problem domain has been well-defined. That is, you must be aware of the problem domain for which you are developing a solution.

Example 3.17: Let us use the problem, c*reate an algorithm that multiplies two numbers and displays the output* as an example to learn how to write algorithms.

Step 1 − Start

Step 2 − declare three integers x, y & z

Step 3 − define values of x & y

Step 4 − multiply values of x & y

Step 5 − store result of step 4 to z

Step 6 − print z

Step 7 − Stop

Example 3.18: Algorithm to add two numbers.

Step 1: Start
Step 2: Declare variables num1, num2 and sum.
Step 3: Read values num1 and num2.
Step 4: Add num1 and num2 and assign the result to the sum.
sum←num1+num2
Step 5: Display sum
Step 6: Stop

Example 3.19: Algorithm to find the largest number among three numbers.

Step 1: Start
Step 2: Declare variables a,b and c.
Step 3: Read variables a,b and c.
Step 4: If a > b
    If a > c
      Display a is the largest number.
    Else
      Display c is the largest number.
  Else
    If b > c
      Display b is the largest number.
    Else
      Display c is the largest number.
Step 5: Stop

Example 6.20: Algorithm to find all the roots of a quadratic equation $ax^2 + bx + c = 0$

Step 1: Start

Step 2: Declare variables a, b, c, D, x1, x2, rp and ip;

Step 3: Calculate discriminant

D ← b2-4ac

Step 4: If D ≥ 0

r1 ← (-b+√D)/2a

r2 ← (-b-√D)/2a

Display r1 and r2 as roots

Else

Calculate real part and imaginary part

rp ← -b/2a

ip ← √(-D)/2a

Display rp+j(ip) and rp-j(ip) as roots

Step 5: Stop

Example 3.21: Algorithm to find the factorial of a number.

Step 1: Start
Step 2: Declare variables n, factorial and i.
Step 3: Initialize variables
factorial ← 1
i ← 1
Step 4: Read value of n
Step 5: Repeat the steps until i = n
5.1: factorial ← factorial*i
5.2: i ← i+1
Step 6: Display factorial
Step 7: Stop

Algorithms in Computer Science can be broadly categorised into searching and sorting algorithms. **Sorting**, such as selection sort, bubble sort, insertion sort, merge sort, and quick sort. **Searching**, such as binary search, exponential search, and jump search. But, there are many types of algorithms that programmers use regularly. Here are some other common algorithm types organised by category:

Hashing (SHA-256, SHA-1),

Brute force (trial and error),

Divide and Conquer (merge sort algorithm),

Greedy (Prim's algorithm, Kruskal's algorithm) and

Recursive (computer factorials).

14.6 Data Flow Diagram

Data flow diagram (DFD) is a graphical representation of data flow through a system or sub-system. Each process has its own data flow, and there are levels of data flow diagrams. Level 0 shows the input and output data for the whole system. Then, the system is broken down into modules, and level 1 DFD shows the data flow for each module separately. Modules may further be broken down into sub-modules if required and level 2 DFD drawn.

The flow of data of a system or a process is represented by DFD. It also gives insight into the inputs and outputs of each entity and the process itself. DFD does not have a control flow, and no loops or decision rules are present. Specific operations, depending on the type of data, can be explained by a flowchart. It is a graphical tool useful for communicating with users, managers, and other personnel. It is useful for analysing existing as well as proposed systems. It provides an overview of what data a system processes, what transformations are performed, what data are stored and what results are produced.

DFD can be represented in several ways. The DFD belongs to structured-analysis modelling tools. Data Flow diagrams are very popular because they help us to visualise the major steps and data involved in software-system processes.

The DFD has four components:

i. ***Process:*** The task performed on the data is known as process. The process is represented by a circle. In some cases, round-edged rectangles are also used to represent a process. Input-to-output transformation in a system takes place because of process function. The symbols of a process are rectangles with rounded corners, ovals, rectangles, or circles.

ii. ***Data Flow***: Data flow describes the information transfer between different parts of the systems. The movement of data in the system is known as data flow. The direction of flow is represented by the arrow (symbol), which can also be bi-directional. A relatable name should be given to the flow to determine the information which is being moved. Data flow also represents material along with information that is being moved. A given flow should only transfer a single type of information.

iii. ***Warehouse (data storage):*** Data storage includes the database of the system. It is represented by a rectangle with both smaller sides missing or, in other words, within two parallel lines. The data is stored in the warehouse for later use. Two horizontal lines represent the symbol of the store. The warehouse is simply not restricted to being a data file; rather, it can be anything like a folder with documents, an optical disc, and a filing cabinet. The data warehouse can be viewed independently of its implementation. When the data flow is from the warehouse, it is considered as data reading, and when data flows to the warehouse, it is called data entry or data updating.

iv. ***Terminator (Entities)***: The terminator is an external entity that stands outside of the system and communicates with the system. Entities include the source and destination of the data. Entities are represented by a rectangle with their corresponding names. It can be, for example, organisations like banks, groups of people like customers or different departments of the same organisation, which is not a part of the system model and is an external entity. Modelled systems also communicate with terminators.

The rules for creating DFD are:

i. The name of the entity should be easy and understandable without any extra assistance (like comments).
ii. The processes should be numbered or put in an ordered list to be easily referred to.
iii. The DFD should maintain consistency across all the DFD levels.
iv. A single DFD can have a maximum of nine processes and a minimum of three processes.

The symbols used in DFD:

i. ***Square Box:*** A square box defines the source or destination of the system. It is also called entity. It is represented by a rectangle.
ii. ***Arrow or Line:*** An arrow identifies the data flow, i.e., it gives information to the data that is in motion.
iii. ***Circle or bubble chart:*** It represents a process that gives us information, otherwise called a processing box.
iv. ***Open Rectangle:*** An open rectangle is a data store. Here, data is stored either temporarily or permanently.

DFDs use hierarchy to maintain transparency; thus, multilevel DFDs can be created. Levels of DFD are:

i. 0-level DFD: It represents the entire system as a single bubble and provides an overall picture of the system.
ii. 1-level DFD: It represents the main functions of the system and how they interact with each other.

iii. 2-level DFD: It represents the processes within each function of the system and how they interact with each other.

iv. 3-level DFD: It represents the data flow within each process and how the data is transformed and stored.

14.6.1 Designing a Data Flow Diagram

The **Gane & Sarson Data Flow Diagram (DFD**) and the **Yourdon & Coad Data Flow Diagram** are two variations of DFDs used in problem solving. While they share some similarities, the major difference between the two is in the way they represent processes.

While Gane & Sarson Data Flow diagrams often use a more visual and structured notation where processes (usually represented as rectangles with rounded edges) are clearly outlined and labelled, Yourdon & Coad Data Flow Diagram use fewer formal notations and can be quite unstructured. Processes are typically represented as circles in Yourdon & Coad DFDs.

The choice of what variation of Data Flow Diagram to use depends largely on the preference of an individual, group or organisation, as both types of DFDs serve the purpose of visually representing the flow of data within a system, aiding in understanding and designing complex systems.

The design of Data Flow Diagrams using any of the two variations will be demonstrated in the following examples.

Example 3.21: Data Flow diagram for a (Job) Hiring System using Gane & Sarson representation.

For this system, we can identify two actors, the "applicant" and the "manager". Hence, the 0-level DFD that represents the entire system as a single bubble and provides an overall picture of the system is given as Gane & Sarson representation in Figure 14.15.

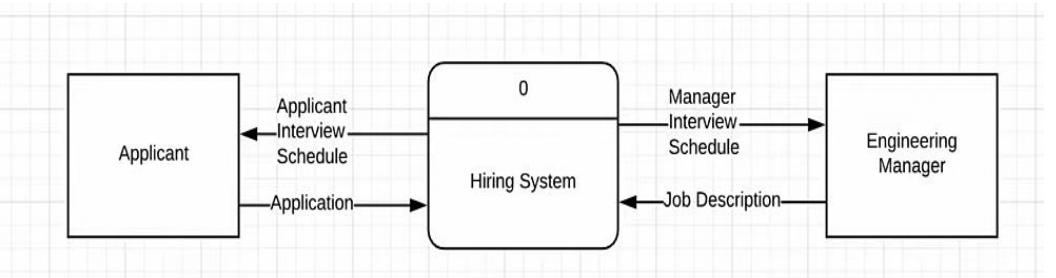


Figure 14.15: 0-level data flow diagram of a job hiring system

In the diagram in Figure 14.15, we have shown how to create a 0-level (a.k.a context-level) data flow diagram. Now, we will take that diagram and transform it into a 1-level diagram. In a 1-level DFD, the symbol representing the system gets exploded or decomposed into a series of processes.

Recall that we mentioned that 1-level diagrams represent the main functions of the system and how they interact with each other. In the 0-level DFD above, the main "Hiring System" (as depicted by the curved edge rectangle in Figure 14.15) can be further decomposed into sub-processes or main functions, as shown in Figure 14.16.

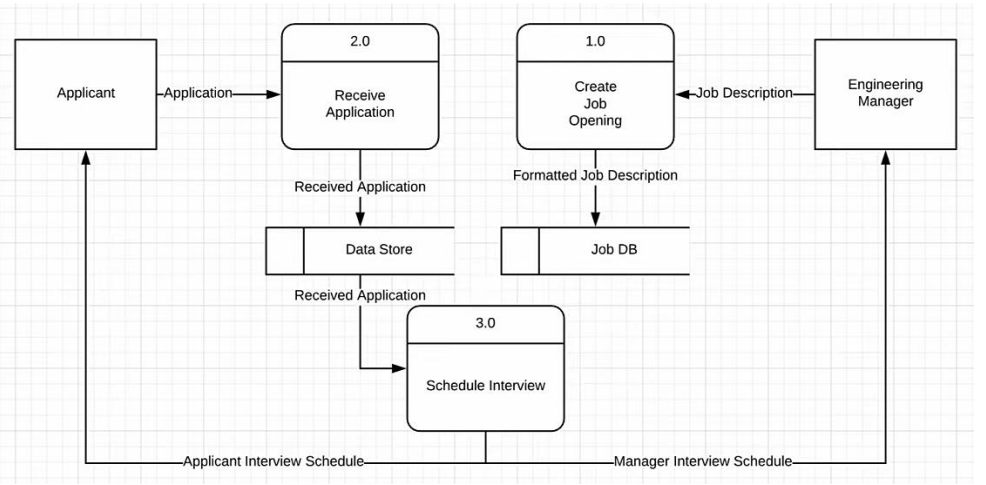


Figure 14.16: 1-level data flow diagram of a job hiring system

In the 1-level DFD Figure 14.16, we have decomposed the hiring system into three child processes. The first process is "Create Job Opening," denoted as 1.0, which creates a job opening. The data flow for this is a "Job Description," which is stored in a "Job Database".

Next, we have the process "Receipt Application," denoted as 2.0, which receives applications. The data flow here is an "Application," and it's stored in an "Application Database".

The third process is "Scheduling an Interview," labelled as 3.0. In this process, the applicant's availability, obtained from the received application, and the manager's schedule are considered. Once a suitable time is found, a data flow goes back to the applicant as an "Applicant Interview Schedule". Additionally, there's a "Manager Interview Schedule" that includes all scheduled interviews through the human resource function.

Again, recall that 2-level DFD represents the processes within each function of the system and how they interact with each other. So far, we have identified three functions, but in the interest of space, we will be focusing only on the "Schedule Interview" main function and decomposing it further into its primary process.

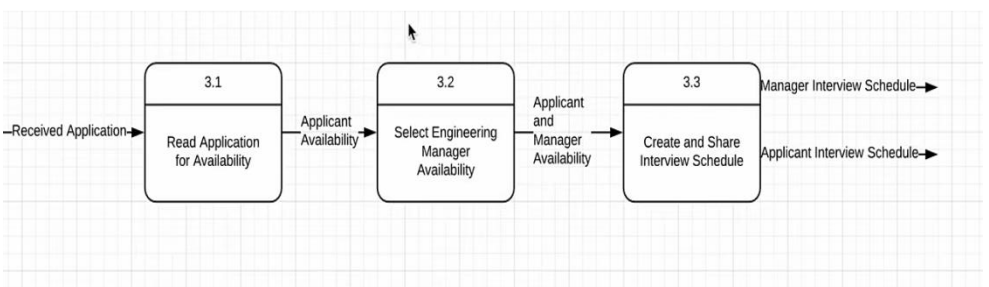


Figure 14.17: **A 2-level data flow diagram of a job hiring system**

Here's the breakdown of the 2-level data flow diagram:

The "Read Application or Availability" process begins by receiving the application or availability information. In this case, it's assumed that the application includes details about the applicant's availability. The data flow "Receive Application" carries this information.

"Select Engineering Manager Availability" is necessary to know the availability of both the applicants and the engineering managers. This process collects the availability information of the engineering manager. The data flow "Select Engineering Manager Availability" carries this data.

"Create and Share Interview Schedule": This final step in the process involves creating and sharing the interview schedule. It results in two separate outputs: the "Manager Interview Schedule," which is sent to the engineering manager, and the "Applicant Interview Schedule," which is sent to the applicant.

Note that in decomposing a main function in the 1-level DFD into a 2-level DFD, you no longer show external entities but rather only the data flows coming into and out of the process you are detailing. Observe how the three data flows, namely 'Receive Application', 'Manage Interview Schedule', and 'Applicant Interview Schedule', that are moving in and out of the 'Schedule Interview' main function in the 1-level DFD are duly represented in the 2-level DFD. This diagram provides a more detailed view of the "Schedule Interview" process, offering a deeper understanding of the system's functioning.

Example 3.22: Railway reservation system (using Yourdon & Coad representation)

0-level DFD: It is also known as a context diagram, designed to be an abstraction view, showing the system as a single process with its relationship to external entities.

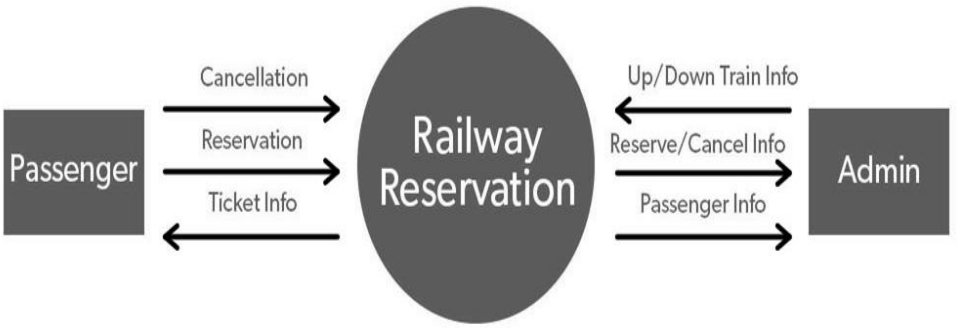


Figure 14.18: **Railway reservation system Illustration I**

1-level DFD: In 1-level DFD, the context diagram is decomposed into multiple bubbles/processes. At this level, we highlight the main functions of the system and break down the high-level process of 0-level DFD into sub-processes.

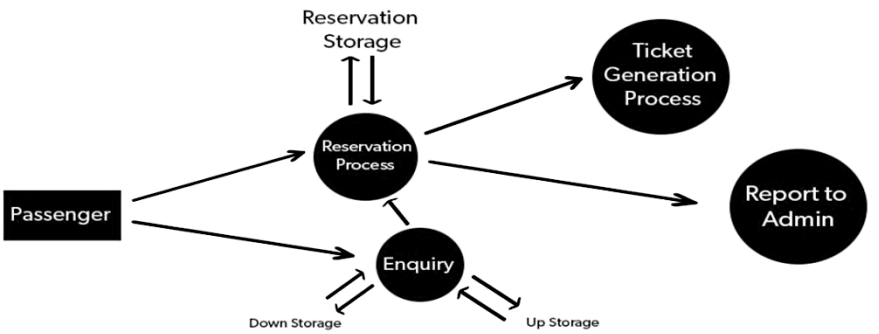


Figure 14.19: **Railway reservation system illustration II**

2-level DFD: 2-level DFD goes one step deeper into parts of 1-level DFD. It can be used to plan or record specific or necessary details about the system's functioning.

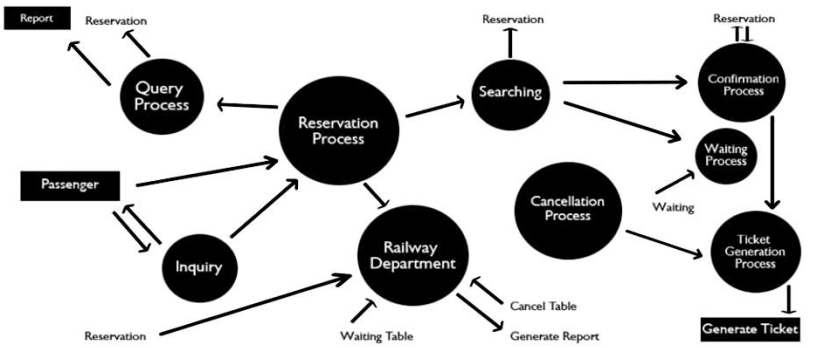


Figure 14.20**: Railway reservation system illustration III**

To create or build a DFD, the process can be broken down into five steps:

1. Identify major inputs and outputs in your system: Every process or system begins with input from an external entity and ends with the output of data to another entity or database. Identifying such inputs and outputs gives a macro view of the system.
2. Build a context diagram: Once you have identified the major inputs and outputs, building a context diagram is simple. Draw a single process node and connect it to related external entities. This node represents the most general process that information follows to go from input to output.
3. Expand the context diagram into a level 1 DFD: The single process node in your context diagram does not provide much information, so you need to break it down into sub-processes. In level 1 data flow diagram, you should include several process nodes, major databases, and all external entities.
4. Expand to a level 2+ DFD: To enhance the detail of your data flow diagram, follow the same process as in Step 3. The processes in your level 1 DFD can be broken down into more specific sub-processes. Once again, ensure you add any necessary data stores and flows. At this point, you should have a

fairly detailed breakdown of your system. To progress beyond a level 2 data flow diagram, simply repeat this process. Stop once you have reached a satisfactory level of detail.

5. Confirm the accuracy of your final diagram: When your diagram is completely drawn, walk through it. Pay close attention to the flow of information and check if it makes sense. Check if all the necessary data stores are included. By looking at your final diagram, other parties should be able to understand the way your system functions.

14.6.2 Some advantages of DFD are:

i. It helps us understand the functioning and limits of a system.
ii. It is a graphical representation, which is very easy to understand as it helps visualise contents.
iii. Data Flow Diagram represents a detailed and well-explained diagram of system components.
iv. It is used as a part of the system documentation file.
v. Both technical and non-technical people can understand Data Flow Diagrams because they are very easy to understand.

14.6.3 The disadvantages of DFD are:

i. At times, DFD can confuse the programmers regarding the system.
ii. Data Flow Diagram takes a long time to generate, and many times, due to this, analysts are denied permission to work on it.

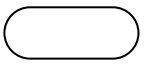
The two types or categories of DFD are logical and physical flow diagrams. A logical DFD visualises the data flow that is essential for a business to operate. It focuses on the business and the information needed, not on how the system works or is proposed to work. However, a physical DFD shows how the system is actually implemented or how it will be. For example, in a logical DFD, the processes would be

business activities, while in a physical DFD, the processes would be programs and manual procedures. A logical flow diagram is a type of flow diagram that mainly focuses on business activities and the business as a whole. It can be considered the fundamental basis of the physical flow diagram because, by modifying the logical flow diagram, we can get the physical flow diagram. Logical flow diagrams explain data and business events that are necessary to accomplish the business events. A physical flow diagram can be developed based on a logical flow diagram. A physical flow diagram depicts the working of the data in the system with the support of paper files, software, hardware, and people involved. The flow of business activities and actions is mapped with the help of a logical flow diagram. The inefficiencies and shortcomings of a system are clarified with the help of a logical flow diagram. The steps involved in the data process, i.e., eliminating and modifying the data, are also obtained from the logical flow diagram. Non-technical people can easily understand the mechanism of a logical flow diagram.

Summary

In this chapter, the programming tools used in problem solving were discussed, covering concepts of flowcharts, pseudocodes, decision tables, decision trees, algorithms, and data flow diagrams. A flowchart is the perfect way to visualise a complex process, and there are over thirty standardised symbols used to create a flowchart. Pseudocode is often used in various fields of programming, whether it is app development, data science, or web development, as it is a technique used to describe the distinct steps of an algorithm in a manner that is easy to understand for anyone with basic programming knowledge.

Multiple Choice Questions



1. The symbol denotes _____.

A. I/O B. Flow C. Terminal D. Decision

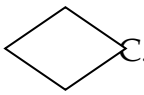
2. In Computer Science, an algorithm refers to a pictorial representation of a flowchart.

A. True B. False

3. The process of drawing a flowchart for an algorithm is called _____.

A. Performance B. Evaluation C. Algorithmic Representation D. Flowcharting

4. Actual instructions in flowcharting are represented in _____.

A. Circles B.  C. Arrows D. Lines

5. The following box denotes:

A. Decision B. Initiation C. Initialization D. I/O

6. A box that can represent two different conditions.

A. Rectangle B. Diamond C. Circle D. Parallelogram

7. A decision table

A. represents the information flow. B. shows the decision paths.

C. documents rules that select one or more actions, based on one or more conditions, from a set of possible conditions.

D. gets an accurate picture of the system. E. None of the above.

8. A detailed flowchart is called _____.

A. Stack B. Macro C. Micro D. Union

9. Which of the following is not an advantage of a flowchart?

A. Better communication B. Efficient coding

C. Systematic testing D. Improper documentation

10. A flowchart that outlines the main segments of a program.

A. Queue B. Macro C. Micro D. Union

11. Decision tables are useful in situations where

A. an action is taken under varying sets of conditions.

B. number of combinations of actions is taken under varying sets of conditions.

C. no action is taken under varying sets of conditions.

D. None of the above.

12. In decision tables, which of the following is true?

A. Number of test cases is equal to the number of rules (columns)

B. Number of test cases is not equal to number of rules (columns)

C. Both (a) and (b

D. None of the above

13. Data Flow Diagrams are used for:

A . Process modeling B. Modelling interactions in a real time environment

C. Data modelling D. None of the above

14. The last step in the process of problem-solving is to

A. design a solution. B. define a problem.

C. practice the solution. D. organise the data.

15. Rectangle shaped symbol is used to show the

A. carried operations B. errors C. decisions D. defined variables

16. The sequence of the statements is shown in the order which is from

A. left to right B. right to left C. top to down D. down to top

17. Algorithm is made up of

A. sequence to print data B. selection C. repetition D. all of the above

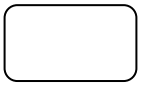
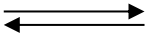
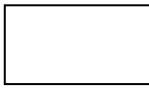
18. A decision tree does not include:

A. statements B. Condition stubs C. Action stubs D. Rules

19. The part of a decision table that links conditions to actions is the section that contains

A. action statements B. Rules C. condition statements D. decision stubs

20. Which of these shapes would be used to show data storage or output in a data flow diagram?

A.  B.  C.  D. 

Exercises

1. Draw and briefly explain five symbols commonly used in a flowchart.

2. Identify the advantages of using flowcharts.

3. Draw a flowchart to solve the problem of a non-functioning light bulb.

4. Draw a flowchart for an algorithm that gets two numbers and prints the sum of their value.

5. Draw a flowchart for the problem of printing even numbers between 9 and 100.

6. Write a pseudocode that reads two numbers, multiplies them together and print out their product.

7. Write a pseudocode that tells a user that the number they entered is not a 5 or a 6.

8. Write pseudocode to print all multiples of 5 between 1 and 100 (including both 1 and 100).

9. Write an algorithm to check whether a number is a prime number or not.

10. Differentiate between logical data flow diagram and physical data flow diagram.

11. Draw the data flow diagram of a hotel reservation showing the 3 levels.

12. Outline two major differences between the decision table and the decision tree.

13. Imagine you're planning to build a simple mobile app. Design a decision tree to help you make key choices during the development process. Consider decisions like selecting a programming language, choosing app features, and deciding on a testing strategy. Assign hypothetical costs to different decisions and estimate the likelihood of success for uncertain outcomes. Your decision tree should provide a clear visual guide, showing the potential risks and rewards for each decision you might face while creating your mobile app.

CHAPTER FIFTEEN
TECHNIQUES OF SOLVING PROBLEMS

Overview

This chapter provides sufficient knowledge about techniques for problem solving. A number of strategies for solving problems were discussed with practical examples. Key concepts and definitions of terms were also discussed.

Objectives

This chapter focuses on providing students with the basic knowledge of different techniques/strategies for solving problems. Specifically, students should be able to:

1. Analyse problems: Students should be able to break down complex problems into smaller components and identify key issues that need to be addressed.
2. Generate and evaluate solutions: Students should be able to generate multiple possible solutions to a problem, evaluate the pros and cons of each option, and select the most appropriate solution.
3. Apply problem-solving strategies: Students should be familiar with a range of problem-solving strategies, such as trial and error, brainstorming, and decision trees, and be able to apply these strategies effectively.
4. Work collaboratively: Students should be able to work effectively in teams, communicate their ideas clearly, and contribute to group problem-solving efforts.
5. Adapt to changing circumstances: Students should be able to adapt their problem-solving approach as circumstances change and be able to modify their strategies as new information emerges.

6. Use critical thinking skills: Students should be able to use critical thinking skills to evaluate information, identify biases, and make reasoned judgments.
7. Apply problem-solving skills to real-world scenarios: Students should be able to apply their problem-solving skills to real-world scenarios and use these skills to solve complex problems in their personal and professional lives.

15.1 Abstraction

Abstraction is a powerful technique used in problem solving that involves simplifying complex problems by focusing on the essential details while omitting unnecessary complexities. It involves creating a higher-level representation or model of a problem that captures the key features and relationships without getting bogged down in the specifics. Abstraction allows problem solvers to manage complexity and deal with problems at a more manageable level, making it easier to understand, analyse, and solve.

The process of abstraction typically involves the following steps:

1. Identifying the problem: The first step in problem-solving is to clearly define and identify the problem that needs to be solved. This involves understanding the context, constraints, and requirements of the problem.
2. Extracting relevant information: Once the problem is identified, the next step is to extract the relevant information from the problem statement or situation. This includes identifying the key elements, variables, and relationships that are important to the problem.
3. Simplifying complex details: Abstraction involves simplifying complex details and filtering out irrelevant information. This may involve generalising or summarising the problem in a higher-level representation that captures the core aspects of the problem while ignoring the unnecessary details.

4. Creating a model: After simplifying the problem, the next step is to create a model or representation of the problem. This could be a diagram, flowchart, concept map, or any other form of representation that captures the essential features of the problem.
5. Analysing the model: Once the model is created, it can be analysed to gain insights, identify patterns, and uncover potential solutions. This may involve using various problem-solving techniques, such as brainstorming, logical reasoning, or experimentation, to generate ideas and evaluate potential solutions.
6. Applying the solution: Finally, after analysing the model and identifying a potential solution, the abstracted solution can be applied to the original problem. This may involve translating the abstracted solution back into the specific context of the problem and implementing it to solve the problem effectively.

Abstraction is particularly useful in complex problem-solving scenarios where the problem is too intricate or overwhelming to be tackled directly. By simplifying the problem and creating abstract representations, problem solvers can gain a clearer understanding of the underlying issues and generate creative solutions.

Problem

You have been assigned the task of building a computer game that simulates the movement of planets in a solar system. The game needs to accurately depict the gravitational interactions between the planets, as well as their orbital paths and speeds. However, you find that the calculations required for simulating the precise physics of planetary motion are complex and time-consuming.

Solution:

Step 1: Identify the core elements of the problem: In this case, the core elements are the planets, their gravitational interactions, and their orbital paths and speeds.

Step 2: Abstract the problem: Instead of trying to simulate the precise physics of planetary motion, you can abstract the problem by simplifying it and creating a higher-level representation. For example, you can represent each planet as a point object with mass and initial velocity, and use simplified equations to simulate their motion. This abstraction allows you to focus on the essential aspects of the problem and ignore the complexities of the detailed physics.

Step 3: Design a simplified model: Based on the abstraction, you can design a simplified model that captures the essential aspects of the problem. For example, you can use Newton's law of gravitation and basic mechanics to calculate the net gravitational force acting on each planet and update their positions and velocities accordingly. This simplified model allows you to compute the planetary motion with reasonable accuracy while avoiding the need for complex calculations.

Step 4: Implement the simplified model: Once you have designed the simplified model, you can implement it in your computer game. You can use programming languages or game engines to code the simulation of planetary motion based on the abstraction and simplified model.

Step 5: Test and refine the simulation: After implementing the simplified model, you can test the simulation by running it with different initial conditions and scenarios. You can compare the results with known data or observations of planetary motion to verify the accuracy of the simulation. If necessary, you can refine the abstraction or the simplified model to improve the accuracy of the simulation.

By using abstraction as a problem-solving technique, you were able to simplify the complex physics of planetary motion into a higher-level representation and design a simplified model that can be implemented in a computer game. This approach allows you to achieve a reasonable level of accuracy in simulating planetary motion while avoiding the complexities of detailed physics calculations.

15.2 Analogy

Analogy is a powerful technique of problem solving that involves drawing parallels or finding similarities between two seemingly unrelated situations or domains. It relies on the principle that if two things are similar in some respects, they are likely to be similar in other respects as well. By using analogy, we can transfer knowledge or solutions from one domain to another, helping us solve problems in new and creative ways.

The process of using analogy as a technique of problem-solving typically involves the following steps:

1. Identifying the problem: Clearly define the problem that needs to be solved. This could be a complex situation, a challenge, or an obstacle that requires resolution.
2. Identifying the source domain: Identify a domain or situation that is unrelated to the problem at hand but shares some similarities or features with it. This could be something from a different field, industry, or context.
3. Extracting analogies: Identify the relevant aspects or features of the source domain that are analogous to the problem at hand. This could involve examining the similarities, relationships, or patterns between the two domains.
4. Applying analogies: Transfer the knowledge or solutions from the source domain to the problem at hand. This could involve adapting or modifying the solutions from the source domain to suit the current problem, or using the insights gained from the analogy to generate new ideas or approaches.
5. Evaluating and refining the solution: Assess the effectiveness of the analogy-based solution and refine it as necessary. This may involve testing, iterating, and refining the solution to ensure that it effectively addresses the problem.

Analogies can be used in various problem-solving contexts, including scientific research, engineering, business, and everyday life.

Problem

A company is struggling with low employee morale and productivity. The management team is trying to come up with ways to address the issue and improve the overall work environment.

Solution: The management team decides to use the analogy of a sports team to solve the problem. They realise that just like in a sports team, a positive team culture, clear goals, and effective communication are essential for success. Drawing from this analogy, they come up with the following solutions:

1. Foster a positive team culture: The management team decides to create opportunities for team-building activities, such as team lunches, offsite events, and recognition programs, to boost team morale and create a sense of camaraderie among employees.
2. Set clear goals: Similar to a sports team that needs a clear game plan, the management team establishes clear expectations, goals, and objectives for each department and the individual employee. This helps to align everyone's efforts and create a sense of purpose in the workplace.
3. Improve communication: Just like how effective communication is crucial in a sports team, the management team implements measures to improve communication channels within the company. This includes regular team meetings, feedback sessions, and open-door policies to encourage open and transparent communication among employees and management.
4. Provide training and development opportunities: Similar to how athletes constantly train and develop their skills, the management team invests in training and development programs for employees to enhance their skills,

knowledge, and career growth. This helps employees feel valued and motivated to perform better.

By using the analogy of a sports team, the management team is able to identify and implement solutions that address the low employee morale and productivity issues.

15.3 Brainstorming

Brainstorming is a widely used and effective technique for problem solving that encourages the generation of creative ideas and solutions through open and collaborative discussion. It involves a group of individuals coming together to collectively generate ideas, insights, and perspectives on a particular problem or challenge.

The process of brainstorming typically follows a few key principles:

1. Free-flowing idea generation: Participants are encouraged to share their ideas freely without any judgment or criticism. This allows for a wide range of ideas to be generated without any restrictions, fostering creativity and innovation.
2. Quantity over quality: The focus is on generating a large number of ideas without evaluating or filtering them initially. This encourages participants to think beyond conventional solutions and explore a wide range of possibilities.
3. Building on others' ideas: Participants are encouraged to build on each other's ideas and collaborate in the development of new ideas. This creates a synergistic effect, where ideas are expanded and refined through collective input.
4. Diverse perspectives: Brainstorming seeks to include diverse perspectives from participants with different backgrounds, experiences, and expertise.

This brings a broader range of ideas and solutions to the table, enriching the problem-solving process.

5. Non-linear approach: Brainstorming does not follow a linear or sequential process. Ideas can be generated in any order and can be linked, combined, or expanded upon as the discussion progresses.
6. Encouragement of wild ideas: Brainstorming encourages participants to think outside the box and share unconventional or "wild" ideas. This stimulates creativity and opens up new possibilities that may not have been considered in a more traditional problem-solving approach.
7. Evaluation and selection: After the brainstorming session, the generated ideas can be evaluated and selected based on their feasibility, potential impact, and alignment with the problem-solving goals. This step typically comes after the idea-generation phase to avoid stifling creativity during the brainstorming process.

Overall, brainstorming is a powerful technique for problem-solving that encourages creativity, collaboration, and diversity of ideas.

Problem

A company is experiencing a decline in sales for its flagship product. The marketing team needs to come up with innovative ideas to boost sales and regain market share.

Solution:

Step 1: Set up a brainstorming session: Gather the marketing team in a conducive environment where everyone can freely express their ideas. Bring materials such as a whiteboard, markers, and sticky notes to capture and organise the ideas.

Step 2: Define the problem: Clearly state the problem to the team, emphasising the need for creative and innovative solutions to boost sales for the flagship product.

Step 3: Generate ideas: Encourage the team to come up with as many ideas as possible, without evaluating or critiquing them at this stage. Let the ideas flow freely, and write them down on the whiteboard or sticky notes.

Some sample ideas generated through brainstorming could be:

1. Launch a limited-time promotion or discount for the flagship product to incentivise customers to make a purchase.
2. Collaborate with social media influencers to promote the flagship product and create buzz.
3. Revamp the packaging and branding of the product to make it more appealing to customers.
4. Offer a buy-one-get-one-free deal or a bundle package to encourage customers to purchase more units of the flagship product.
5. Host a product launch event or a customer appreciation day to create excitement and generate interest in the product.
6. Develop a referral program where customers can earn rewards for referring new customers to purchase the flagship product.

Step 4: Evaluate and prioritise ideas: Once the brainstorming session is complete, review and evaluate the ideas generated. Identify the most feasible and impactful ideas that align with the company's goals and resources. Prioritise the ideas based on their potential to boost sales and regain market share.

Step 5: Create an action plan: Develop a detailed action plan for implementing the selected ideas, including timelines, responsibilities, and resources needed. Assign tasks to team members and set deadlines for each action item.

Step 6: Implement and monitor progress: Execute the action plan and closely monitor the progress. Track the results of the implemented ideas and make

adjustments as needed. Regularly review and evaluate the success of the implemented solutions and iterate the process as necessary.

By using brainstorming as a problem-solving technique, the marketing team can generate a wide range of creative ideas and come up with innovative solutions to address the declining sales of the flagship product.

15.4 Trial and Error

Trial and error is a problem-solving technique that involves attempting different solutions to a problem, observing the results, and learning from the outcomes to refine subsequent attempts. It is an iterative process where various options are tested sequentially or simultaneously until a successful solution is found. The approach is based on the idea that through repeated experimentation, mistakes can be made and corrected, and valuable insights can be gained to eventually arrive at a viable solution.

The trial and error technique typically involves the following steps:

1. Generating multiple hypotheses: The problem solver generates a range of possible solutions or hypotheses to address the problem. These hypotheses may be based on prior knowledge, intuition, or creativity.
2. Implementing solutions: The problem solver implements each hypothesis or solution, often in a systematic and organised manner. This may involve applying different approaches, techniques, or methods to address the problem.
3. Observing and evaluating outcomes: The problem solver observes the results of each trial or attempt and evaluates the outcomes. This may involve measuring the effectiveness, efficiency, or feasibility of each solution and comparing them to the desired outcome or goal.
4. Learning from outcomes: The problem solver learns from the outcomes of each trial and error attempt. This may involve analysing the results,

identifying patterns or trends, and gaining insights into what works and what doesn't.

5. Refining subsequent attempts: Based on the insights gained from the outcomes, the problem solver refines subsequent attempts. This may involve modifying, optimising, or eliminating certain hypotheses or solutions and generating new ones based on the feedback obtained from previous trials.
6. Repeating the process: The trial and error process is repeated iteratively until a satisfactory solution is found or the problem is resolved.

Trial and error can be an effective technique for solving problems in situations where there is no clear or known solution and where experimentation and learning from mistakes are necessary to arrive at a solution. However, it can also be time-consuming, resource-intensive, and may involve risks or consequences associated with unsuccessful trials.

Problem: A farmer wants to build a rectangular pen for his animals using 60 feet of fencing. He has a barn that will serve as one side of the rectangle, so he only needs to fence the other three sides. What are the dimensions of the rectangle that will maximise the area of the pen?

Solution:

Step 1: Let's assume the width of the rectangle is 1 foot. In this case, the length of the rectangle would be 59 feet (since the barn serves as one side). So, the area of the pen would be 59 feet (length) multiplied by 1 foot (width), which is 59 square feet.

Step 2: Now, let's assume the width of the rectangle is 2 feet. In this case, the length of the rectangle would be 58 feet (since the barn serves as one side). So the area of the pen would be 58 feet (length) multiplied by 2 feet (width), which is 116 square feet.

Step 3: We can continue this process by incrementally increasing the width of the rectangle and calculating the area of the pen each time. We can also keep track of the maximum area we have found so far.

After trying different widths, we can see that when the width is 30 feet, the length is 30 feet as well (since the barn serves as one side), resulting in an area of 30 feet (length) multiplied by 30 feet (width), which is 900 square feet. This is the maximum area we have found.

Therefore, the dimensions of the rectangle that will maximise the area of the pen are a length of 30 feet and a width of 30 feet.

15.5 Hypothesis Testing

Hypothesis testing is a statistical technique used in problem-solving to make decisions or draw conclusions about a population based on sample data. It involves the following steps:

1. Formulating a hypothesis: A hypothesis is a statement or assumption about the population being studied. It typically includes a null hypothesis (H0), which assumes that there is no significant difference or effect, and an alternative hypothesis (Ha), which suggests that there is a significant difference or effect.
2. Selecting a significance level: The significance level (also known as alpha) is the probability threshold used to determine whether the observed results are statistically significant or not. It is typically set at 0.05 or 0.01 but can be adjusted depending on the problem and the level of confidence desired.
3. Collecting and analysing data: Data is collected from a sample, which is a subset of the population, and analysed to obtain sample statistics such as mean, standard deviation, etc.

4. Calculating a test statistic: A test statistic is calculated using the sample data and a specific test statistic formula, which depends on the type of hypothesis being tested (e.g., t-test, z-test, chi-squared test, etc.).
5. Determining the critical region: The critical region is the range of values for the test statistic that would lead to the rejection of the null hypothesis. This is determined based on the significance level chosen earlier.
6. Comparing the test statistic with the critical region: The calculated test statistic is compared with the critical region. If the test statistic falls within the critical region, the null hypothesis is rejected in favour of the alternative hypothesis, indicating that there is a statistically significant difference or effect. If the test statistic does not fall within the critical region, the null hypothesis is not rejected, and no statistically significant difference or effect is concluded.
7. Interpreting results and drawing conclusions: Based on the outcome of the hypothesis test, conclusions are drawn about the population being studied. If the null hypothesis is rejected, it suggests evidence in support of the alternative hypothesis, while if the null hypothesis is not rejected, it suggests that there is no statistically significant difference or effect.

Hypothesis testing is widely used in various fields, such as science, medicine, business, social sciences, and more, to make informed decisions and draw conclusions based on sample data while considering the level of confidence and significance desired.

Problem: A restaurant owner wants to determine if a new menu item, a spicy chicken sandwich, is significantly different in terms of customer satisfaction compared to their regular chicken sandwich. The owner collects data from a random sample of 100 customers who tried the new menu item and 100 customers who tried the regular chicken sandwich. The owner wants to use hypothesis testing to determine

if there is a statistically significant difference in customer satisfaction between the two menu items.

Solution:

1. Formulate hypotheses: The first step in hypothesis testing is to formulate the null hypothesis (H0) and the alternative hypothesis (Ha). In this case, the null hypothesis is that there is no significant difference in customer satisfaction between the two menu items, and the alternative hypothesis is that there is a significant difference in customer satisfaction between the two menu items.

H0: The mean customer satisfaction for the spicy chicken sandwich is equal to the mean customer satisfaction for the regular chicken sandwich. Ha: The mean customer satisfaction for the spicy chicken sandwich is not equal to the mean customer satisfaction for the regular chicken sandwich.

2. Choose a significance level: The significance level, denoted as alpha (α), is the probability of rejecting a true null hypothesis. It is usually set before experimenting. Let's say, in this case, the significance level is set at $\alpha = 0.05$, which is a common choice in many statistical analyses.
3. Select an appropriate test statistic and probability distribution: The choice of test statistic depends on the type of data and the specific hypothesis being tested. In this example, since we are comparing means between two groups, we can use a t-test for independent samples. The test statistic will follow a t-distribution with degrees of freedom calculated based on the sample sizes and assuming equal variances.
4. Collect and analyse data: The restaurant owner collected data from a random sample of 100 customers who tried the new menu item and 100 customers

who tried the regular chicken sandwich. The data is analysed to calculate the test statistic and the p-value, which is the probability of observing a test statistic as extreme or more extreme than the one observed, assuming the null hypothesis is true.

5. Make a decision: Using the calculated p-value and the chosen significance level, the decision is made whether to reject or fail to reject the null hypothesis. If the p-value is less than the significance level (p-value < α), then the null hypothesis is rejected, and it is concluded that there is a statistically significant difference in customer satisfaction between the two menu items. If the p-value is greater than the significance level (p-value ≥ α), then the null hypothesis is not rejected, and it is concluded that there is not enough evidence to suggest a significant difference in customer satisfaction between the two menu items.
6. Interpret results: Based on the decision made in step 5, the results are interpreted, and conclusions are drawn. If the null hypothesis is rejected, the restaurant owner may conclude that the spicy chicken sandwich has a significantly different level of customer satisfaction compared to the regular chicken sandwich. If the null hypothesis is not rejected, the restaurant owner may conclude that there is not enough evidence to suggest a significant difference in customer satisfaction between the two menu items.

Hypothesis testing is a commonly used technique in statistical analysis to make decisions based on data and draw conclusions about population parameters. It helps in making informed decisions and drawing valid inferences from sample data to the larger population.

15.6 Reduction

Reduction is a problem-solving technique that involves transforming a complex problem into a simpler and more manageable form while preserving the essential characteristics of the problem. It is commonly used in various fields, including mathematics, science, engineering, computer science, and everyday life problem-solving.

The process of reduction typically involves breaking down a complex problem into smaller, more understandable parts or subproblems and then finding solutions to those smaller parts or subproblems. The idea is that by solving the smaller parts or subproblems, the overall complex problem can be tackled more effectively.

Reduction can be used in several ways to solve problems:

1. Problem simplification: By reducing a complex problem into smaller, simpler parts, it becomes easier to understand and analyse. This allows for a clearer focus on each part and can lead to better insights and solutions.
2. Problem decomposition: Reduction can involve breaking down a problem into smaller subproblems that can be solved independently or sequentially. By solving each subproblem separately, the overall problem can be solved more efficiently.
3. Problem transformation: Reduction can involve transforming a problem into a different representation or format that is easier to solve. For example, converting a problem into a mathematical equation or a graphical representation can simplify the problem and make it more amenable to a solution.
4. Problem abstraction: Reduction can involve abstracting a problem to its most essential elements while omitting unnecessary details. This can help in focusing on the core aspects of the problem, which may lead to more effective problem-solving strategies.

5. Problem mapping: Reduction can involve mapping a complex problem to a simpler or more familiar problem that has already been solved. This can allow for the application of known solutions or techniques to the new problem, saving time and effort.

Overall, reduction is a powerful problem-solving technique that can simplify complex problems, facilitate analysis and understanding, and lead to more effective solutions. It requires breaking down a problem into smaller parts, transforming it into a simpler form, and leveraging known solutions or techniques to tackle the problem more efficiently.

Problem: You are given an array of integers, and you need to find the maximum sum of a subarray with the condition that no two elements in the subarray are adjacent. In other words, you cannot select two adjacent elements from the array to include in the subarray.

Solution:

We can solve this problem using a reduction technique by breaking it down into smaller subproblems. Let's denote the given array as "arr" of length "n". We can reduce the problem of finding the maximum sum of a subarray with non-adjacent elements in "arr" to finding the maximum sum of a subarray for smaller sub-arrays.

We can define a function "maxSum" that takes an array "arr" and its length "n" as input and returns the maximum sum of a subarray with non-adjacent elements. The function "maxSum" can be defined recursively as follows:

1. Base Case:
   - If "n" is 0, return 0.
   - If "n" is 1, return the first element of "arr".
2. Recursive Case:
   - Calculate the maximum sum of a subarray with non-adjacent elements for "arr" of length "n-1" (excluding the last element). Let's denote it as "maxSumExcludeLast".

- Calculate the maximum sum of a subarray with non-adjacent elements for "arr" of length "n-2" (excluding the last two elements). Let's denote it as "maxSumExcludeLastTwo".
- Return the maximum of "maxSumExcludeLast" and "maxSumExcludeLastTwo" plus the last element of "arr".

This approach breaks down the problem into smaller subproblems and solves them recursively. This is an example of how reduction can be used as a technique of problem solving to solve a complex problem by breaking it down into smaller, more manageable subproblems.

15.7 Literal Thinking

Literal thinking, also known as concrete thinking, refers to a problem-solving approach where an individual interprets information or situations based solely on their literal or surface-level meaning without considering any abstract or metaphorical aspects. It involves a straightforward, fact-based, and practical approach to problem solving without delving into deeper implications, nuances, or context.

When using literal thinking as a problem-solving technique, individuals tend to focus on what is immediately apparent and rely on explicit facts and information. They do not engage in abstract or interpretive thinking and may struggle with understanding abstract concepts, metaphors, or symbolic representations.

Literal thinking can be helpful in certain situations, especially when dealing with concrete and straightforward problems that require clear and objective solutions. For example, in tasks that involve following instructions, adhering to rules or guidelines, or solving math or science problems, literal thinking can be an effective approach.

However, literal thinking also has limitations. It may hinder creativity, critical thinking, and the ability to perceive underlying meanings or hidden patterns in complex or ambiguous situations. It may also lead to oversimplification, overlooking important details or ignoring subjective perspectives.

Overall, literal thinking can be a useful problem-solving technique in certain contexts, but it may not be appropriate for all situations. It is important to balance literal thinking with other types of thinking, such as abstract, critical, and creative thinking, to enhance problem-solving skills and achieve more comprehensive solutions.

Problem: You are planning a road trip from Kano to Lagos, which is a distance of approximately 2,800 miles. Your car gets an average gas mileage of 25 miles per gallon, and gas costs ₦3.50 per gallon. You want to budget enough money for gas for the trip. How much money do you need to budget for gas?

Solution using literal thinking:

1. Identify the given information: The distance of the road trip is 2,800 miles, the car's gas mileage is 25 miles per gallon, and the cost of gas is ₦3.50 per gallon.
2. Determine the gas consumption: Divide the distance of the road trip by the car's gas mileage to determine how many gallons of gas will be needed. Using literal thinking, you would simply divide 2,800 miles by 25 miles per gallon:

2,800 miles ÷ 25 miles per gallon = 112 gallons

So, you would need 112 gallons of gas for the road trip.

3. Calculate the total cost of gas: Multiply the number of gallons of gas needed by the cost per gallon to calculate the total cost of gas. Using literal thinking, you would simply multiply 112 gallons by N3.50 per gallon: 112 gallons x N3.50 per gallon = N392.

So, you would need to budget ₦392 for gas to complete the road trip.

Note: Literal thinking involves taking information at face value and using basic arithmetic operations to arrive at a solution. It does not involve complex

problem-solving strategies such as critical thinking, creative thinking, or lateral thinking, which may be required for more complex problems.

15.8 Mean End Analysis

This is a standard method of selecting transition. It is a technique of problem-solving that uses state space. In a state space, each state denotes a space, and each link denotes a transition/move from one state to another. This move/transition is made until a final state, which is the accepted state or goal state, is reached.

The current state is where you are in solving problems, and the goal state is where you want to be. The problem is to find a sequence of transitions/moves that lead from the initial state to the goal state.

Mean End Analysis identifies procedures that cause a transition from the current state to the goal state. Thus, the identified procedure reduces the observed difference between the current state and the goal state. Figure 15.1 describes a state space (Problem space) showing the path P taken from the initial state to a goal state.

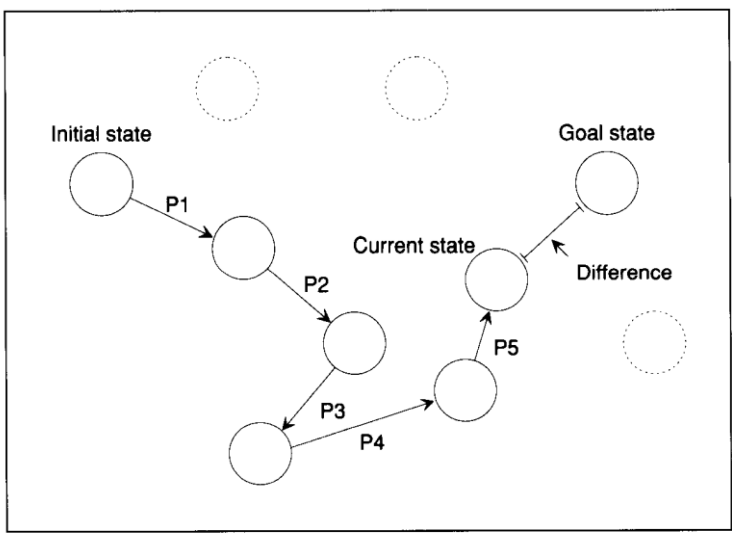


Figure 15.1: A typical State space

The steps for conducting a Mean End Analysis are as follows:

- Until the goal is reached or no more procedures are available
    - Describe the current state, the goal state, and the difference between the two
    - Use the difference between the current state and goal state, possibly with the description of the current state or goal state, to select a promising procedure.
    - Use the promising procedure and update the current state
- If the goal is reached, announce success; otherwise, announce failure

15.9 Method of Focal Length

The method of focal objects is a technique for problem-solving or creative thinking. It involves synthesising the seemingly non-matching characteristics of different objects into something new.

The method of Focal Length has the following steps;

Step 1: Select the object focal point for improvement

Step 2: Select random object focus.

Step 3: List the characteristics of the selected object.

Step 4: Combine the characteristics of the selected object to the object's focal point,

Step 5: Develop the idea from the combination in step 4 based on the free association

Step 6: Evaluation and selection of viable ideas.

Example

Another way to think of focal objects is as a memory cue: if you are trying to find all the different ways to use a brick, give yourself some random "objects" and see if you

can find the application. Given the "blender," for example, I will try to think of all the ways bricks can be used with blender as a lid. Another concept for the brick game: find patterns in the decisions and then break those patterns. If you constantly find ways to build things with bricks, think of ways to use bricks that are not associated with construction. Breaking the stereotypes, combined with a focal object cue, may lead to contradictory decisions. Grind the brick and use it as a pigment.

15.10 Morphological Analysis

This type of problem-solving technique is a sub-discipline of linguistics that deals with the internal structure of words. Consider the following sets of English word pairs:

Verb	**Noun**
Bake	Baker
Eat	Eater
Run	Runner
Write	Writer

In these word pairs, we observe a systematic form–meaning correspondence: the presence of *-ER* in the words in the right column (Noun) correlates with the meaning component "one who Vs," where V stand for the meaning of the corresponding verb in the left column.

The observation of such patterns is the basis for assigning the words in the right column an internal morphological structure [[x]V–@r]N where the variable x stands for the phonological form of the base verb. We thus consider these nouns to be complex words. The morphological schema that generalises over these sets of paradigmatically related words may be formalised as follows:

In morphological analysis the use of the notion “morpheme”, traditionally defined as the minimal meaning-bearing unit of a language. The word *BAKER*, for instance, might be said to consist of the lexical morpheme *BAKE* and the bound morpheme -*ER*. However, the systematic paradigmatic relationships between words may also be signalled by other means than morpheme concatenation, such as stem alternation, reduplication, stress, and tone patterns. Therefore, the notion of "morpheme" is a useful analytic notion for the description of the internal structure of words, but not the starting point of morphological analysis and morphological structure. The two basic functions of morphological operations are word formation and inflexion. Word formation processes create new words and hence expand the lexicon of a language. Inflexion is the grammatical subsystem that deals with the proper form of words in specific syntactic contexts.

15.11 Research

The word “research” originated from the old French word “recerchier", meaning to search and search again. It literally implies repeating a search for something and implicitly assumes that the earlier search was not exhaustive and complete in the sense that there is still scope for improvement. Research, in common parlance, refers to a search for knowledge. It may be defined as a scientific and systematic search for pertinent information on a specific topic/area. In fact, research is an art of scientific investigation. The Advanced Learner's Dictionary of Current English lays down the meaning of research as "a careful investigation or inquiry especially through search for new facts in any branch of knowledge". Redman and Mory define research as "a systematised effort to gain new knowledge". Some people consider research as a movement, a movement from known to unknown. It is actually a voyage of discovery.

Research is a scientific approach of answering a research question, solving a problem or generating new knowledge through a systematic and orderly collection, organisation, and analysis of information with the ultimate goal of making the research useful in decision-making. Systematic research in any field of inquiry involves three basic operations-

1. Data collection: It refers to observing, measuring, and recording information.

2. Data analysis: It refers to arranging and organising the collected data so that we may be able to find out what their significance is and generalise about them.
3. Report writing: It is an inseparable part of the final outcome of a research study. Its purpose is to convey information contained in it to the readers or audience.

15.11.1 Characteristics of Research

Research is a process through which we attempt to achieve systematically and with the support of data, the answer to a question, the resolution of a problem, or a greater understanding of a phenomenon. Figure15. 2: Research Processes: this process has eight distinct characteristics. That Research…

1. Originates with a question or problem.
2. Requires a clear articulation of a goal.
3. Follows a specific plan of procedure.
4. Usually divides the principal problem into more manageable sub-problems.
5. It is guided by the specific research problem, question, or hypothesis.
6. Accepts certain critical assumptions.
7. Requires the collection and interpretation of data in attempting to resolve the problem that initiated the research.
8. It is, by its nature, cyclical, or more exactly, helical.

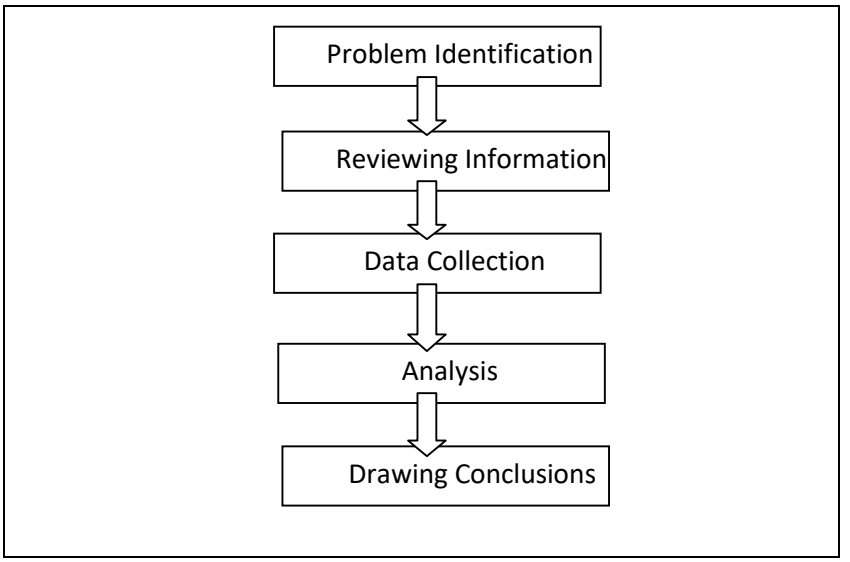


Figure 15.2: Depict a typical research process

15.12 Root Cause Analysis

Root mean analysis Simply written, RCA is a problem-solving technique or a tool designed to help identify not only *what* and *how* an occurrence occurred but also *why* it happened. Only when one is able to determine why an occurrence or failure occurred will one be able to specify workable corrective measures that prevent future proceedings of the type observed.

Understanding why an occurrence occurred is the key to developing effective recommendations. Imagine an occurrence during which an operator is instructed to close valve A; instead, the operator closes valve B. The typical investigation would probably conclude operator error was the cause.

Different researchers have different definitions for RCA. Some of the definitions are: Root causes are specific underlying causes, root causes are those that can reasonably be identified, and Root causes are those the management has control to fix.

15.12.1 Four steps process of RCA

The RCA is a four-step process involving the following:

1. Data collection.
2. Causal factor charting.
3. Root cause identification.
4. Recommendation generation and implementation.

15.13 Proof

Proof is a technique of problem-solving that involve a collection of statements that start with something definitely true and use correct logical steps to reach another truth!

In mathematics, there are many different types of problems. Some will be asked to find the answer to a

question. Some will be to come up with a method that solves a type of problem. And sometimes, some will be asked to prove that a certain statement is true (or indeed false)! The first type of problem is very familiar. Here are some examples:

1. What is the last digit of (9!)?
2. In how many ways can three people choose from 4 different hats to wear?

These types of problems may be easy or even very hard, but we know that what we need is a number at the end. The second type of problem is a bit more complex, which is a combinatorics problem. The third type of problem is what we are dealing with. Examples of a 'proof' question include the following:

1. people are in a class, and some of them shake hands. If we asked each person how many times they shook hands and added all these numbers up, it prove that the final answer is always even.
2. Prove that if we divide any perfect square by 4, the remainder is always 0 or 1.

15.14 Divide and Conquer

The divide-and-conquer solves problems and has a strategy of solving a problem by breaking it into sub-problems that are themselves smaller instances/chunks of the same type of problem, recursively solving these sub-problems and appropriately combining their answers. The real work is done in three different phases: the partitioning of problems (divide phase) into sub-problems; at the very tail end of the recursion, when the sub-problems are so small that they are solved outright (recursively solved small problem); and in the combining of partial answers (conquer phase). These are held together and coordinated by the algorithm's core recursive structure.

15.14.1 Basics of divide and conquer

Recursion: The process in which a function calls itself directly or indirectly is called recursion, and the corresponding function is called a recursive function. Recurrence relation: A Recurrence is an equation or inequality that describes a function in terms of its values on smaller inputs. Recurrence relation solving are methods of solving problems like the substitution method and iteration methods such as the Recursion-tree method and Master method

15.14.2 Divide and Conquer Technique Mechanism

Figure 15.3 describes the mechanism taken to solve a problem using the Divide and Conquer technique.

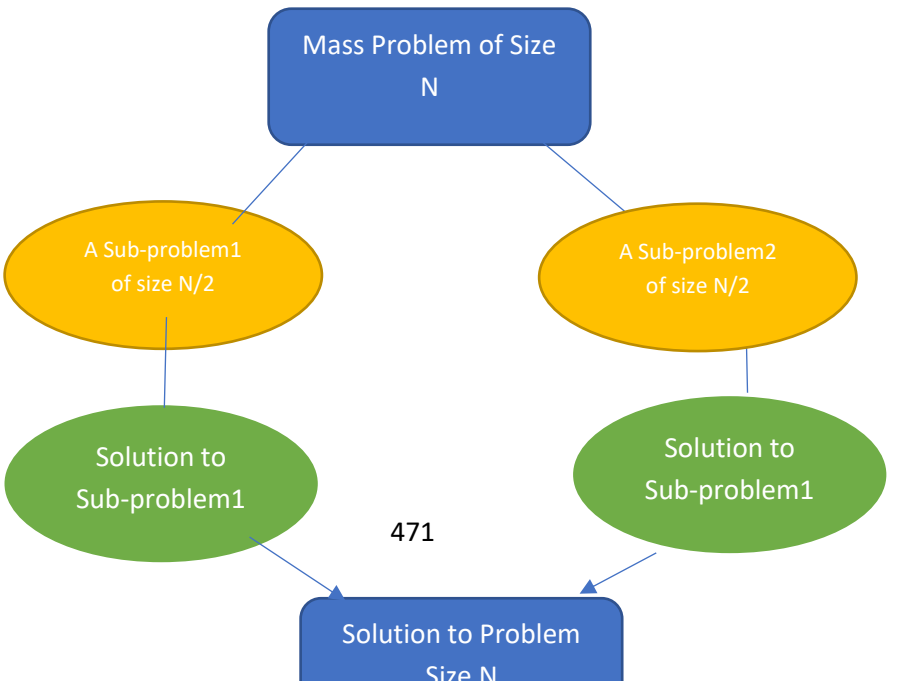


Figure 15.3: Divide and Conquer mechanism

15.14.3 Divide-and-conquer technique example

Divide and Conquer Examples

Sorting: merge sort and quick sort, Binary search, Matrix multiplication: Strassen's algorithm and Convex-hull algorithms

Summary

In this chapter, several techniques for solving problems were discussed. First is the use of abstraction for problem-solving. This technique is used to solve problems by simplifying the problem and creating abstract representations of the problem. With abstract representations, problem solvers can gain a clearer understanding of the underlying issues and then generate creative solutions. Secondly, the use of analogy for solving problems was also discussed. This approach relies on the principle that if two things are similar in some respects, then they are likely to be similar in other respects as well. By using analogy, we can transfer knowledge or solutions from one domain to another. Thirdly, is the approach that involves a group of individuals coming together to collectively generate ideas, insights, and perspectives on a particular problem or challenge, also known as the brainstorming technique. The Trial-and-Error technique was then discussed as an approach that is based on the idea that through repeated experimentation, mistakes can be made and corrected, and valuable insights can be gained to eventually arrive at a viable solution. Fifth, is the hypothesis testing technique that is used to make informed decisions and draw conclusions based on sample data, while considering the level of confidence and significance desired. Sixth is the reduction technique, which requires breaking down a problem into smaller parts, transforming it into a simpler form, and leveraging known solutions or techniques to tackle the problem more efficiently. Next is the literal thinking technique which is a straightforward, fact-based, and practical

approach to problem-solving without delving into deeper implications, nuances, or context. Furthermore, the Mean End Analysis techniques were discussed. This approach identifies procedures that cause a transition from the current state (problem) to the goal state (solution). Morphological analysis is another technique that deals with the use of the internal structure of words in problem solving. Research as a technique for problem solving was discussed as a scientific approach of answering research questions (problems) by generating new knowledge through a systematic and orderly collection, organisation, and analysis of information. The root mean analysis problem-solving technique was then described to solve problems by identifying not only *what* and *how* an occurrence occurred, but also *why* it happened. This will enable the specification of workable corrective measures that prevent future occurrences of such problems. The divide-and-conquer approach for solving a problem has a strategy of solving problem by breaking it into sub-problems that are then recursively solved to give the final result. Problem-solving strategy adopted for a particular problem instance essentially depends on the problem instance characteristics.

Exercises

1. Describe a problem you faced recently and explain how you solved it.
2. How would you approach solving a complex problem with multiple potential solutions?
3. What strategies do you use to break down a problem into smaller, more manageable parts?
4. Give an example of a time when you had to decide under pressure. How did you ensure that your decision was the best one?
5. How do you gather and analyse data when trying to solve a problem?
6. What are some common barriers to problem-solving, and how do you overcome them?
7. Using the analogy problem-solving technique, draw parallels between the assembly line in a manufacturing plant and the pizza delivery process. Identify specific aspects of the assembly line that can be

applied to pizza deliveries and explain how these concepts could lead to more efficient pizza delivery operations.

8. Imagine you have a collection of N unique, unsorted integers, and you want to find the median of these numbers. The median is the middle value in a sorted list. However, you don't have a direct sorting algorithm available. Design an algorithm using the reduction problem-solving technique to find the median of this collection of integers. Explain your approach, including how you will reduce the problem into smaller subproblems and solve them.

9. You have a lock with a 4-digit combination, and you know that each digit is a number from 0 to 9, and each digit can be used more than once. However, you don't know the correct combination. Describe how you would find the correct 4-digit combination through trial and error.

10. You have a jar containing 100 marbles, and each marble is either red or blue. You are blindfolded, and your task is to determine the total number of red marbles in the jar. You can only remove and inspect one marble at a time, and you can't see the marbles. How can you find out how many red marbles are in the jar using the literal thinking approach?

CHAPTER FIVE
Process of Computational Problem Solving

Overview

This chapter explains the steps involved in applying a computer as a tool for solving some problems with the use of a computing system. It contains in-depth exposure to the techniques of studying the problem so as to understand what is needed to arrive at the expected solution. The required input, known as data, and how they are organised, if any, is explained. The transformation that the data will undergo is also discussed. How to determine the preferred computer language to be used in instructing the computer is also explained. Lastly, how the set of instructions is tested is discussed and demonstrated with examples.

Objectives

The objectives of the course are to:

1. explain the analysis of a computational problem;
2. describe data and algorithm used in solving a particular problem;
3. demonstrate how to implement sample programs using Python language;
4. demonstrate how to test sample programs in Python language; and
5. discuss how to test and correct errors in programs.

The Concept of a Problem

Any task that requires a solution is a Problem. A Problem may require a solution that has mathematical content, while the statement of the problem is not purely mathematical, Leslie (2003). The problem in focus in this chapter is a computable Problem. A computable problem can be solved in principle using a computing device and whose steps are concrete, definite and unambiguous Immerman (2021).

Computational Problem solving

Computational problem-solving is the process of providing solutions to problems with the use of computation. This is done using a logical step-by-step procedure. The computational problem-solving process makes understanding of steps involved in

solving a problem easy to understand by others Dierbach (2013). By means of a computer, this logical sequence of steps can be executed as a program.

Solving problems requires sound thinking and care. Understanding the following issues is necessary for providing adequate and effective solutions.

a) Identification of the problem to be solved
b) Understanding the reason for seeking a solution
c) Adequate knowledge of the domain or sector where the problem exists. For instance, working to provide solutions in the health sector requires some knowledge of the health sector. This applies to other sectors as well.
d) Identification of the options available for providing solutions. This may not necessarily require developing a fresh solution. It could simply require modifying the existing solution or changing the platform to conform to the current technological trend.
e) Acquiring the skills needed to provide the required solution. In some cases, it may require identifying individuals who could provide the necessary skills

5.1 Analyse Problem (Analysis)

Problem analysis involves understanding the problem to be solved. This involves a definition of the problem. What problem is there to be solved? An example could be impersonation during major national examinations. This problem must be addressed to ensure the credibility and integrity of results from these examinations.

In addition, another important step in problem analysis is to identify what the solution must do. This is the target. Without identifying what the solution must do, it will be difficult to provide a solution that will address the problem already identified. Using the example of impersonation in national examinations, the solution to be provided must curb impersonation and ensure that the candidates sitting for the examination are those who had registered for the examination. With this, a clear picture of the expected solution has been obtained.

It is also important to identify the stakeholders and the users of the solutions being proposed. The users are a subset of the stakeholders. The stakeholders are those who will benefit from the solution to be provided, while the users are those who will have direct interaction with the proposed solution. Continuing with the example of the national examination impersonation problem, the stakeholders include the national examination body, the candidates sitting for the examinations, the parents, and the governments, among others.

Quick quiz: *Can you think of other stakeholders?*

However, the users include the candidate's writing examination, the administrators, the examination body and perhaps the proctors.

Quick quiz: *Can you think of other users not mentioned here?*

Identifying the users and the stakeholders will help us think about what things are needed to provide a solution to this problem. This will also help us to know from whom we can get those things needed to provide the needed solution. While these individuals might not know what to do, they certainly know there is a problem, and in discussing the problem with them, we can get what is required to provide the solution.

It is now safe to have a mock step-by-step process for solving the problem. This, in other words, is called an algorithm. It is possible to have more than one proposed solution. However, one will adequately care to suit our current situation. Making a choice depends largely on many factors, such as the cost, the existing infrastructure, ease of use, training required and political will to accept the solution.

Quick quiz: *Can you think of three different solutions to the problem of national examination impersonation and one you think is most appropriate based on the factors highlighted above?*

The next step is to verify the solution. This involves following the steps indicated in the algorithm exactly to see if the solution proposed really does solve the problem.

Other things necessary during the problem analysis are

1. a clear statement of the problem
2. the required output
3. what information we have to work with to get the desired output

Strategies to be used to solve problems must be deliberately known because many things that can be classified as problem-solving exercises are done many times daily. However, when programming is involved, conscious efforts are made to formally document the algorithm for achieving them. For instance, many times a day, foods are cooked. Doing this requires following a recipe.

5.1.1 Clearly Understand the Problem

This requires that some questions are answered to be able to understand the problem to be solved. These questions are usually asked until the problem to be solved is clearly understood. The following questions might be needed to understand the problem to be solved.

i. What do I have to work with—that is, what kind of data?
ii. What do the users look like?
iii. What tasks do the users perform on their data?
iv. How much input is there?
v. How do I know if I have the kind of input required to solve the problem?
vi. What should my output look like?
vii. How do the various tasks work together to solve the big problem?
viii. What special error conditions might come up?

5.1.2 Know what Constitute a Solution

There are different ways to get a solution to the problem. However, it is important the solution proposed clearly and unambiguously addresses the problem. While there might not be a perfect solution, the solution proposed must address the problem presented by the stakeholders.

It is important, however, that there is no need to reinvent the wheel. Therefore, if the solution already exists, it is safer to use it, except it is majorly deficient. The solution might also require just providing a sub-task and plugging it into the existing solution, thus solving the problem. In other instances, a bigger problem might be sliced into bits or divided into smaller problems and get solutions to them. These might, in turn, be brought together for the big solution.

In summary, a solution must address the problem identified by either the programming team or the stakeholders themselves. The solution must not create another problem for the users and stakeholders. An effective solution must not take away the users from their normal processes. This means that the users do not need unnecessarily long training to be able to use the solution. The solution provided should not require daily maintenance efforts by the programming team. This will put additional costs on the stakeholders.

5.2 Describe Data and Algorithm (Design)

Data and algorithms are essential to providing a solution to an identified problem. Data, the input values, when processed, will determine what output will be derived. These must be properly described to reflect the true picture of the data needed in order to generate a correct output. Algorithm is defined simply as unambiguous instructions for solving a problem. An important component of an algorithm is the input statement through which the data will be passed. Thus, both are to be properly described for optimal performance of the solution proposed.

5.2.1 Determine what Data is Needed

Determining the data needed requires an understanding of the process involved in the problem to be solved. Each task must be clearly understood, and the kind of input that goes into the process must be identified. In addition, the kind of output expected must also be understood. Generally, data is a value used in computation. This is held as a variable or label. The data type must be understood. A data type is a set of data with predefined values. Examples of data types are integer, floating point, number, character, and string, among others. Using the example of the national examination impersonation problem, if the solution is to use a fingerprint along with the candidate's biodata, it is incumbent that the fingerprint is captured

and stored as a fingerprint template, which is a digitised version of the fingerprint. Determining this will help to make adequate preparation for the skills and tools required to capture this kind of data. The way this is captured will be different from the way names, date of birth and other data will be captured. Therefore, the kind of data needed will be determined by the output required and the processes to be performed.

5.2.2 Determine how Data is to be Structured

Data structure is a particular way of storing and organising data in a computer so that it can be used efficiently. It is a special format for organising and storing data. General data structure types include arrays, files, linked lists, stacks, queues, trees, graphs and so on. Data are stored in a file, and multiple related files are called databases. In a relational database, files are stored as tables. This implies that data are inserted into a table.

Generally, data structures are classified into two types. These are:

i. **Linear data structures***:* In this type of structure, elements are accessed in a sequential order, but it is not compulsory to store all elements sequentially. *Examples:* Linked Lists, Stacks and Queues.

ii. **Non–linear data structures:** This type of structure allows elements of this data structure to be stored/accessed in a non-linear order. *Examples:* Trees and graphs.

The implication of this is that, without a proper understanding of how data in the proposed solution will be stored and accessed, it will be difficult to provide an effective solution. Therefore, from the beginning, how data is to be structured should be determined, basically involving how data is to be stored and accessed.

5.2.3 Find and/or Design Appropriate Algorithm

An algorithm is the step-by-step unambiguous instructions to solve a given problem. This could be expressed as a pseudocode or flowchart. A pseudocode is an English-like statement that shows step-by-step instructions

for solving a problem. The flowchart is a diagrammatic representation of such steps. The algorithm can be written in the following format.

Start

Input statement

Processing statements

Output statements

Stop

For an algorithm to be appropriate, it must be unambiguous, easy to understand and clear. Running through the algorithm must lead to a solution to the problem. It must not be terminated until the problem is solved. The output to be produced must also be the desired output. For instance, continuing with the national examination impersonation problem.

Start

Open the UTME20XX candidate file

Read *candidate biodata*

Take candidate fingerprint

Match the biodata and fingerprint with the ones in the database

If *match found*

    Candidate = "accepted"

Else

    Candidate = "rejected"

Stop

The above is a simple algorithm. In some instances, the algorithm might be more complex. However, it should lead to the solution to the problem if it must be accepted.

Quick quiz: *Replicate the above algorithm using the flowchart*

5.3 Implement Program (Implementation)

An algorithm designed for a particular task must be implemented using a desired programming language. This implementation involves the translation of the algorithm into the programming language that the computer understands. For any program to be used to implement a task, there must be a Compiler or Interpreter for it. Hence, a Python Program that has been written must be interpreted by Python Interpreter before the computer can perform the tasks specified in the Program.

5.3.1 Requirements for Python Program Implementation

In order to implement a program written in Python, the first step is to ensure that the Python Interpreter is installed in the computer system to be used. The interpreter can work in many operating systems, such as Windows, Unix, Linux, Mac OS and in virtual machine environments like Dot Net. Since Python is an interpreted language, it saves a lot of time when developing programs because there will be no need to intermittently compile and link programs when testing is being carried out on the written code.

5.3.1.1 Python Language Family

Python Interpreter is not proprietary (ownership) software. Hence, it can be freely downloaded from the internet and installed on the computer. The latest version of Python interpreter can always be obtained by googling the internet. In order to write programs faster and easier, it is recommended to always use an Integrated Development Environment (IDE). An IDE is a bundle of components that allows the programmer to write/edit programs, compile, test, and correct errors while in the process of development. Some of the notable IDEs for Python are as follows: IDLE, Python for VS Code, Eric, PyDev, Thonny, PyCham, Spyder, PyScripter, Wing, and NetBeans.

5.3.1.2 Modes of Running Python Interpreter

After the installation of a Python interpreter on the Computer, It can be activated in any of these three ways: Immediate Mode, Script Mode, and Integrated Development Environment Mode. In the Immediate Mode, Python as an instruction is first typed in the command line to invoke the interpreter in immediate mode. Python code is then directly typed in line by line, and the execution is implemented immediately after the line. In the Script Mode, Python code is stored inside a text file known as a script. The text file is then run by the interpreter. This text file is a special Python file identified by the file extension name 'py'. In an Integrated Development Environment Mode, the job of the programmer is made very easy. Hints for possible codes to be inserted or corrected are shown. An IDE removes a lot of redundant efforts and speeds up the time of program development through the provision of a lot of assistance.

5.3.2 Representation of Data within Programming Language

Problem-solving with the use of the computer involves representing the needed input known as data within the computer program. This program is a set of codes written in a particular language that is understood by the computer. There are a lot of programming languages that have been designed and implemented to be used by the computer when solving a particular problem.

Many programming languages are designed for problem-solving because of the diverse nature of problem areas (domains) and the specific needs of each problem. When choosing a particular programming language, considerations are made in terms of the uniqueness of the language in solving the problem at hand. Decisions about choosing a language may also be made as a result of the general acceptability of the programming language and ease of use.

Data to be used in programming languages are often of different types. Each type has a way it must be represented in the language. Our focus in this chapter will be on what can be used to stand in place of data that is used in the Python programming language.

5.3.2.1 Python Data Representation

Data are represented in programs by what are called variables. Variables can be likened to computer memory locations where items are stored. In order to be accessible for retrieval and storage of items, they are given names. These names are called variable names. In order to store data in a memory location, the data is assigned to the variable representing the memory location.

5.3.2.2 Features of a variable

There are two things that are important in variables: the name and the content. The name is what is used to identify the variable, while the contents are the items stored in the variable. In Python Programming Language, a variable is recognised immediately after it is used. There is no need to state that a particular variable will be in use within the program before using it. This is referred to as the declaration of the variable. In Python, variables are automatically declared the first time they are used in a program.

Examples of data representation using variables:

Example 1: Python code for representing **University of Medical Sciences Ondo** with variable name **university_name**

Solution 1:

```
university_name = "University of Medical Sciences Ondo"
```

Example 2: Representation of **University of Abuja** with variable name **university_name**

Solution 2:

```
university_name = "University of Abuja"
```

Example 3: Representation of a student's age as **20**with variable name **student_age**

Solution 3:

student_age = 20

Example 4: Representation of a student's age as **19** with variable name **student_age**

Solution 4:

student_age = 19

Example 5: Representation of a student's state of origin as **Kano** using variable name **state_of_origin**

Solution 5:

state_of_origin = "Kano"

Example 6: Representation of a student's state of origin as **Ondo** using variable name **state_of_origin**

Solution 6:

state_of_origin = "Ondo"

5.3.3 Types of Variables used for data representation

The types of variables that are available are Constant Variables, Global Variables, Class Variables, Instance Variables, and Local Variables. Each of these variables has its own usage and scope that can be used or made available for manipulation.

Constant Variables are used to represent data that will not change during the course of executing the program. Global Variables are used to represent data that can be made available for use by other subprograms (functions/methods) in the whole program. Class Variables are those that are available for use only within the class (entity) that was specified or declared. Instance Variables are those that are recognised within a class and will always be made available the moment a member of the class is available or declared.

5.3.3.1Rules for Representing Data Using Variables

Some of the rules are:

1. Variable can be letters in either lowercase or uppercase or a combination of the two.
2. Variable can be a combination of letters and digits.
3. Variable cannot start with a digit.
4. Variable can have an underscore (_) like university_name.
5. Variable cannot bear the same name as a Python Keyword.
6. Variable cannot have a special symbol as part of its name like @, #, % $.
7. Variable name is expected to indicate what it is expected to contain. Example, **age**, **salary**, **university name**, **height**. Although this is not compulsory, it is a good programming style.
8. Variable representation should not contain embedded spaces.

5.3.4 Implementation of Algorithms in Programming Language

Algorithms are written in general formats so that the implementation can be done using any chosen Programming Language. Each step of an algorithm is converted into its equivalent Programming Language.

Examples of some Algorithms and their implementation

Example 1: Write an Algorithm to accept two numbers, multiply the first by the second and display the result.

Solution 1:

Start the algorithm

Accept the first number into number1

Accept the second number into number2

Multiply number1 by number 2 and store the result in number3

Display number 3

Stop the algorithm

Example 2: Write an Algorithm to accept the height of a boy in meters, convert it to centimetres and display the result.

Solution 2

:

Start the algorithm

Accept the height of boy into height_meters

Multiply height_meters by 100 and store into height_centimeters

Display height_centimeters

Stop the algorithm

Example 3: Write an Algorithm to accept the name of a student, matriculation number, faculty, and department, and display each item on a single line.

Solution 3:

Start the algorithm

Accept the name of the student into the student_name variable

Accept the matriculation number of the student into student_matric variable

Accept the faculty name of the student into student_faculty variable

Accept the department name of the student into student_department variable

Display student_name and advance to next line

Display student_matric and advance to next line

Display student_faculty and advance to next line

Display student_department

Stop the algorithm

Example 4: Write an Algorithm to solve a quadratic equation ($ax^2 + bx + c = 0$) by accepting coefficients a, b and c.. Apply the quadratic equation formula and display the two results.

Solution 4:

Start the algorithm

Accept the coefficient 'a' into 'a' variable

Accept the coefficient 'b' into 'b' variable

Accept the coefficient 'c' into 'c' variable

Multiply b by b and store into b_square

Multiple 4 by a by c and store into temp1

Subtract temp1 from b_square and store into temp2

Compute the square root of temp2 and store into temp3

Subtract b from temp3 and store into temp4

Multiply a by 2 and store into temp5

Divide temp4 by temp5 and store into solution1

Subtract b from temp3 and store into temp6

Subtract temp6 from 0 and store into temp7

Divide temp7 by temp5 and store into solution2

Displaysolution1 and advance to next line

Displaysolution2

Stop the algorithm

5.4 Testing and Debug (Testing)

Any algorithm developed can be translated into a Programming Language by converting the steps into the equivalent codes of the language. The order of the steps in the algorithm is of utmost importance.

5.4.1 Testing of Programs on a selected set of Problem Instances

The algorithms for the problems stated in examples 1 through 4 of section 5.3.2 above will be tested in this section. The testing of the solutions to these problems will be carried out using a Python IDE called IDLE. The first stage is to activate the IDLE, which will make the functions of the Python interpreter available. The testing will be done using the Script Mode of executing Python Programs. With this approach, each set of codes for the solution of each example problem instance will be stored in a script. Example 1 will be stored in example1.py, example2 in example2.py, example3 in example3.py and example 4 in example4.py.

Example 1:

Algorithm to accept two numbers, multiply the first by the second and display the result.

The suggested code is given below, and the output is depicted in Figure 5.1.

Python Test Code

#Accept the first number into number1

number1 = input('Enter first number: ')

Accept the second number into number2

number2 = input('Enter the second number: ')

#Multiply number1 by number 2 and store the result into number3

number3 = int(number1) * int(number2)

#Display number 3

print('The result of multiplying {0} by {1} is {2}'.format(number1, number2, number3))

Output 1:

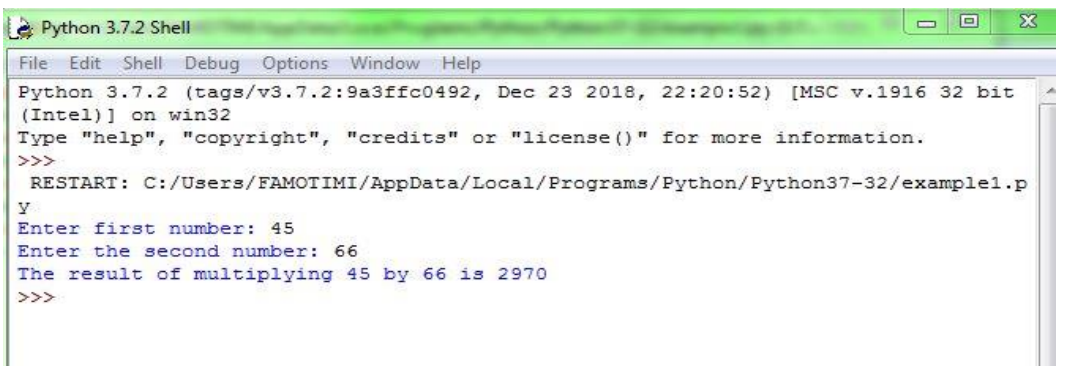


Figure 5. 1: Output of a code multiplying two input numbers and displaying the result

Example 2:

Algorithm to accept the height of a boy in meters, convert it to centimetres and display the result.

The suggested code is given below, and the output is depicted in Figure 5.2.

Python Test Code

```
#Accept the height of boy into height_meters
height_meters = input('Enter height of boy: ')
#Multiply height_meters by 100 and store into height_centimeters
height_centimeters = float(height_meters) * 100
#Display height_centimeters
print('The height in centimeters is  {0}'.format(height_centimeters ))
```

Output 2:

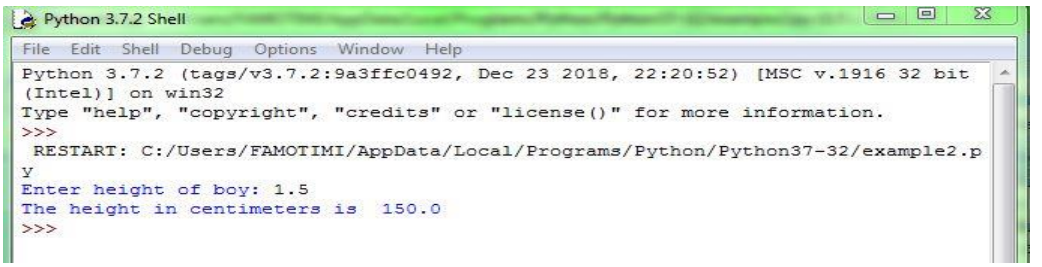


Figure 5. 2: Output of a code that converts metres to centimetres, displaying the result

Example 3: Write an algorithm to accept the name of a student, matriculation number, faculty, and department, and display each item on a single line.

The suggested code is given below, and the output is depicted in Figure 5.3.

Python Test Code

```
#Accept the name of the student into student_name variable

student_name = input('Enter the name of the student: ')

#Accept the matriculation number of the student into student_matric variable

student_matric = input('Enter the matriculation number of the student: ')

#Accept the faculty name of the student into student_faculty variable

student_faculty = input('Enter the faculty name of the student: ')

#Accept the department name of the student into student_department variable

student_department = input('Enter department name of the student: ')

#Display student_name and advance to next line

print('The student\'s name is  {0} '.format(student_name))

#Display student_matric and advance to next line

print('The student\'s matriculation number is  {0} '.format(student_matric))

#Display student_faculty and advance to next line

print('The student\'s faculty name is  {0} '.format(student_faculty))

#Display student_department

print('The student\'s student_department name is  {0} '.format(student_department))
```

Output 3:

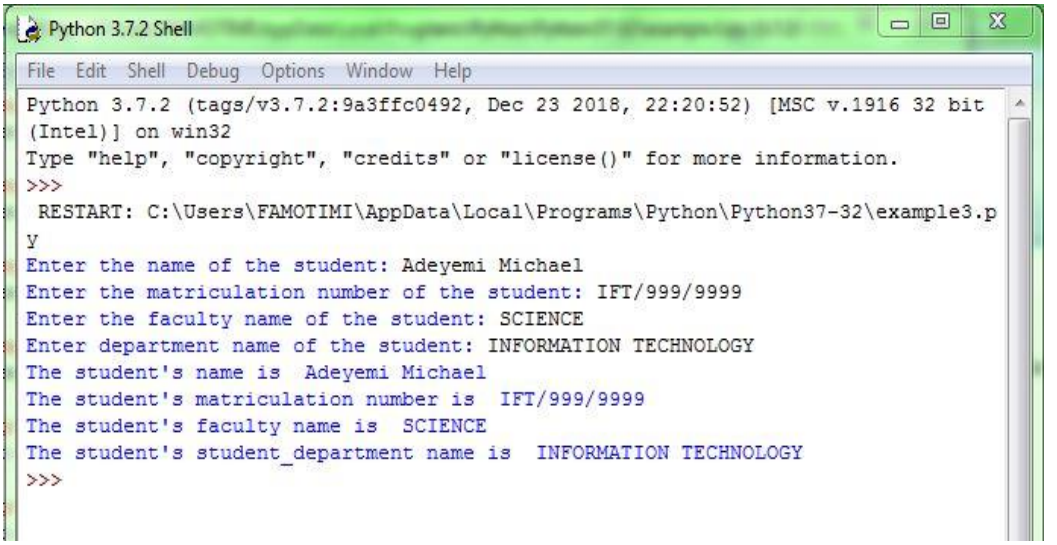


Figure 5. 3: Output of a code that accepts the name of a student, matriculation number, faculty, and department, then displaying each item on a single line

Example 4: Algorithm to solve a quadratic equation, ($ax^2 + bx + c = 0$), by accepting coefficients a, b and c.. Apply the quadratic equation formula and display the two results.

The suggested code is given below, and the output is depicted in Figure 5.4.

Python Test Code

```
quadratic_equation.py - C:/Users/FAMOTIMI/AppData/Local/Programs/Python/Python37-32/quadrat...
File  Edit  Format  Run  Options  Window  Help
# Solve the quadratic equation ax**2 + bx + c = 0
# import complex math module
import cmath
#a = 1
#b = 5
#c = 6
# To take coefficient input from the users
a = float(input('Enter a: '))
b = float(input('Enter b: '))
c = float(input('Enter c: '))
# calculate the discriminant
d = (b**2) - (4*a*c)
# find two solutions
sol1 = (-b-cmath.sqrt(d))/(2*a)
sol2 = (-b+cmath.sqrt(d))/(2*a)
print('The solution are {0} and {1}'.format(sol1,sol2))
```

Output 4:

```
Python 3.7.2 Shell
File  Edit  Shell  Debug  Options  Window  Help
Python 3.7.2 (tags/v3.7.2:9a3ffc0492, Dec 23 2018, 22:20:52) [MSC v.1916 32 bit
(Intel)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
 RESTART: C:/Users/FAMOTIMI/AppData/Local/Programs/Python/Python37-32/quadratic_
equation.py
Enter a: 1
Enter b: 5
Enter c: 6
The solution are (-3+0j) and (-2+0j)
>>>
```

Figure 5. 4. Output of a Python code that solves a quadratic equation,

($ax^2 + bx + c = 0$) and displays the result

5.4.2 Correcting and Understanding the Causes of any Error Found

When implementing a Python program, at times, it may not run successfully as a result of some errors. There are many causes of these errors, such as data type mismatch or not following some basic principles of the language.

Programs may be corrected easily when using a good Integrated Development Environment. This tool will highlight the area where the error occurs. With this hint, the bug can easily be fixed, and the program can be re-run.

Summary

This chapter has explained the analysis of a computational problem. It discussed the algorithms used in applying computers as a tool for solving some problems with the use of a computing system. Various techniques that can be adopted for studying problems and thinking through the expected solution were discussed. How data can be organized, if any, is explained. The chapter also explained how to select the preferred computer language to be used in instructing the computer. Lastly, there was a demonstration of the testing of the program codes to verify they are performing the expected functions using some examples.

Exercises

1. (a) Write an algorithm that will accept a number, compute the cube root and display the result

   (b) Implement the algorithm in 1 above using Python Language

2. (a) Write an algorithm that will convert Fahrenheit to Celsius using the formula

C = (5 / 9) x (F – 32) where F is Fahrenheit and C is Celsius

   (b) Implement the algorithm in 2 above using Python Language

3. (a) Write an algorithm that will accept an integer (n) and compute the value of $n * n^2 * n^3$

   (b)Implement the algorithm in 3 above using Python Language

4. (a) Rewrite the program below by fixing the errors

```
a = (input('Enter a: '))
```

```
b = a * a
print('The square of a is  {0} '.format(b))
```

CHAPTER SIXTEEN
Problems in Daily Life, Problem Analysis, Concepts, and Types of Problems

Overview

This chapter is dedicated to exploring the common challenges that individuals encounter in everyday life, introducing the fundamental principles of problem analysis, and discussing various types of problems. The main objective of this section is to give a comprehensive definition of what constitutes a problem, identify the common issues faced in everyday life, and offer insights into effective problem-solving strategies, including an introduction to computational problem-solving techniques. After finishing this section, readers ought to have a deeper comprehension of the various types of problems they may encounter and be equipped with the necessary knowledge and tools to effectively analyze and solve them.

Objectives

By the end of this chapter, students should be able to:

i. state what constitutes a problem.
ii. identify at least four common problems people face in life.
iii. categorize problems into different types.
iv. explain at least five different methods for solving problems.
v. understand the computational problem-solving techniques.

16.1 What is a problem?

Life is full of problems, and in fact, problems are inevitable in life. A problem is a type of difficulty encountered as one goes through life from the day of birth to the day of death. Problems can be anything on the way to achieving the desired objectives. Some of these problems might be easy to solve, while others are very difficult to

solve. Some problems can be inconsequential, while others can be more substantial and need a more in-depth analysis to solve.

The most common problem among university students is procrastination. Procrastination is the act of deferring or suspending a job or activity that needs to be executed immediately. The struggle with procrastination emanates from several assignments or projects to be finished, and the student may be overwhelmed or unsure of where to start and where to end the process. For instance, a student may have an assignment to submit in a few weeks but keep postponing the work until the last minute. This may lead to stress and serious problems as the day of submission of the assignment approaches. It may also lead to the submission of a poorly done assignment. The student may also develop guilt problems for not doing justice to the assignment early, which may lead to poor self-esteem and motivation.

In computer science, a problem can be said to be a numerical, analytical, or algorithmic job that requires to be resolved or optimized. The problem can either be simple or complex and can comprise diverse fields of computer science, such as data structures, algorithms, programming, and software engineering. Some of the problems may involve sorting, Searching, Optimization, Machine learning, and or Network optimization. Then, what is a problem?

A problem is a condition or challenge that demands a solution. It includes an impediment or obstacle that needs some type of action or intervention to be addressed or solved. Problems may emanate from diverse settings, such as personal, professional, social, or academic, and may be complicated and brutal. Section 3.4 discusses the initial step in the problem-solving process, which is identifying and defining the problem.

16.2 List Problems in Daily Life

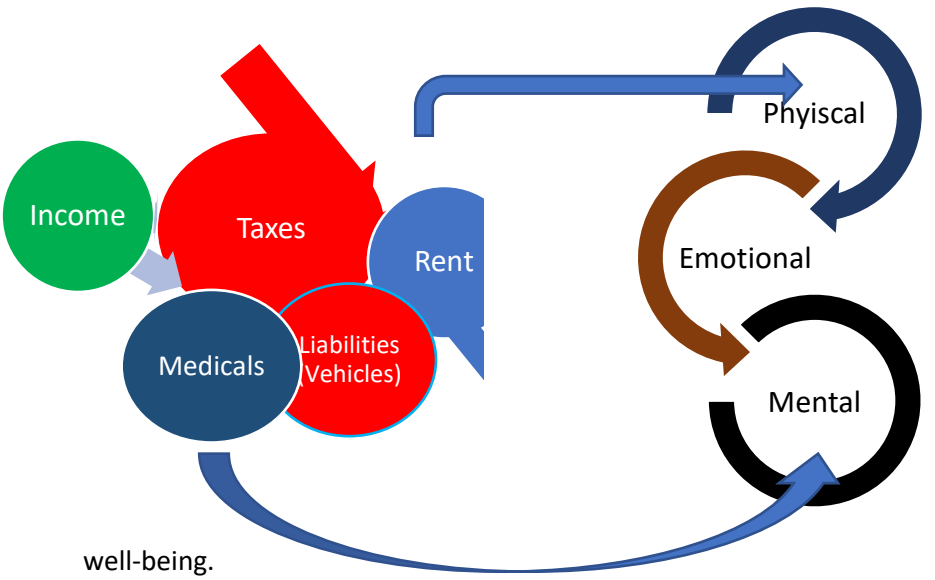
The list of problems in life is unending. However, it ranges from personal problems, financial problems, social problems, career problems, health problems, relationship problems, environmental problems, and legal problems to spiritual problems.

The contents of each of the problems are enormous.

The problems are interconnected in diverse ways because they all affect different aspects of an individual's life. Some ways in which problems are interrelated are discussed seriatim:

1. **Financial problems** may cause trauma and nervousness, which can cumulate into relationship problems, health problems, and even career problems.
2. **Relationship problems**, such as skirmishes with a spouse or family member, can result in stress, depression, and other mental health issues. These problems can then affect an individual's capability to accomplish at work and influence their career.
3. **Health problems**, any type of health challenge, whether physical or mental, can have an effect on an individual's finances, relationships, and career. For instance, a chronic illness may result in absenteeism from work and medical bills, resulting in financial stress and possibly leading to relationship problems.
4. **Career problems**, such as loss of job or discontentment with work, can result in financial problems, relationship problems, and even health problems owing to the trauma and anxiety they cause.
5. **Personal problems**, such as low self-confidence or addiction, can influence an individual's relationships, career, and even physical health.
6. **Social problems**, such as loneliness or discrimination, may influence an individual's mental health, relationships, and career opportunities.
7. **Environmental problems**, such as contamination or climate change, may influence an individual's health, financial stability, and even personal beliefs and values.
8. **Legal problems**, such as lawsuits or criminal charges, may have a vital influence on an individual's finances, relationships, and career prospects.

9. **Spiritual problems**, such as a predicament of faith or lack of focus, may influence an individual's mental health, relationships, and overall sense of well-being.



In all, these glitches can interrelate with each other and generate a complex web of problems that may be hard to navigate. Figure 6.1 illustrates how financial stress can impact an individual's physical and mental health. Individuals' physical, mental, and emotional stability determines their ability to get income and cope with medical, rent, school fees, and other liabilities such as vehicular problems.

Figure 16.1: Impact of Financial and Health Problems

Figure 16.2 is connected by exploring how personal issues, such as self-esteem or communication skills, trust, and intimacy, can influence relationships, self-care, personal development, and personal growth. Developing a plan to address those personal issues can improve the health of the relationship.

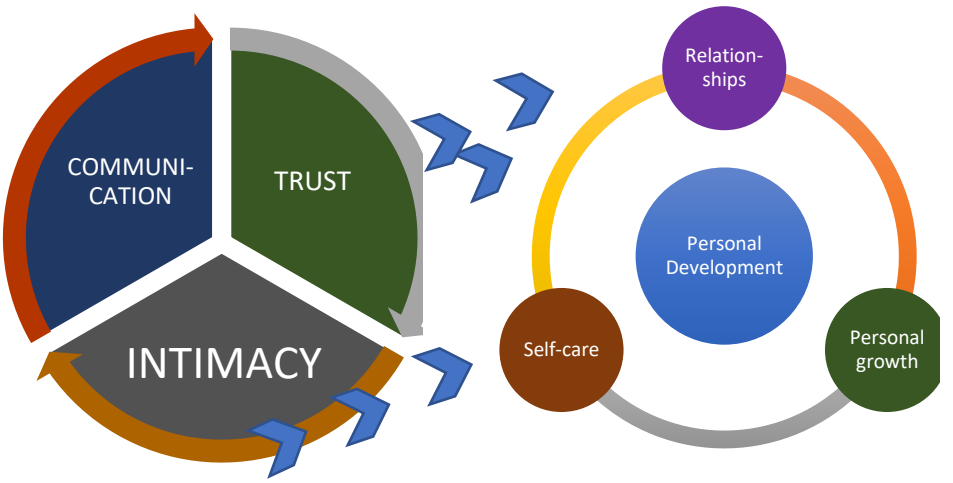


Figure 16.2: Relationship and personal problems

Figure 16.3 is connected by exploring how social and environmental issues are interconnected, such as how pollution affects public health, education and so on. Developing a plan to address both the social and environmental aspects of the problem through community action, such as advocating for policy change, conserving resources and reducing waste, can go a long way to solving the problem.

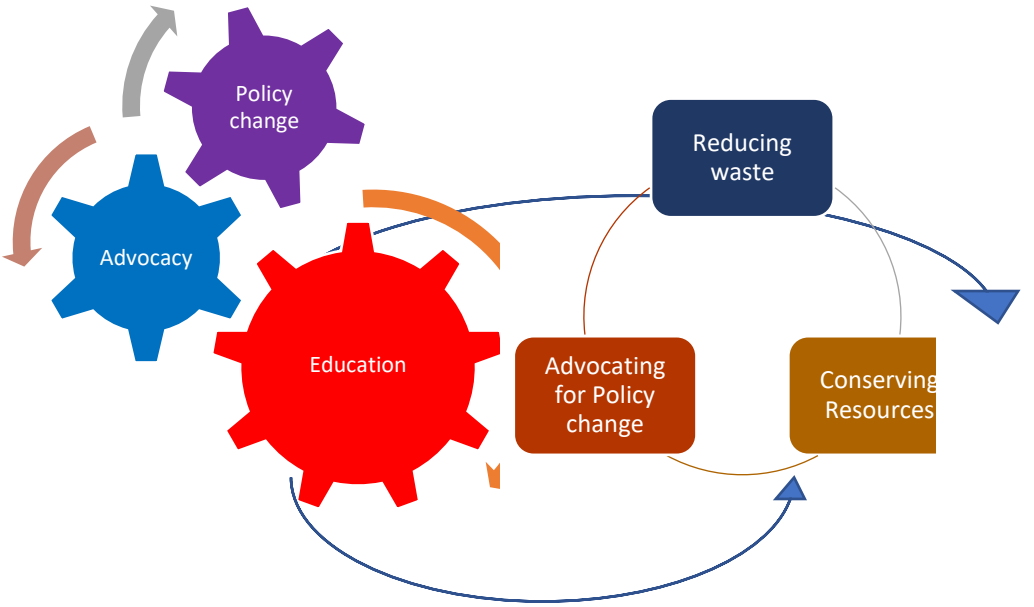


Figure 16.3: Social and environmental problem

Figure 16.4 is connected by exploring how legal issues, such as discrimination or harassment, can influence an individual's career prospects. Developing a plan to address both the legal and career aspects of the problem, such as seeking legal recourse or pursuing training and networking opportunities, can go a long way in solving the problem.

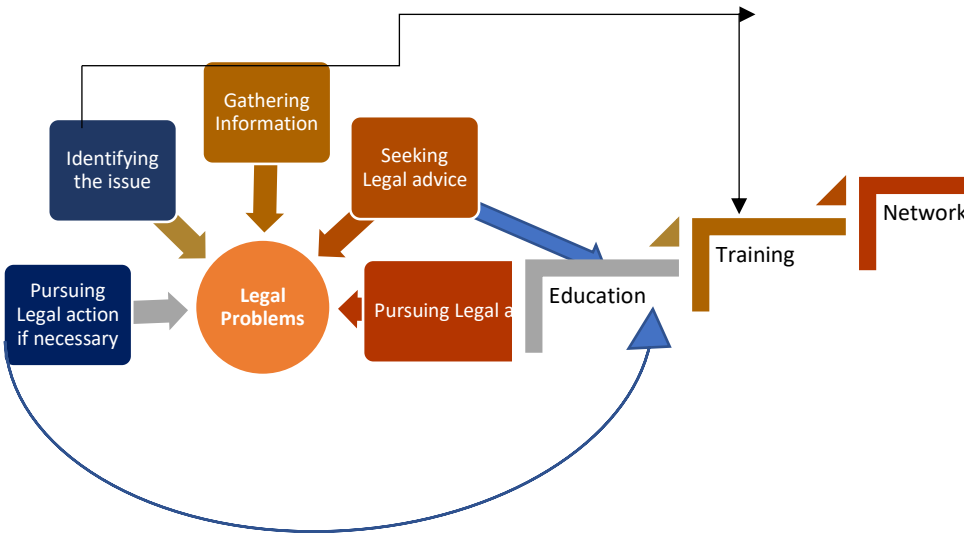


Figure 16.4: Legal and career problems

It is good to note that all the problems in Figures 3.1 to Figure 3.4 are interrelated to give an infinite list of problems in daily life that seek solutions. Understanding the enormity of the problem that is being discussed requires connecting the Figures. This would involve identifying how different aspects of the problem are interconnected and developing a plan to address those interconnected issues in a comprehensive way. It is also imperative to address these problems cautiously and look for help when required as a guide from getting out of control.

16.3 Types of Problems

There are diverse types of problems, each with its distinct features and how to solve them. Two main groups of problems are routine problems and non-routine problems.

1. Routine problems
2. Non-routine problems

16.3.1 Routine problems

These are problems that are understandable and have definite ways to solve them. These problems are not strange and can be solved through known rules, algorithms, or formulas. Routine problems are not difficult to solve and do not need a lot of critical thinking or ingenuity to solve. Examples of these problems are simple math problems, spelling tests, and basic grammar exercises.

16.3.2 Non-routine problems

Problems that are non-routine problems are problems that cannot be easily understood and do not have definite ways to solve them. These problems are strange and need critical reasoning, ingenuity, and an analytical mind to solve. In this type of problem, there are several answers to one question, and the solution may be different depending on who is solving the problem and the method used. Instances of these complex issues encompass mathematical problems, scientific experiments, and real-world challenges like the issue of global warming. It is good to still note that both routine and non-routine problems can again be classified into solvable problems or unsolvable problems:

Solvable problems: These are understandable problems with definite solutions. Solutions can be found to the problems through logical thinking, analytical reasoning, or ingenious problem-solving aptitude. Typically, these problems are routine in nature, but occasionally, they may also present as non-routine issues with clear and comprehensible solutions. There are many examples of solvable problems, such as mathematical problems, logical problems, technical problems, analytical problems, strategic problems, social problems, and personal problems.

Unsolvable problems: These are non-understandable problems without definite solutions. Unsolvable problems have no definite method of solving them and maybe presently beyond scientific knowledge or technology. Examples of such problems are metaphysical questions such as purpose or meaning to life, certain mathematical problems such as squaring the circle, and some physical problems such as perpetual motion machines.

In mathematics and computer science, examples of problems considered unsolvable are the halting problem (given a computer program and its input, determine whether

the program will eventually halt or run forever), the busy beaver problem (given a set of rules for a Turing machine, determine the maximum number of steps the machine can perform before halting) and the Post correspondence problem (given a set of dominoes with strings on each side, determine whether it is possible to arrange them in a sequence such that the strings on the top and bottom of the sequence match).

16.4 What is Problem Solving?

Problem-solving is the method of recognizing, investigating, and resolving a problem in order to accomplish an anticipated outcome or goal. It includes the application of reasoning, critical thinking, and imagination to recognize and appraise possible solutions and then choose the most active solution or course of action. Effective problem-solving necessitates the ability to accurately define the problem, gather relevant information, generate and assess potential solutions, and subsequently implement and monitor the chosen solution. Problem-solving can be useful in several contexts, from personal and interpersonal issues to complex organizational and societal encounters.

A common problem in a university is a student facing the problem of how to balance their coursework and extracurricular activities. This may lead to the student being overwhelmed and stressed.

In problem-solving, the next thing is to recognize, investigate, and resolve the problem in order to accomplish an anticipated outcome or goal. Here is a common step for solving this type of problem.

Assess the Situation: First, the student needs to look critically at the problem, assess the present situation, and identify the particular area causing the stress. The student also needs to make a list of all their coursework and extracurricular activities, along with any deadlines, meetings, or other commitments.

Prioritize: Secondly, the student's activities can be ordered based on their significance and urgency. The student should do a self-examination to know what assignments are due soonest. What activities are most important to the individual? What can the individual realistically handle?

Create a Schedule: A schedule or planner can be created to outline when the student will work on each task once the priorities are identified by the student. Hence, a digital calendar or a paper planner can help to map out the days and weeks' activities.

Set Goals: A set of precise, quantifiable goals for each task or activity ought to be set by the student. For instance, as an alternative to just writing, "Study for the computer science exam," the student can break the task down into smaller, more manageable goals like "Read chapters 1-3," "Create flashcards for key terms," and "Practice sample questions."

Take Action: The plan notwithstanding, the student needs to take action. Work on each job or activity based on the schedule and trail their progress towards their goals. There should also be room for flexibility and adjustment of the plan as the situation demands. The student can also be flexible and adjust the plan as needed based on unexpected circumstances or changes in their priorities.

Seek Support: The student should not keep the problem but solicit assistance from peers, professors, or mental health professionals if the student still feels overwhelmed or stressed. The student can discuss this with the appropriate authority or quarter, such as their professors, about extensions or accommodations if necessary, or seek out funds on campus like tutoring or counselling services.

With these steps, the student can easily find the solution to the problem of balancing the coursework and extracurricular activities and avoiding being overwhelmed and stressed.

From the above, it is obvious that Problem-solving is a procedure for solving a particular problem that requires some steps to be followed. The steps to be followed for problem-solving depend on the problems to be solved. The problem-solving process includes defining the problem, gathering information, analyzing it, generating potential solutions, evaluating alternatives, selecting the best option, implementing it, and monitoring and assessing the outcomes.

In solving a computer science problem, the steps may be different from the ones given in the last paragraph. For instance, *given a question to* ***Create a Python program that finds the second largest integer in a list of numbers***. The steps to follow in solving the problem are to define the problem, research, define the requirements, create a plan, implement the plan using Python code, test the solution with some input data, analyze the results, and maintain the solution, as shown in Example 6.1.

1. **Define the problem**
   The task is to develop a program that identifies the second-largest integer in a given list of numbers.
2. **Research**
   Research a solution to the problem by finding or creating algorithms for identifying the second largest number in a list.
3. **Define the requirements**
   The program should take a list of integers as input and output, the second-largest number in the list.
4. **Create a plan**
   Solve this problem by first sorting the list in descending order and then returning the second component in the list.
5. **Implement the plan using Python code**

```
def find_second_largest(numbers):

    sorted_numbers = sorted(numbers, reverse=True)

    return sorted_numbers[1]
```

6. **Test the solution with some input data**

```
assert find_second_largest([1, 2, 3, 4, 5]) == 4

assert find_second_largest([10, 5, 20, 30, 15]) == 20

assert find_second_largest([1, 1, 1, 1]) == 1
```

7. **Analyze the results**

Verify if the solution correctly identifies the second-highest number in the list for all tests.

8. **Maintain the solution**
   Maintain the correct solution. If any problems arise or new requirements are added, modify the code to meet those requirements.

Example 6.1: Program that takes a list of integers and returns the second-largest number.

16.5 Different Methods of Solving Problems

Several different methods exist for solving problems. Some of them are the Scientific Method, Root Cause Analysis, Design Thinking, Critical Thinking, Decision-Making Models, Brainstorming, Simulation and Modelling, Systemic Problem Solving, and Computational Problem-Solving methods.

The Scientific Method is a systematic and evidence-based approach to problem-solving that involves making observations, forming hypotheses, conducting experiments, and drawing conclusions to gain a deeper understanding of natural phenomena. Root Cause Analysis is a systematic problem-solving technique that focuses on identifying and addressing the fundamental causes of an issue to prevent its recurrence. Design Thinking is a human-centred problem-solving approach that emphasizes empathy, creativity, and iterative prototyping to generate innovative solutions. Critical Thinking is a systematic and analytical approach to problem-solving that involves evaluating information, arguments, or situations objectively and making well-informed decisions based on evidence and logic. Decision-making models are structured frameworks or approaches that guide the selection of the best course of action among multiple options by considering relevant factors and criteria. Brainstorming is a creative problem-solving technique that encourages the generation of diverse ideas and solutions through open and collaborative discussion. Simulation and Modelling involve creating virtual representations of real-world systems or processes to analyze, experiment with, and optimize different scenarios for problem-solving and decision-making. Systemic Problem Solving is a method that focuses on understanding and addressing problems within larger systems or contexts, considering the interrelationships between various elements to find holistic solutions. Computational problem-solving is an analytical approach that

employs algorithms and computational tools to systematically address complex problems and generate solutions. However, further discussion shall dwell more on the Computational problem-solving method.

16.6 What is Computational Problem Solving (CPS)?

This is the procedure of using computer technology and programming procedures to get solutions to glitches that may be too complex or time-consuming to solve by hand. It includes segmenting or breaking down a problem into smaller, more manageable parts and then developing algorithms and writing code to solve each part.

Computational problem-solving is, in most cases, used in relation to professions such as engineering, physics, mathematics, finance, and science, and also in day-to-day activities. It is a vital knowledge area that permits entities to automate monotonous jobs, analyze large volumes of data, and develop novel understandings and solutions to complex glitches.

To attain a great height in computational problem solving, entities are required to have a robust knowledge of programming ideas, concepts, and methods, together with the skill to think rationally and methodically. It is also essential to be able to cooperate efficiently with others and to connect their ideas clearly and succinctly.

The general steps involved in computational problem solving are to comprehend the problem, examine the problem, develop an algorithm, execute the algorithm, assess and debug, optimize the result, and document the solution. An example is given in Example 3.2 to illustrate how to use this computational problem-solving method to solve a problem. The problem is, to ***sum up all even integers in a given list of integers using a program***. The steps are:

1. **Understand the problem**
   The problem is to find the total of even integers from a given list of numbers.
2. **Analyze the problem**
   Loop through the list to see whether the number is even. If it is, add it to a running total.

3. **Develop an algorithm**

Initialize a variable called "total" to 0

Loop through each number in the list

Determine if the number is even

If the number is even, increment the "total" variable by its value.

After looping through all the numbers, return the "total" variable

4. **Implement the algorithm in Python**

```
def sum_of_even_numbers(numbers):
    total = 0
    for num in numbers:
        if num % 2 == 0:
            total += num
    return total
```

5. **Test and debug**

Verify the correctness of the function by testing it with various inputs.

6. **Optimize the solution**

The time complexity of the algorithm for this problem is O(n), which is the best possible.

7. **Document the solution**

Document the function with a docstring and comments to be sure it is easy to understand for other developers.

```
def sum_of_even_numbers(numbers):
    """
    The function "sum_of_even_numbers" returns the
    total of all even integers in the provided list.

    Args:
        Numbers (list): A list of integers.
    Returns:
        int: The sum of all even numbers in the list.
    """
    total = 0
    for num in numbers:
        # Check if the number is even
        if num % 2 == 0:
            # Add the number to the total
            total += num
    return total
```

Example 6.2: A program designed to compute the sum of all even numbers present in a given list

16.7 Techniques for Computational Problem Solving

Computational problem-solving encompasses the use of computational apparatuses and methods to find solutions to complex problems. Many methods can be used to find solutions to problems. These solutions can be grouped into three stages: design, analysis, and experimentation.

16.7.1 Design

The design stage includes identifying the problem, developing a model or algorithm to solve it, and implementing the model using a programming language. This stage needs skills in problem-solving, algorithm design, and programming. The method used in this stage contains:

1. Object-oriented design
2. Functional programming

3. Structured programming
4. Model-driven design
5. Agile development

16.7.1.1 Object-oriented design

In this programming paradigm called object-oriented design (OOD), an object is the main point of focus. A class instance is a representation and manipulator of data. It is constructed based on the principles of abstraction, encapsulation, inheritance, and polymorphism.

Abstraction is the procedure for lessening complex systems or entities to their vital characteristics. Abstraction is accomplished in OOD by defining classes that stand for abstract ideas, which include a car, a bank account, or a person. Classes specify the characteristics and actions of objects but not how they are carried out.

Encapsulation is the procedure of concealing the execution details of an object and revealing only its interface, which contains a set of public methods. This permits the object to be used without the user having knowledge of how it works and also prevents external code from directly retrieving the object's data.

Inheritance allows a class to acquire traits and actions from a parent class. This permits the recycling of the previous code and helps generate a hierarchical structure of related classes.

Polymorphism is the capacity of objects of diverse classes to be used reciprocally as long as they execute the same interface. This permits greater flexibility and extensibility in the design of complex systems.

Take a look at an OOD with a Python example in Example 6.3.

```
class Car:

def __init__(self, make, model, year, colour):

    self.make = make
```

```
        self.model = model
        self.year = year
        self.colour = colour
        self.speed = 0
    def accelerate(self):
        self.speed += 10
    def brake(self):
        if self.speed >= 10:
            self.speed -= 10
    def get_speed(self):
        return self.speed
```

Example 3.3 A class called "Car" that has four attributes

In example 6.3, a class called "Car" with four attributes is defined. The attributes are "make", "model", "year", and "colour". Three methods are defined: "accelerate", "brake", and "get_speed". When a new object is made, Python invokes a unique method named "init" in the class. Thus, it initialises the object with the make, model, year, and colour passed as arguments and sets the initial speed to 0. The "accelerate" function adds 10 to the car's speed, while the "brake" function reduces it by 10, but only if the speed is 10 or greater. The "get_speed" method returns the current speed of the car.

The code in Example 3.4, "car", is a newly created object of the "car" class.

```
my_car = Car("Toyota", "Camry", 2021, "silver")
```

Example 3.4: creation of a new object of the car class called car

A car object is generated with the following features: make = "Toyota", model = "Camry", year = 2021, and colour = "silver".

Then, call methods on the object car is presented in Figure 3.5 thus:

```
my_car.accelerate()

print(my_car.get_speed()) # Output: 10

my_car.brake()

print(my_car.get_speed()) # Output: 0
```

Example 6.5: call methods on the object car

16.7.1.2 Functional programming (FP)

Functional programming is a programming approach that emphasises the use of pure functions that produce results solely based on their inputs and have no side effects. The main focus in FP is on what functions do rather than how they do it. Some vital examples of concepts of functional programming are:

Immutable data: In this case, it implies that once data is created, it cannot be changed. Instead of changing it, new data structures are created based on existing ones.

Higher-order functions: Functions can act as arguments or return values in other functions. This gives room for powerful abstractions and composability.

Recursion: Recursion is used widely in functional programming, often replacing traditional looping constructs.

Pure functions: Pure functions are functions that produce consistent output for the same input and have no side effects.

Python supports a lot of programming Paradigms, such as FP. An example of a functional [program in Python is given in Example 6.6.

```
# A pure function is defined to compute the sum of a list of numbers and return the result.

    def sum_numbers(numbers):

        return sum(numbers)

    # A higher-order function is defined to accept a function as an argument and apply it to a list of numbers.

    def apply_function_to_numbers(numbers, function):

        return function(numbers)

    # Define a list of numbers

    my_numbers = [1, 2, 3, 4, 5]

    # Apply the sum_numbers function to the list of numbers using apply_function_to_numbers

    result = apply_function_to_numbers(my_numbers, sum_numbers)

    # Print the result

    print(result) # Output: 15
```

Example 3.6: Function program in Python

In Example 3.6, the built-in sum() function in Python is utilised in a pure function called sum_numbers to calculate the sum of a list of numbers. A higher-order function ***apply_function_to_numbers*** that takes a function and applies it to a list of numbers is defined. A list of numbers called my_numbers is created, and apply_function_to_numbers is employed to apply the sum_numbers function to my_numbers. The console displays the sum of the numbers in the list as a result.

Figure 6.6 demonstrates the concepts of functional programming, such as pure functions, higher-order functions, and immutability.

16.7.1.3 Structured programming (SP)

SP is a programming model that has the goal of enhancing the lucidity, value, and dependability of code by using control structures that impose logical structure and modularity. The vital concept that supports structured programming is to break down a program into smaller, more manageable pieces called modules or functions, each of which accomplishes a precise job or operation. The main three fundamental structures of SP are:

1. **Sequence:** The stepwise refinement structure implements a sequence of commands, one after the other, in a precise order.
2. **Selection:** The conditional statements structure implements diverse blocks of code relying on the result of a logical condition. Common selection structures involve if-else statements and switch-case statements.
3. **Repetition:** The iteration structure implements a Code segment continually until a stated condition is met. Common repetition structures involve loops, while loops, and do-while loops.

Example 6.7 is a Python program that utilises loop and conditional statements and is demonstrated as a structured program as follows.

```
# A program created to compute the sum of numbers in a list.

def calculate_sum(numbers):
    total = 0
    for num in numbers:
        if num % 2 == 0:
```

```
            total += num
    return total
# Example usage
my_list = [1, 2, 3, 4, 5, 6]
result = calculate_sum(my_list)
print("The sum of even numbers in the list is:", result)
```

Example 3.7: Structured program in Python

In example 3.7, a function named calculate_sum is defined to receive a list of numbers as input, iterate through the list, and add up the even numbers. In the calculate_sum function, the for-loop loops through each element in the list, and the if statement checks if the number is even before adding it to the running total. Conclusively, return the total sum of even numbers.

16.7.1.4 Model-driven design

Model-driven design (MDD) is a method of software development that includes producing models to signify diverse features of a software system, together with its structure, behaviour, and functionality. MDD models serve as a design plan for the software system and can be used to produce code automatically.

MDD comprises several steps, such as:

1. **Requirements** gathering and analysis: In this stage, the needs for the software system are acknowledged and analysed. The needs are used to produce a set of models that stands for the system's functionality and behaviour.

2. **Model creation**: Once the needs are acknowledged, models are produced to stand for the diverse features of the software system. These models may contain data models, process models, and user interface models.
3. **Model transformation**: In this stage, the models are changed into implementable code. This can be done manually, but often it is automated using code generators or other tools.
4. **Testing and validation**: Once the code has been produced, it is tested and validated to guarantee that it meets the needs of the software system. In Example 3.8, a Model-driven design (MDD) is created representing the software architecture and behaviour before executing the code. The model is then changed into implementable code automatically.

Now, the problem to be solved is to ***create a simple calculator application using MDD***.

1. Start by producing a model that explains the calculator's behaviour and user interface.
2. Represent this model using Unified Modeling Language (UML).
3. Next, produce a UML class diagram that describes the classes and their relationships. In solving this problem, note that a Calculator class that has methods for executing addition, subtraction, multiplication, and division is needed. Similarly, a UserInterface class that handles user input and outputs the results is needed as shown in Figure 6.8.

```
class Calculator:
    def add(self, a, b):
        pass
```

```
    def subtract(self, a, b):
        pass
    def multiply(self, a, b):
        pass
    def divide(self, a, b):
        pass
class UserInterface:
    def get_user_input(self):
        pass
    def display_result(self, result):
        pass
```

Subsequently, produce a state machine diagram that describes the calculator's behaviour. Represent this using the Python ***transitions*** library. Example 6.9 is an example state machine diagram program that shows the different states the calculator can be in and the events that trigger state transitions.

```
class CalculatorStateMachine:
    states = ['ready', 'operand1', 'operator', 'operand2', 'result']
    def __init__(self, calculator):
        self.calculator = calculator
        self.machine = Machine(model=self,
states=CalculatorStateMachine.states, initial='ready')
```

```
        self.machine.add_transition('input_operand1','ready',
'operand1')

        self.machine.add_transition('input_operator', 'operand1',
'operator')

        self.machine.add_transition('input_operand2', 'operator',
'operand2')

        self.machine.add_transition('calculate_result', 'operand2',
'result')

        self.machine.add_transition('clear', '*', 'ready')

    def on_input_operand1(self):

        self.calculator.operand1 = self.calculator.ui.get_user_input()

        self.calculator.ui.display_result(self.calculator.operand1)

    def on_input_operator(self):

        self.calculator.operator = self.calculator.ui.get_user_input()

        self.calculator.ui.display_result(self.calculator.operator)

    def on_input_operand2(self):

        self.calculator.operand2 = self.calculator.ui.get_user_input()

        self.calculator.ui.display_result(self.calculator.operand2)

    def on_calculate_result(self):

        if self.calculator.operator == '+':

            result = self.calculator.add(self.calculator.operand1,
self.calculator.operand2)
```

```
            elif self.calculator.operator == '-':
                result = self.calculator.subtract(self.calculator.operand1,
self.calculator.operand2)
            elif self.calculator.operator == '*':
                result = self.calculator.multiply(self.calculator.operand1,
self.calculator.operand2)
            elif self.calculator.operator == '/':
                result = self.calculator.divide(self.calculator.operand1,
self.calculator.operand2)
            self.calculator.ui.display_result(result)
        def on_clear(self):
            self.calculator.operand1 = None
            self.calculator.operator = None
            self.calculator.operand2 = None
            self.calculator.ui.display_result('')
```

Example 6.9: Different states of the calculator

Lastly, produce the main program that initialises the calculator object and the state machine object. The main program handles user input and transmits events to the state machine, as shown in Example 6.10.

```
class Calculator:
    def __init__(self):
```

self.ui

Example 6.10: Initialisation of the calculator object and the state machine object

16.7.1.5 Agile development

Agile development is a reiterative and stretchy method of software development that prioritises partnership, teamwork, and customer satisfaction. It focuses on working in small, cross-functional teams that work meticulously together throughout the development process. The Agile development procedure depends on incessant feedback and adjustment and is characterised by the delivery of small, working pieces of software in short timeframes, known as sprints. Agile development has become a generally used method of software development because of its ability to deliver high-quality software that meets customer needs in a timely and flexible manner.

Agile development is a substitute for traditional waterfall software development, which is a sequential and consecutive method of software development. In software development, there is a reliance on the Agile Manifesto, which sets out a set of values, principles, and standards. The Agile development approach is based on four core values, which are:

1. Emphasising people and their interactions rather than relying solely on processes and tools.
2. Focusing on producing working software rather than extensive documentation.
3. Prioritising collaboration with customers over adhering strictly to contract terms.
4. Being responsive to changes and challenges as they arise rather than adhering rigidly to a predetermined plan.

Some of the stages of the processes in Agile development are:

1. **Planning:** The team defines the project requirements and creates a plan for the upcoming sprint

2. **Development:** The team works on the development of the product in the defined sprint timeframe.
3. **Testing:** The team performs product testing to ensure that it meets the requirements and quality standards.
4. **Review:** The team reviews the completed work with stakeholders to gather feedback and plan the next sprint
5. **Repeat:** The team repeats the process with a new sprint, building on the feedback from the previous sprint.

16.7.2 Analysis

This stage appraises the performance of the executed model or algorithm and recognises areas allotted for enhancement. This stage needs skills in data analysis, statistical modelling, and performance appraisal. The method used in this stage contains:

1. Statistical analysis
2. Optimisation method
3. Simulation modelling
4. Machine learning
5. Data visualisation

16.7.3 Experimentation

The experimentation stage contains testing the implemented model or algorithm under diverse circumstances and appraising its performance. This stage needs skills in experimental design, data collection, and statistical analysis. The method used in this stage contains:

1. Design of experiments
2. A/B testing
3. Monte Carlo simulations
4. Sensitivity analysis
5. Statistical hypothesis testing

Design of Experiments (DoE)

Design of experiments (DOE) is a statistical methodology used to methodically plan, implement, and analyse controlled experiments to get perceptions and optimise processes. The main goal of DOE is to determine the cause-and-effect relationship between the input factors and output responses of a system or process while minimising the effects of random variation.

The steps involved in DOE are defining the problem, selecting the factors, determining the levels, choosing the design, conducting the experiment, analysing the results and drawing conclusions. DOE is used in diverse fields, such as manufacturing, engineering, agriculture, medicine, and social sciences, to improve product quality, increase efficiency, reduce costs, and optimise processes.

A/B testing

A/B testing is a method used in advertising, web design, and other fields to liken two versions of a product or service and decide on which one performs better. This is characteristically done by randomly sharing a sample of users into two groups, with each group being exposed to a separate version of the product or service. The two versions of them are compared depending on the type of metrics, such as conversion rate, click-through rate, or user engagement used.

A/B testing aims to recognise which version of the product or service leads to better performance so that the better version can be executed generally. A/B testing is a useful tool for improving advertising efficiency, optimising website design, and enhancing user experience. To draw meaningful conclusions from A/B testing, it is crucial to ensure that the sample size is sufficient and that the results are statistically significant.

Monte Carlo simulations

Monte Carlo simulations are computational techniques that use random sampling methods to model complex systems and produce numerical approximations of their behaviour. They are useful for analysing systems that cannot be easily solved analytically and for generating estimates of the likelihood of certain outcomes. The

name "Monte Carlo" was created in reference to the famous casino in Monaco, as the technique relies on random processes similar to games of chance.

A mathematical model of a system is defined, and then random samples are drawn from a distribution that represents the system's input parameters in Monte Carlo simulations. These samples are then used to produce a huge number of simulations of the system, which permit the study of its behaviour and estimate of its properties.

Monte Carlo simulations are generally used in a diversity of areas of specialisation, such as physics, chemistry, engineering, finance, and economics. They can be used to model everything from the behaviour of subatomic particles to the performance of monetary portfolios. One of the important points of Monte Carlo simulations is that they can handle complex, nonlinear systems with many input parameters and can often give more accurate estimates than traditional analytical or numerical methods.

Sensitivity analysis

Sensitivity analysis is a method used to control how changes in the input variables of a model affect the outputs or outcomes of the model. It is generally used in areas of specialisation like economics, engineering, and finance to appraise the strength of a model and to recognise which variables have the most vital influence on the results. Sensitivity analysis can be executed using various methods, such as one-way sensitivity analysis, which scrutinises the consequence of altering one input variable at a time, and multi-way sensitivity analysis, which scrutinises the consequence of altering multiple input variables at the same time. The outputs of sensitivity analysis can be used to notify decision-making, improve the accuracy of models, and classify areas for further research.

Statistical hypothesis testing

Statistical hypothesis testing is a procedure of arriving at conclusions or making decisions about a population depending on a sample of data. This is a summary of a hypothesis testing process: define a null hypothesis and an alternative hypothesis, collect data, calculate test statistics, and determine the probability of observing the

test statistics under the assumption that the null hypothesis is true. If the probability is small, the null hypothesis is rejected in favour of the alternative hypothesis, indicating a significant difference between the population parameter of interest and the null hypothesis.

The level of significance, represented by alpha, is the likelihood of declining the null hypothesis when it is true. The alpha level in hypothesis testing determines the balance between the risk of rejecting a true null hypothesis (Type I error) and the risk of retaining a false null hypothesis (Type II error). The p-value represents the probability of observing test statistics or more extreme results if the null hypothesis is correct. If the p-value is smaller than alpha, the null hypothesis is declined.

Statistical hypothesis testing is generally used in several study areas, such as science, medicine, finance, and social sciences, to test hypotheses, arrive at conclusions, or make decisions about a population depending on a sample of data.

Summary

From the discussions, it can be concluded that a problem is a situation or challenge that needs to be solved or addressed. Problems can range from small daily inconveniences to complex, large-scale issues. Some examples of problems in daily life include traffic congestion, long wait times, difficult relationships, financial struggles, and health concerns.

Problems can be categorised into two main types: routine and non-routine. Routine problems are ones that have a clear solution or can be solved using a standard process, such as fixing a leaky faucet or cooking a meal. Non-routine problems, on the other hand, are more complex and do not have a clear solution, such as dealing with a difficult classmate or addressing systemic racism.

Problem-solving is the method of recognising, examining, and solving problems. It is a process that includes identifying the root cause of a problem, generating solutions, evaluating them, and implementing the best solution.

A specific type of problem-solving called computational problem-solving, which involves using algorithms and programming techniques to solve problems, was

discussed. Techniques for computational problem-solving include design, analysis, and experimentation. The process of design consists of developing a plan or blueprint for building a computer program or algorithm that can solve a problem. The analysis involves evaluating the efficiency and effectiveness of the algorithm or program. Experimentation involves testing the algorithm or program on real-world data to ensure it is effective and efficient.

Finally, exercises are used to enhance problem-solving skills and help them become better equipped to handle various challenges that arise in their personal and professional lives.

Exercises

1. What is a problem?
2. List some problems in daily life.
3. What are the types of problems with respect to routine problems/solvable problems and non-routine problems/unsolvable problems?
4. What is problem-solving?
5. What is computational problem-solving?
6. What are the techniques for computational problem-solving?
7. Use problem-solving steps to write a Python program that finds the sum of the digits of a number.
8. Write a Python program using problem-solving steps to find the length of the longest word in a sentence.
9. Write a Python program to find the largest palindrome number, which is the product of two three-digit numbers, using problem-solving steps.
10. Write a Python program to remove duplicate elements from a list using problem-solving steps.

CHAPTER SEVENTEEN
Problem-Solving Concepts for the Computer

Overview

This chapter provides sufficient knowledge about problem-solving concepts for the computer. It focuses on the systematic process of designing, implementing, and using programming tools such as variables and data structures during the problem-solving stage. Key concepts and definitions of problem-solving concepts that make the computer system more intuitive with human logic than machine logic are discussed.

Objectives

This chapter focuses on providing students with the basic knowledge of problem-solving concepts in computer science. Specifically, students should be able to:

i. describe constants and variables distinctly.
ii. understand the rules for constructing variable names.
iii. gain knowledge of the different data types used in problem-solving and programming.
iv. understand how to express these data types with their corresponding variables.
v. gain knowledge of Expressions and Equations.
vi. understand what operators are and how to use operators for problem-solving.
vii. learn the concept of functions and their implementation.

17.1 Definition of Constants and Variables

Problem-solving is the process of transforming the description of a problem into a solution by using our knowledge of the problem area and by implementing appropriate problem-solving strategies, techniques, and tools. Program constants

and variables are one of the important concepts in problem-solving strategies in computer science.

It is important to know that there are six basic operations performed during the computer software development process, which are as follows:

1. Input data
    a) Read data from a file (as constants or through variables)
    b) Get data from the keyboard (as constants or through variables)
2. Perform arithmetic computations using actual mathematical symbols or the words for the symbols
3. Assign a value to a variable

    There are three possible cases:

    a) Variable initialisation
    b) Assign a value to a variable as a result of some processing
    c) Keep a piece of information for later use (Save/Store)
4. Compare two pieces of information and select one of two (or more) alternative actions
5. Repeat a group of actions on the basis of some pre-defined condition or number of iterations
6. Output information
    a) Write information to a file
    b) Display information on the screen.

From these basic operations as described above, you will find out that these key concepts of constants, variables and data types play a key role in the life cycle of any computer programming development activity. This is the reason why students must understand the in-depth problem-solving concepts of constants, variables and data types and their implementation.

Constants are values that do not change during the execution of a program. They are usually declared and initialised at the beginning of a program, usually in UPPERCASE, and their values cannot be altered throughout the program's execution. In Python, as a case study, constants are usually defined using all capital letters to differentiate them from variables. For example:

PI = 3.14159

MAX_SCORE = 100

In the above example, `PI` and `MAX_SCORE` are constants.

Constants are useful in situations where you need to use a fixed value repeatedly in your program, such as the value of pi in mathematical calculations.

Variables

Variables are values used to store data that can be changed during the execution of a program. It is used to hold data that can be processed and manipulated by the program. In Python, variables are created by assigning a value to a name. This allows the data to be **retrieved** and **modified** when needed by the program. For example:

x = 5

name = "Chukwuemeka Odi Agwu"

In the above example, `x` and `name` are variables.

A variable can hold **one** value (a piece of data) at a time. This value can change throughout the execution of a program. When a new value is placed in (or assigned to) a variable, it replaces the last value.

In programming, variables are designated to hold a variety of data, and these include numeric data, character data, and logical data.

17.2 Rules for Naming Variables

In most programming languages, variables must follow certain rules when they are named. We will use Python programming language as a case study. Below are some of the conventions or rules for naming variables. These are:

1. They must start with a letter or an underscore ( ).
2. They can only contain letters, numbers, and underscores.
3. They cannot be reserved words or keywords. (Reserved words have special meaning to programming language translators and when they are used. Examples are, if, while, print, read, etc.)
4. Variable names are case-sensitive. For example, `myVariable` and `myvariable` are two different variables.
5. It must not contain special characters, such as!@£$%&* or punctuation characters. However, an underscore can be used. Spaces are not permitted.
6. Variable names should be descriptive and meaningful.

Ensure that these rules are followed when naming variables in a Python program because they improve the readability of your code; this makes them easy to maintain.

17.3 Data Types

In problem-solving and computer programming, data types are used to define the varieties of data that a variable or constant can hold. Python supports a wide range of data types, including:

1. Numeric data types (integers, floating-point numbers, and complex numbers)
2. Character data type (strings)

3. Boolean data type (True or False)
4. Sequence data types (lists, tuples, and sets)
5. Mapping data type (dictionary)
2. No data type
3. Object data type

Different data types have different properties and behaviours, and it is important to choose the appropriate data type for the task at hand. Fig 17.1 shows the various categories of Python data types.

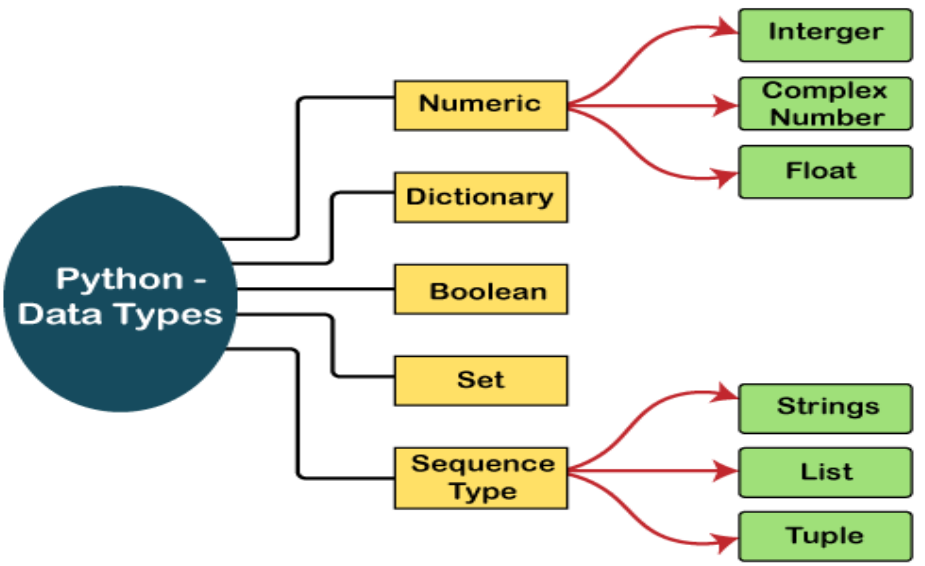


Figure 17.1: Data Types in Python

It is important to know that in the programming world, a data type is nothing but an attribute to instruct the compiler or interpreter on how the programmer or developer intends to use the data.

17.3.1 Numeric Data

In Python, there are three main types of numeric data types: integers, floating-point numbers, and complex numbers. Integers are whole numbers, such as 1, 2, 3, and so on. Floating-point numbers are numbers with decimal points, such as 1.23, 4.56, and so on. Complex numbers are often defined as numeric data that has a real and imaginary part, such as 2 + 3j or 4 - 5j.

Here are some examples of numeric data type usage in Python:

Type() in Python is used to find out the type of data in action, as shown in examples 1, 2, and 3. The line of sentences that starts with a # sign shows that the line is a comment.

Example 1: Using the Type() function to output integer data type

```
# Integer data type

x = 5

print(type(x))


 # Output: <class 'int'>
```

Example 2: Using the Type() function to output float data type

```
# Floating-point data type

y = 3.14

print(type(y))


# Output: <class 'float'>
```

Example 3: Using the Type() function to output the complex data type

```
# Complex data type
z = 2 + 3j
print(type(z))

# Output: <class 'complex'>
```

17.3.2 Character Data

In Python, a character is represented by a single-character string. This means that even though the string contains only one character, it is still considered a string data type. For example, the string 'a' is a character data type in Python.

Here are some examples of character data types in Python:

Example 4: Using the Type() function to output the character data type

```
# Character data type
a = 'a'
print(type(a))

 # Output: <class 'str'>
```

Example 5: Using the Type() function to output the string data type

```
# String data type
sentence = 'Hello, World!'
print(type(sentence))

# Output: <class 'str'>
```

17.3.3 Alphanumeric Data Types

Alphanumeric data types refer to data that contains both letters and numbers, and they are always used in computer programs. In Python, alphanumeric data types are represented by strings.

Here are some examples of alphanumeric data types in Python:

Example 6: Using the Type() function to output the string data type

```
# String data type containing only letters
name = 'Chukwuemeka'
print(type(name))

# Output: <class 'str'>
```

Example 7: Using the Type() function to output the alphanumeric data type

```
# String data type containing letters and numbers

regNo = 'EBSU/CSC/2021/100197'

print(type(regNo))


# Output: <class 'str'>
```

17.3.4 Logical Data

In Python, the logic data type refers to the Boolean data type, which has two possible values: True and False. Boolean values are typically used in conditional statements and loops.

Here are some examples of logic data types in Python:

```
# Boolean data type

x = True

print(type(x))


 # Output

<class 'bool'>
```

17.3.5 Other Data Types

1. Integer (int): This is a whole number that can be positive, negative or zero. In Python, integers are represented by the int data type.

Example:

```
x = 5

y = -3

z = 0
```

2. Float: This is a number with a decimal point. In Python, floats are represented by the float data type.

Example:

```
a = 3.14

b = -2.5

c = 0.0
```

3. Boolean (bool): A Boolean data type can store only two values: True or False.

Example:

```
is_raining = True

is_sunny = False
```

4. String (str): There is a sequence of characters enclosed in single (' ') or double (" ") quotes. In Python, strings are represented by the str data type.

Example:

```
name = "Chukwuemeka"
```

address = 'kilometer 21 Abakaliki-Enugu expressway, Abakaliki Ebonyi state Nigeria.'

5. List: A list is formed when items are separated by commas and subsequently enclosed in square brackets [ ]. Lists are unique data types because they can contain items of different data types.

Example:

students = [“Chukwuemeka,”” Paul”, “comfort”]

numbers = [1, 2, 3, 4, 5]

mixed = ['apple', 2, True, 3.14]

6. Tuple: A tuple is similar to a list, but its values cannot be changed once it is created. Tuples are enclosed in parentheses ( ).

Example:

coordinates = (3, 4)

colours = ('red', 'green', 'blue')

7. Dictionary: A dictionary is a collection of key-value pairs enclosed in curly braces { }. These pairs are carefully separated by colons, and then the key-value pairs are separated by commas.

Example:

person = {'name': 'Chukwuemeka', 'age': 24, 'city': 'Abakaliki’ }

student = {'id': 92280, 'name': 'Patterson', 'grades': [80, 90, 95]}

8. Set: A set is an unordered collection of unique items enclosed in curly braces {}. Sets can only contain items of immutable data types.

Example:

colours = {'red', 'green', 'blue'}

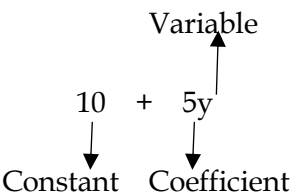
numbers = {1, 2, 3}

17.4 Expressions and Equations

In computer science, an expression is an addition or subtraction of terms. Each term contains a variable or constant that is combined by operators evaluated to yield another value. This is different from equation, which equates two terms, two expressions or a term and expression by an 'equal to' sign.

17.4.1 Expressions

An expression is a mathematical statement that contains terms made up of numbers, variables or both. Terms are parts of an expression that are added or subtracted. For instance, the constant 10 is the first term, and the variable 5y is the second term in the expression below.



In the expression 10 + 5y, 10 is a constant, 5 is a coefficient, and y is a variable.

Expressions are used to model real-world problems. For instance,

An employer agreed to pay #1,250 for an hour of overtime work made by his employee. Therefore, the Overtime amount for employees that worked for h hours can be expressed as;

#1,250 x h hours = #1,250h

Although an expression can contain data types such as integers, strings, characters, and floating points, the system handles these terms as variables and constants. Operators determine the ways terms are manipulated in an expression. Expressions are categorised based on what they are evaluated to. For instance, Boolean expression (named after the mathematician George Boole) returns either "True" or "False." Whereas numerical expression evaluates to numbers.

a) Boolean Expressions

A Boolean expression evaluates expression to return either true or false (or their synonyms such as “YES” or “NO”, “AGREE” OR “DISAGREE”, etc.). Computer programmers set Boolean expressions as conditions for control execution of block of code. For instance,

5 < 7 returns ‘TRUE’

F =! G returns ‘FALSE’

3 >= 9 returns ‘FALSE’

A Boolean expression returns either 'true' or 'false' and is created by using Boolean operators. Examples of Boolean operators are:

Table 17.1**:** Boolean operator

Operator	**Meaning**
<	Less Than
>	Greater Than
==	Equal to

<=	Less Than or Equal to
>=	Greater Than or Equal to
! =	Not

b) Numerical Expressions

A numerical expression is a statement made up of a combination of numbers and operators such as addition, subtraction, multiplication or division. Simply put, numerical expression is a statement that involves numbers and operations, which can be simplified by the usage of proper operators with one of the terms or numbers. Numerical expressions must only contain numbers and one or more operations. Examples are: 8 + 9; 84 – 53; 12 * 7 + 9; 76/5 * 7 -3. Numerical expression is one of the basic and important mathematical concepts that a computer science student needs to grasp to solve problems. Real-world mathematical problems are solved by first converting them into numerical or algebraic expressions. This is by noting down the numbers in the problem and assigning appropriate operators between the numbers to form numerical expressions.

Example: Torsen is an orange farmer with 50 oranges. He gives 20 oranges each to his friends Emeka and Adeleke in the morning. That evening, his son brought an additional 25 oranges from the farm. That evening, Torsen gives 20 oranges to his neighbour Adamu. At the end of that day, how many oranges does Torsen have?

Solution: 50 - 20 – 20 + 25 – 20

c) Algebraic Expressions

Since numeric expression contains only numbers, expressions containing variables (represented by letters such as x or y) are not considered numerical expressions. Expression containing variables are called algebraic expressions. Examples of

algebraic expressions are: 5x + 8; 180 – y + 5x. Most real-life problems can be modelled in either numeric or algebraic expressions.

17.4.2 Equations

An equation denotes that the expression on either side of the equal to (=) sign is the same. Although both expression and equation combine numbers and/or variables, an equation has an 'equal to' sign (=) in between. It is important to find the value of the variable to satisfy an equation. This is called the solution or roots of the equation. The four common categories of equations are linear equations, quadratic equations, polynomial equations and simultaneous equations.

a) Linear Equation

In a linear equation, the highest power of the variable is 1. Consequently, the degree of the linear equation is one. The graph of a linear equation always forms a straight line. An example of a linear equation in one variable is ax + c = 0, whereas that of two variables is ax + by = c, where a, b are coefficients and c is a constant. Generally, the standard form of a linear equation is y = mx + c, where c represents the y-intercept and m is the slope.

Linear Equation Graph

A linear equation is represented by a straight line on a graph. The graph of the linear equation with x variable forms a straight line parallel to the y-axis. Similarly, that of equation with y variables forms a straight line parallel to the x-axis. Also, the graph of linear equations in two variables, x and y, forms a straight line. The graph below shows the straight-line representation of the linear equation x – 2y = 2.

Graphical Representation of x – 2y = 2

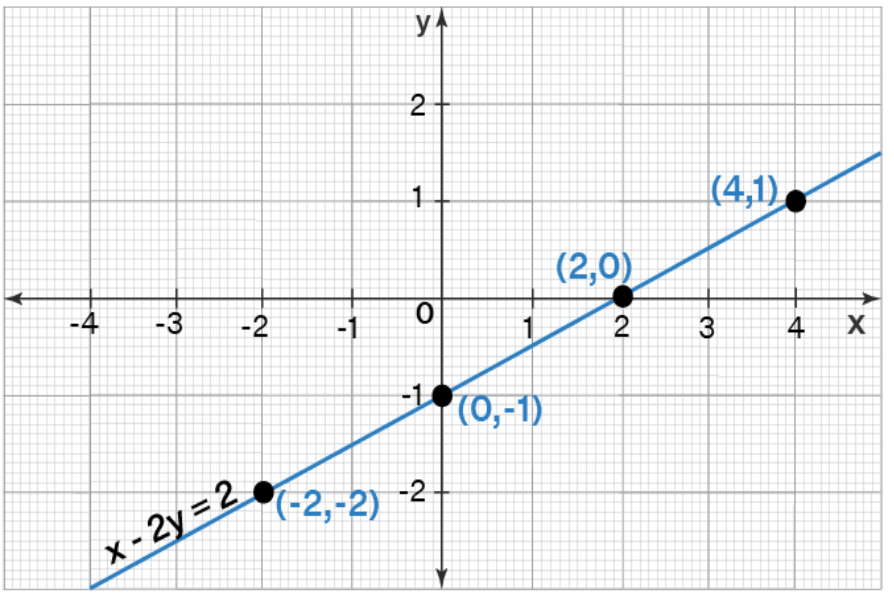


Figure 17.2: Graphical Representation of Linear equation

b) Quadratic Equation

The highest power of variables in a quadratic equation is two; as such, the degree of a quadratic equation is two. The standard form of quadratic equation in one variable is $ax^2 + bx + c = 0$, where $a \neq 0$. A quadratic equation will always have two solutions or roots. However, these roots can be equal, either real or complex. There are four (4) basic methods of solving quadratic equations:

i) Factorization Method

This method based its solution on the product of two number AB =0, then either A = 0 or B = 0. Therefore, factor the expression on the left-hand side and equate each factor to zero, as shown in the example below.

Example; Solve the equation $3x^2 + 2x - 5 = 0$

$3x^2 - 3x + 5x - 5 = 0$

$3x(x - 1) + 5(x - 1) = 0$

$(x - 1)(3x + 5) = 0$

Either $x - 1 = 0$ or $3x + 5 = 0$

$x = 1$ or $x = -5/3$

ii) Completing the Square Method

This method focuses on making the left-hand side of the quadratic equation a 'perfect square' by adding the square of half the coefficient of x to both sides after moving the constant term to the other side of the equation.

Example: Solve the quadratic equation, $2x^2 + 3x - 1 = 0$

Move -1 to the right-hand side

$2x^2 + 3x = 1$

Divide through by 2,

$x^2 + 3x/2 = 1/2$

Add $(3/4)^2$ to both side

$x^2 + 3x/2 + (3/4)^2 = ½ + (3/4)^2$

$(x + ¾)^2 = ½ + 9/16$

$(x + ¾)^2 = 17/16$

$x = -3/4 \pm \text{sqr}(17)/4$

$x = [-3 + \text{sqr}(17)]/4$ or $x = [-3 - \text{sqr}(17)]/4$

$x = 0.28$ or $x = -1.78$

iii) Formula Method

A general formula for solving any quadratic equation can be obtained by applying the completing the square method on the standard form of quadratic equation;

$ax^2 + bx + c = 0, a \neq 0$

First, move the constant c to the right-hand side of the equation

$ax^2 + bx = + c$

Divide through by a

$x^2 + bx/a = - c/a$

Add $(b/2a)^2$ to both side

$x^2 + bx/a + (b/2a)^2 = - c/a + (b/2a)^2$

$(x + b/2a)^2 = -c/a + b^2/4a^2$

$(x + b/2a)^2 = (-4ac + b^{2)}/2a$

$x = - b/2a \pm sqr (b^2 – 4ac)/2a$

$x = [-b \pm sqr (b^2 – 4ac)]/2a$, this formula can solve any quadratic equation.

Example; Solve the equation $2x^2 + 3x – 1 = 0$.

A = 2, b = 3 and c = -1.

Substitute in the formula

$x = [-b \pm sqr (b^2 – 4ac)]/2a$

$x = [-3\pm sqr (3^2 – 4.2.-1)]/2.2$

$x = [-3\pm sqr (9 + 8]/4$

$x = [-3 + sqr (17]/4$ Or $x = [-3 - sqr (17]/4$

$x = 0.28$ or $x = -1.78$

iv) Graphical techniques

To find the roots of a quadratic equation graphically, plot the graph of the equation on a suitable scale and interval of x. The roots are values of x when y = 0. In other words, it is the point where the curve intersects with the x-axis. There are three possibilities, as shown in Figure 17.3. First is when the curve intersects with the x-axis at two points to get two distinct roots. The second is when the curve touches the x-axis at one point to get one repeated root. Finally, when the curve did not touch the x=axis as such, there is no solution.

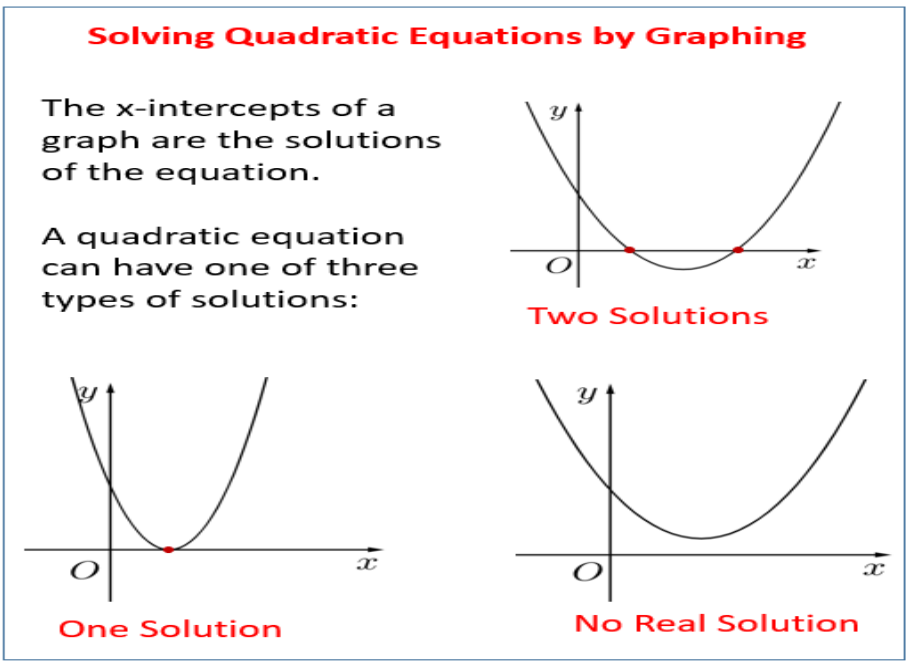


Figure 17.3: Graph of Quadratic Equations

c) Polynomial Equations

A polynomial equation is the sum of separate terms of the positive power of a variable with a constant term, which may be zero. It is an algebraic equation consisting of exponential variables with coefficients. The general polynomial in x, of nth degree, can be written in this form;

$a_nx^n + a_{n-1}x^{n-1} + a_{n-2}x^{n-2} + \ldots\ldots + ax^1 + a_0 = 0, \quad a_n \neq 0$, Where $a_n, a_{n-1}, , a_{n-2},$ ….a are coefficients and $x^n, x^{n-1}, x^{n-2}, \ldots\ldots x^1$ are variables. Any of these terms can be zero except a_n.

For instance, the linear equation $5x + 7 = 0$ is a first-degree polynomial, the quadratic equation $2x^2 + x + 3 = 0$ is a second-degree polynomial, and the cubic equation $2x^{3+}x^2 + 3x + 7 = 0$ is a third-degree polynomial.

d) Simultaneous Equation

Simultaneous equations are two equations in x and y. A unique solution of the equations can be obtained unless the two equations represent parallel lines, where there is no solution. The methods of solving quadratic equations are the Substitution Method, Elimination Method and Graphical Method

i) Substitution Method

Solve the equation $3x - 2y = 5$ …….. I

$2x - y = 7$ …….. II

From II, $y = 7 - 2x$ …… III Substitute III in I

$3x - 2(7 - 2x) = 5$

$3x - 14 + 4x = 5$

$7x = 19 \Longrightarrow x = 19/7$

ii) Elimination Method

Solve the equation $3x - 2y = 5$ …….. I

$2x + 5y = -7$…….. II

Multiply I by 5: $15x - 10y = 25$

Multiply II by 2: $4x + 10y = -14$

ADD: $19x = 11 \Longrightarrow x = 11/19$

iii) Graphically Solution of Simultaneous Equations

Solving simultaneous equations by graphical method requires plotting the lines of the equations on the coordinate plane. The solution lies on the point of intersection of the two lines. Figure 7.4 shows the solution of the simultaneous equation; x + y = 10 and x - y = 4. The two lines intersect at the coordinate (7, 3). Therefore, the solution is x = 7 and y = 3.

Solving Simultaneous Equation x + y = 10 and x - y = 4 by Graphical Method

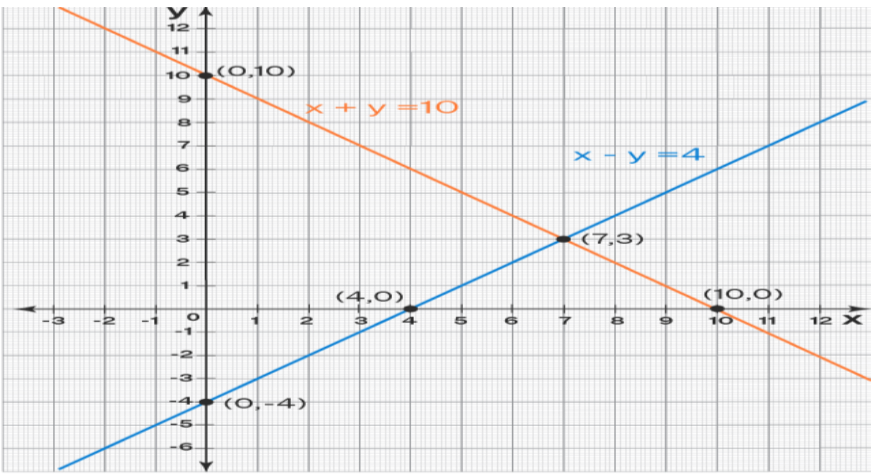


Figure 17.4: Solving Simultaneous Equation by Graphical Method

17.5 Operators

An operator in the programming language is a symbol that presents actions of manipulating certain terms in expressions or functions to return a value. There are many types of operators used in different programming languages for computing. The types of operators are broadly categorised based on their function into six classes as follows.

i) Arithmetic Operators

Arithmetic operators are used to carry out mathematical manipulations such as addition, subtraction, multiplication, and division on a given operand to return a single result. Table 17.2 presents the five common arithmetic operators.

Table 17.2:Arithmetic Operators

Symbol	**Operation**	**Explanation**
+	Addition	Totalling the value of terms in either side of the + operator, (a + b).
-	Subtraction	Deducting the value after from the one before the – operator, (a - b).
*	Multiplication	Multiplying the values on both sides of the * operator, (a * b).
/	Division	Dividing the value before by the value after the / operator, (a / b).
%	Modulus)	Returns the remainder of dividing the value before by the value after the % operator (a % b).

ii) Logical Operators

Logical operators are to check if a certain condition is satisfied depending on the values of the expression to return 'TRUE' or 'FALSE'. Table 17.3 presents three common logical operators.

Table 17.3: Logical Operators

Symbol	**Operation**	**Explanation**

&&	Logical AND	Returns TRUE only if values of expression on both side of && operator evaluate to TRUE and FALSE otherwise, (a && b).
//	Logical OR	Returns TRUE' if at least one of either side of the expression evaluates to TRUE, it returns FALSE, (a // b).
!	Logical NOT	Returns TRUE if the expression evaluates to FALSE and vice versa, (!a)

iii) Relational Operators

Relational Operators are used to define the relationship between two expressions by comparing their values. Relational operators are widely used for comparison; for instance, a picture is captured so that based on the comparison, certain conditions must be met to warrant the system return a true. Otherwise, return false. Table 17.4 presents relational operators and their symbols.

Table 17.4: Relational Operators

Symbol	**Operation**	**Explanation**
= =	Equal to	Returns TRUE if both expressions are equivalent and FALSE otherwise, (a == b).

!=	Not equal	Returns TRUE if both expressions are not equivalent and FALSE otherwise, (a != b)
>	Greater than	Returns TRUE if the expression before is greater than the one after the > operator and FALSE otherwise, (a > b).
<	Less than	Returns TRUE if the expression before is smaller than the one after the < operator and FALSE otherwise, (a < b).
>=	Greater than or equal	Returns TRUE if the expression before is greater than or equal to the one after the >= operator and FALSE otherwise, (a >= b).
<=	Less than or equal	Returns TRUE if the expression before is smaller than or equal to the one after the <= operator and FALSE otherwise, (a < =b).

iv) Bitwise Operators

Bitwise operators (also called Binary operators) are used to perform bit-by-bit manipulation of data. In Python, Bitwise operators act on integers only to return decimal values. Table 17.5 presents the three Bitwise operators

Table 17.5: Bitwise Operators

Symbol	Operation	Explanation
&	Bitwise AND	Returns 1 if and only if both bits are 1, (a&b).
|	Bitwise OR	Returns 1 if either bit is 1, (a|b).
^	Bitwise XOR	Returns 1 if either bit is 1, but 0 if both bits are 1, (a ^ b).

v) Assignment Operators

Assignment operators are used to assign values to variables. It enables the creation, initialisation and increase or decrease of variables in computer program. The use of an assignment operator is a powerful strategy for writing robust code to allow the reuse of variables throughout the program. Table 17.6 shows examples of assignment operators with their explanations.

Table 17.6: Assignment Operators

Symbol	Operation	Explanation
=	Assignment	Assigns the value of the expression after to the variable before the = operator, (a=b).
+=	Add and assign (a+=b)	Adds the term after to the term before and assigns the result to the before the += operator. The operator means a=a+b.
-=	Subtract and assign (a-=b)	Subtracts the term after from the term before and assigns the result to the before the -= operator. The operator means a=a-b.

*=	Multiply and assign (a*=b)	Multiply the term after with the term before and assigns the result to the before the *= operator. The operator means a=a*b.
/=	Divide and assign (a/=b)	Divides the term before by the term after and assigns the result to the before the /= operator. The operator means a=a/b.
%=	Modulus and assign (a%=b)	Returns the remainder of dividing the term before with the term after % operator and assigns the result to the variable on the left. The operator means a=a%b.

vi) Increment/Decrement Operators

There are two operators for increment and decrement of variables in most programming languages, denoted by ++ and –, respectively. However, Python does not use these operators since similar result can be obtained by += and -=, respectively. In addition, these operators are commonly used in loop, whereas Python's 'for' loop is generally implemented by range () function instead of the '++' and '- -' operators. The common increment and decrement operators, as used in other programming language, is presented in Table 17.7.

Table 17.7: Increment/Decrement Operators

Symbol	**Operation**	**Explanation**
++	Post-increment	Increments the value of the variable by 1 for successive iteration, (a++)
++	Pre-increment	Increments the value of the variable by 1 for before iteration, (++a)

--	Post-decrement	Decrements the value of the variable by 1 for successive iteration, (a--).
--	Pre-decrement	Decrements the value of the variable by 1 for before iteration, (--a).

17.6 Functions

In the computer program, a function is a block of related code that is organised to perform a single task. It provides the means of formulating a relationship between a set of inputs to yield an output. This enhances modularity in writing computer program and promotes reusability of codes. Generally, functions are represented as y= f(x). For instance, $f(x) = x^3$. Reads as 'f of x is equal to x cube'. Functions are often used to model real-life situations. A function model is used to show the relationship between two or more quantities. For instance, revenue payment is a function of interest earned. Once the execution of the program encounters a 'function call,' the flow of control jumps to the lines of code in the instruction inside the function. The flow of control returns to continue from the point immediately after the 'function call' with the result of the function executed from top to bottom. Function is of two types: Built-it function and User-defined function.

a) Built-in Functions

Built-in functions are functions that are already prepared in an application, programming packages or other end users' tools. The built-in functions are prepared to support the optimum usage of business applications and programming languages. There are many built-in functions to support the core functionalities of applications and programming languages without importing any external library or module. Six examples of built-in functions used in Python programming language are input(), print(), max(), min(), sum() and range().

i) input() : Enable users to enter items into the system

ii) print() : Displays the result to the output devices

iii) max() : Returns the highest valued variable

iv) min() : Returns the lowest valued variable

v) sum() : Returns the total sum of variables in iteration

vi) range(): Returns asset of consecutive numbers starting from 0 by default.

b) User-defined Functions

User-defined functions are functions built by the programmer to suit the routines and procedures of the programming language as well as fulfil the system's requirements. This is a fundamental building block of any programming language to provide a user with customised instructions to read and write parameters according to their needs. In Python, a user-defined function starts with the keyword 'def', followed by the 'function name'. A user-defined function may take arguments in a bracket. This is followed by a block of instructions in the body of the function. The syntax of a user-defined function in Python is:

```
def  FunctionName (arg1, arg2, …….):

     Instruction line 1

     Instruction line 1

     ……..

     return value
```

The syntax for a function call is:

```
FunctionName(arg1, arg2, ..... , );
```

Where arg1, arg2, ..., argN are arguments passed to user-defined functions.

```
# An example of a Python program to demonstrate the use of user-defined functions

def overtime_due(rate,hours):
```

```
    overtime_due = rate * hours
    return overtime_due


rate = 5
hours = 6
print("The overtime payable =", overtime_due(rate,hours))
```

Output

```
The overtime payable = 30
```

17.7 How to begin the problem-solving process

Programmers must solve many different types of problems. One problem that students often encounter in programming classes is how to find the lowest or highest value in a list of numbers. Review the following example and note the steps a programmer could take to solve the problem.

Example problem:

Find and display the largest number in this list:

[1218, 9957, 1949, 5683, 2665, 5382, 1322, 9458, 4061, 8400,
6955, 1419, 8585, 211, 5299, 5600, 1414, 285, 9515, 5989, 3387,
7613, 3489, 8499, 9981, 1448, 8976, 2363, 564, 9481, 784, 800,
1018, 6613, 8448, 7591, 9561, 983, 1605, 8979, 4068, 4978, 4050,
3479, 5356, 8644, 3235, 438, 8417, 432, 4478, 1619, 2075, 66,
1912, 6749, 127, 9492, 5964, 9854, 646, 3730, 2335, 2105, 6566,
3898, 7832, 8238, 5361, 4237, 7791, 1261, 7227, 8850, 9801,
9511, 9109, 5453, 8071, 65, 4590, 8377, 6805, 1060, 5293, 8673,
4219, 440, 6102, 9931, 2185, 2745, 6261, 9513, 2981, 491, 5477,

1426, 7276, 664, 5996, 6232, 5380, 4085, 1685, 9611, 6102, 431,
505, 8748, 5631, 2704, 8309, 702, 3741, 4178, 6896, 326, 932,
3854, 4567, 870, 3721, 1460, 2563, 8607, 5230, 2947, 5171, 6248,
1383, 3637, 1952, 1513, 2170, 2852, 3450, 3282, 3607, 8896]

Step 1 – Identify the problem that must be solved.

The first step is to identify the problem that needs to be solved. In this example, the largest number in the list must be found and displayed.

Step 2 – Understand what the problem presents.

The problem presents a list of numbers. A quick glance at the list shows all the numbers to be positive. Thus, it is a list of multiple positive numbers.

Step 3 – Determine how to solve the problem.

There are multiple approaches that could be taken to solve any given problem. In this example, the number with the largest value must be found, so one solution would be to cycle through each number and determine if it is the largest value. This can be accomplished by comparing the numbers and discarding the ones that have lower values.

Step 4 – Develop a method to achieve the solution.

It is established that all the numbers are positive, meaning that none of the numbers will be less than the value of 0. A process can be used that will compare each number in order of occurrence to a check number that will initially be set to 0. If the number being compared has a higher value than the check number, then that value will be the new check number. Then, proceed to repeat the process with each subsequent number in the list. When the process is finished, the number with the highest value will be the final check number.

The solution can be obtained through the following process.

First, make a check number; set it to 0.

The check number needs to be lower than the highest value, but the highest value is currently unknown. In this example, there are no negative signs (-) on any of the numbers; thus, there are no negative numbers. Therefore, it can be assumed that 0 will be lower than the highest number in the list.

Second, look at each number in the list and compare it to the check number.

If a number is greater than the check number, change the check number to be the same as the number being examined; however, if a number is less than the current check number, leave the check number unchanged.

The following sequence shows how those steps would be applied to the list:

3 Check number = 0.
4 The first number in the list: 1218.
5 1218 is greater than 0, so the check number is changed to 1218.
6 The check number is now 1218.
7 Move to the next number in the list.
8 Next number in the list: 9957.
9 9957 is greater than 1218, so the check number is changed to 9957.
10 The check number is now 9957.
11 Move to the next number in the list.
12 Next number in the list: 1946.
13 1946 is not greater than 9957, so the check number is left unchanged.
14 The check number remains 9957.
15 Move to the next number in the list.

Continue the process until each number in the list has been examined.

Lastly, display the final check number.

When the process is complete, the check number will be the largest number. For this example, it is 9981. Therefore, 9981 will be displayed.

17.8 How to use pseudocode during the planning process

Pseudocode informally describes the step-by-step process that will be implemented to solve a problem or accomplish a task, and it is an integral part of the planning stage. The pseudocode for this example is as follows:

Make a check number; set it to 0.

Look at each number in the list and compare it to the check number.

If a number is higher than the check number, change the check number to that number. If it is not, the check number remains the same.

Repeat this step until each number in the list has been checked.

Display the final check number.

The blocks of code shown in Fig. 17.5 demonstrate how the steps would be written in the Python, Java, and C++ programming languages. The "listOfNumbers" variable that is featured in the code represents the list of numbers that were presented in the problem.

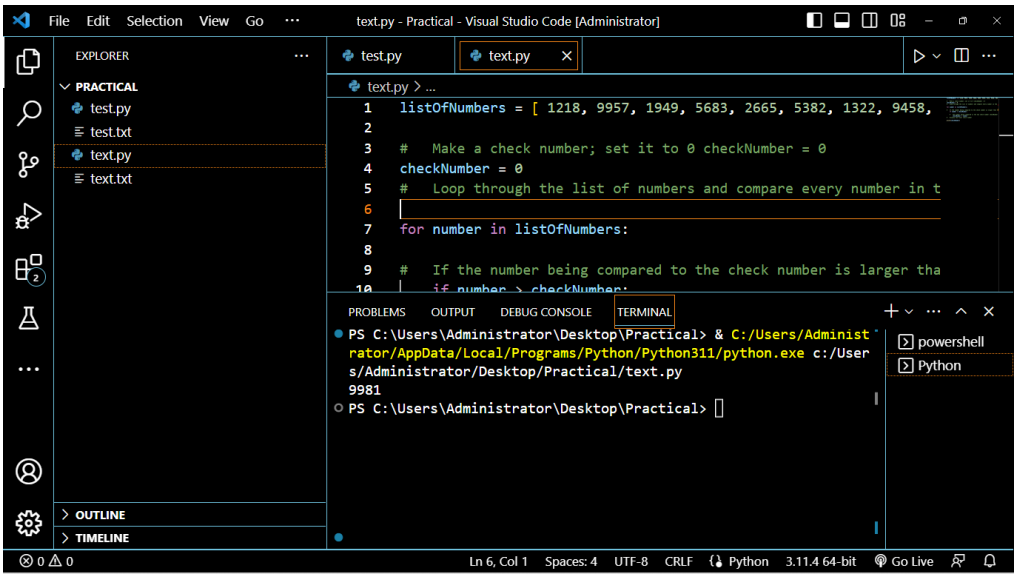


Figure 17.5: Block of Code that demonstrates how the steps would be written in Python

Summary

This chapter presented a comprehensive understanding of problem-solving concepts in computer science with emphasis on the definition of constants and variables, data and data types, including its implementation using Python programming language. Also learned is the verification of variables' data types as well as type conversion in Python. The problem-solving concepts of using expressions and equations was well explained. The definition of operators and how they are utilised in problem-solving and computer programming was sufficiently discussed. Finally, the chapter x-rayed how functions are implemented for problem-solving in computer science.

Questions

1. Lists and explains, for example, the different categories of data type in Python.
2. List and explain, for example, the categories of operators in Python.
3. List down the rules for naming variables with examples.

4. Describe in detail what you understand by expression.
5. Demonstrate with Python how to display data types with variables and constants.
6. Write a Python program to explain the execution of a user-defined function.
7. What is the importance of using Function in problem-solving and computer program?

Multiple Choice Questions

1. Which of these is not a core data type?

A. List

B. Tuple

C. Dictionary

D. Class

2. Which of the following functions is used to know the data type of a variable in Python?

A. type()

B. typeof()

C. datatype()

D. var type()

3. Which of the following is the correct extension of the Python file?
A. .python
B. .pl
C. .py
D. .p

4. Which keyword is used for function?
A. Fun

B. Define

C. def

D. Function

5. Which of the following functions is a built-in function in Python?
A. seed()

B. sqrt()

C. factorial()

D. print()

6. How to add a single-line comment in Python?

A. /* This is a comment */
B. # This is a comment
C. // This is a comment
D. !! This is a comment

7. How to create a variable in Python with a value of 22.6?

A. int a = 22.6

B. a = 22.6

C. Integer a = 22.6

D. None of the above

8. How do you get the total length of a Tuple in Python?

A. count() method

B. length() method

C. len() method

D. str() method

9. To get today's date, which Python module is to be imported?

A. calendar module

B. datetime module

C. dateutil module

D. date can module

Answers

1. D
2. A
3. C
4. C
5. D
6. B
7. B

8. C
9. B

CHAPTER EIGHTEEN
Problem-Solving Concepts for the Program Structure

Overview

This chapter provides sufficient knowledge about problem solving for the computer as it relates to the program structure. It sheds light on some important program structures like decision logic structure and loop logic structure. Key concepts and definitions of terms were discussed while snippets of code implementation of some of the design concepts are given in Python programming language.

Objectives

This chapter focuses on providing students with the basic knowledge of computer problem-solving concepts. Specifically, students should be able to:

a. define what problem solving is.
b. list and explain types of problem solving structures.
c. understand what the IF/THEN/ELSE statement means and its structure.
d. define clearly the CASE/SWITCH structure.
e. clearly identify the different loop logic structures.
f. implement some of the design concepts highlighted with Python programming language

Problem-Solving Concepts for the Program Structure

A computer is a device used in solving problems relating to computation and other human-related tasks WHILE computer-based problem solving is an organised process of designing, implementing and using computer programming tools WHILE passing through the problem-solving stage.

The process of transforming the description of a problem into a solution by using our knowledge of the problem domain and by relying on our ability to select and use appropriate problem-solving tactics, methods and tools is known as problem-solving.

The task of writing programs for the computer is known as programming, which is also a problem-solving activity. Writing a program is the same as writing an instruction for the computer to solve something for us.

18.1 Decision Logic Structure

A *Decision Structure* is a technique or concept in a computer program that permits the program to decide and change its behaviour based on that decision. The outcome of a logical test enables us to make the decision, and a logical test is a computation whose result is either true or false.

18.1.1 IF/THEN/ELSE Statement

In the **IF** Control Structure, we have a **conditional control structure** that executes depending on a **particular condition**. If that particular condition is ***True***, THEN the **IF** block will execute ELSE that block is skipped and will not be executed. It is not mandatory to attach the ELSE part if it is not necessary. An example could be *IF any worker has done overtime for the monthly period, calculate his Overtime Pay*. Fig. 18.1 shows the flowchart for the IF control structure.

Figure 18.1*: Flow Chart diagram of IF Control Structure*

Example 8.1

An example of this structure using Python programming language is:

```
if expression:
    statement
```

```
# Simple Python Program to print the largest of the three numbers.
```

```
x = int (input("Enter x: "));
```

```
y = int (input("Enter y: "));

z = int (input("Enter z: "));

if x>y and x>cz

# Here, we are checking the condition. If the condition is true, we wil
l enter the block

print ("From the above three numbers given x is largest");

    if y>x and y>z:

# Here, we are checking the condition. If the condition is true, we wil
l enter the block

print ("From the above three numbers given y is largest");

    if z>x and z>y:

# Here, we are checking the condition. If the condition is true, we wil
l enter the block

print ("From the above three numbers given z is largest");
```

Output:

```
Enter x: 100

Enter y: 120

Enter z: 130
```

From the above three numbers given, z is the largest.

The **IF-ELSE** Control Structure is another conditional control structure that executes depending on the outcome of a particular condition. If that condition is ***true***, THEN the **IF block** part is executed. Otherwise, the **ELSE block** part is executed. Fig. 18.2 gives a diagram of the IF-ELSE control structure. *An example could be, IF the Voter's age is up to or above 18 years, ALLOW the person to vote; OTHERWISE, do not allow the person to vote.*

Figure 18.2*: Flow chart diagram of IF-ELSE statements*

Example 8.2:

Program to check if a person is eligible to vote or not

```
# Simple Python Program to check whether a person is qualified
to vote or not.
```

```
age = int (input("Enter your current age: "))

# Here, we are taking an integer num and taking input dynamically

if age>=18:

# Here, we are checking the condition. If the condition is true, we wil
l enter the block

   print("You are qualified to vote !!");

   else:

   print("Sorry! you are not qualified to vote!!");
```

Output:

```
Enter your age: 42

You are qualified to vote !!
```

Another control structure is the **IF-ELSE-ELSE-IF** control structure, which is used in checking for multiple conditions or options. Every part of the IF-ELSE-IF block has one condition each, and every time the structure executes, the condition is checked with each block. If the condition becomes true, THEN that block is executed, WHILE every other block is skipped, and if **none** of the conditions is true, then only the **ELSE** part is executed. Fig. 18.3 gives us a diagram of the IF-ELSE-ELSE-IF control structure.

Figure 18.3*: Flow chart diagram of IF-ELSE-IF-ELSE statements*

The IF-ELSE-IF-ELSE structure is implemented in Python using the ELIF statement, which works exactly the same way as we have described for the IF-ELSE-IF-ELSE statement.

The syntax of the ELIF statement in Python is given as:

```
if expression 1:
   # block of statements
elif expression 2:
```

```
    # block of statements
elif expression 3:
    # block of statements
else:
    # block of statements
```

Example 8.3: # Simple Python program to understand elif statement

```
number = int(input("Enter the number?"))
# Here, we are taking an integer number and input dynamically
    if number==10:
# Here, we are checking the condition. If the condition is true, we wil
l enter the block
  print("The given number is equals to 10")
    elif number==50:
# Here, we are checking the condition. If the condition is true, we wil
l enter the block
  print("The given number is equal to 50");
    elif number==100:
```

```
# Here, we are checking the condition. If the condition is true, we wil
l enter the block
    print("The given number is equal to 100");
else:
    print("The given number is not equal to 10, 50 or 100");
```

Output:

```
Enter the number.15
The given number is not equal to 10, 50 or 100
```

18.1.2 CASE/SWITCH Statement

Another control structure that allows an option to be selected from multiple alternatives is the **CASE** or SWITCH statement. It is used to allow the value of a variable or expression to change the control flow of program execution via a multiway branch. An example is given in Fig. 18.4.

Figure 18.4: CASE/SWITCH statement in C programming language

The SWITCH statement first assesses an expression, THEN search for the first CASE clause whose expression evaluates to the same value as the result of the input expression (using the strict equality comparison) and transfers control to that clause, executing all statements following that clause.

Example 8.4:

Unlike many other programming languages, Python does not have a CASE or SWITCH statement, but at our elementary level, we can implement the CASE/ SWITCH statement by using the IF-ELSE statement. This statement is used to determine whether a specific statement or block of statements will be performed or not, i.e., whether a block of statements will be executed if a specific condition is true or not.

```
bike = 'Yamaha'

if bike == 'Hero':

  print("bike is Hero")

elif bike == "Suzuki":

  print("bike is Suzuki")

elif bike == "Yamaha":

  print("bike is Yamaha")

else:

  print("Please choose correct answer")
```

Output:

```
bike is Yamaha
```

Example 8.5:

But in Python 3.10 and later versions, a CASE statement is given with a MATCH statement used in place of SWITCH.

```
# This code runs only in Python 3.10 or above versions

def number_to_string(argument):

  match argument:

    case 0:
```

```
        return "zero"
    case 1:
        return "one"
    case 2:
        return "two"
    case default:
        return "something"
head = number_to_string(2)
print(head)
```

18.2 Loop Logic Structure

A loop structure is also known as a Repetition structure or an Iteration structure. In such a structure, the program asks a question, and IF the answer requires an action, it is performed, and the original question is asked again and again until the answer is such that the action is no longer required.

For example, a program written to compute a department's student CGPA for each student will begin by computing the CGPA of one student and continue performing that action in a loop until there are no more students CGPA to be computed, and only then will the program move on to its next action. Each movement through the loop is called an *iteration*. Loops are one of the three vital control structures in programming.

A computer programmer who needs to use the same lines of code many times in a program can use a loop to save time.

For example, take the simple task of eating a full plate of rice. Even though you may not realise it, you're constantly asking yourself, "Is the entire rice finished or am I

fully satisfied?" When the answer is **no**, you **take another spoonful of rice**. You continue to repeat this question until the answer is **yes**, and then you **stop**.

Loops allow programmers to efficiently code tasks that are repetitive instead of having to write the same code over and over again.

18.2.1 Incrementing

Increment is a term that decides how the loop control variable is increased each time the loop repeats successfully (that is, each time condition is evaluated to be true).

For a loop to work well, it must devise some means of counting from the point of origin up to the point of termination. Defining a loop control variable to start the loop is one aspect, WHILE the control variable could be incremented or decremented, which is another aspect. A third aspect is for the test to be done each time the component part or body of the loop is executed to know IF the point of termination has been reached or exceeded.

18.2.2 WHILE/WHILE-END

In a "WHILE" Loop, a specific block of code or statements is repeatedly executed for an unknown number of times until a condition is met.

For example, IF we want to ask a user for an alphabet between A and K, we don't know how many times the user may enter a particular alphabet or even enter an alphabet outside this range, so we keep asking "WHILE the alphabet is between the range of A and K", keep on executing the block of codes given.

The beginning of the loop is indicated by the keyword WHILE, and the end of the loop has the keyword ENDWHILE.

The general form of the loop is:

```
WHILE condition
    sequence
ENDWHILE
```

Figure 18.5 shows a flowchart of the WHILE loop.

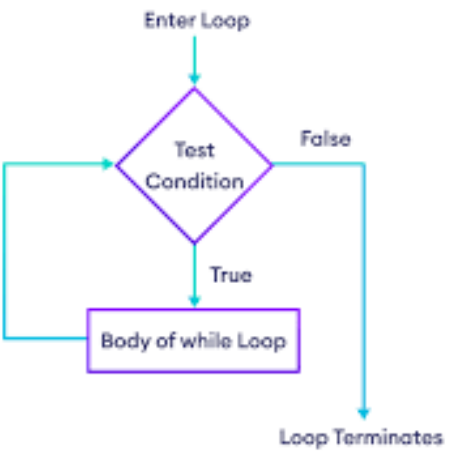


Figure 18.5**:** WHILE loop diagram

The body of the loop is entered only when the condition specified is true. The "sequence or statements" is performed for each repetition. At the conclusion of each repetition or iteration, the condition is evaluated, and the loop continues as long as the condition remains true.

Another good example is that we may want to know how many times a given number can be divided by 2 before it is less than or equal to 1. If we know a specific number, such as 32, we can say 5 times, but for a given symbolic variable, "NUMBER", which represents any number in the world, how many times is **not** known a priori (beforehand)? In this CASE, we could use a WHILE loop to determine that answer:

The "pseudocode" for such an algorithm is: WHILE the number is bigger than one keeps dividing it by two. Additionally, keep a count of how many times we do the division.

GET our number

SET our initial count to 0

WHILE our number is greater than 1

divide the number by 2

increase our count by 1

END

Why do we use WHILE Loops?

1. WHILE loops execute blocks of code over and over again, which makes them a powerful iteration statement.
2. The WHILE loop will repeat as often as necessary to accomplish its goal.
3. Generic Syntax is very easy to understand.
4. We could avoid infinite loops- easily with the WHILE loop.

An example of the WHILE loop implementation in Python programming language would be:

Example 8.6:

A program code in Python for printing numbers from 1 to 10 using the WHILE loop statement;

```
i=1
while i<=10:
  print(i, end=' ')
  i+=1
```

Output is:

1 2 3 4 5 6 7 8 9 10

18.2.3 Repear/Until

The REPEAT/UNTIL loop is similar to the WHILE loop except that the conditional test is performed at the bottom of the loop instead of at the top and only when the sequence of statements must have been executed at least once. Two keywords, REPEAT and UNTIL, are used.

The general form is:

REPEAT *sequence of statements* UNTIL condition

or

```
REPEAT
        Sequence of statements
UNTIL condition
```

The "sequence of statements" is always performed at least once because the test is performed after the sequence of statements is executed. At the conclusion of each iteration, the condition is evaluated, and the loop repeats IF the condition is false. The loop terminates when the condition becomes true. Fig. 18.6 gives a flowchart description of the REPEAT/UNTIL loop.

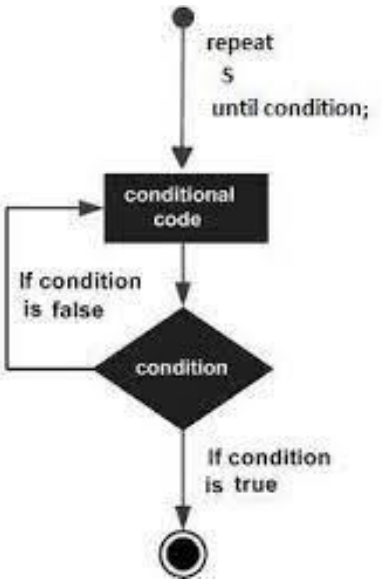


Figure 18.6**:** REPEAT..UNTIL loop diagram

Example 8.7:

The following example finds the smallest power of 2 that is greater than B:

A = 1

B = 10

REPEAT A = A * 2 UNTIL A > B

With a WHILE loop, the code within the iteration may never be executed once the condition is false. With a REPEAT UNTIL loop, the code is always executed at least once.

Example 8.8

Python does not have the REPEAT-UNTIL loop, but we can implement it using the WHILE loop by looping until a certain condition evaluates to **TRUE**. To do that, we can choose between a FOR loop or a WHILE loop to start repeating lines of code.

In the following example, we will make a simple for loop over a list. This loop will loop until the condition we put in the if-clause is met or all items have been iterated over:

```
total_points = 0
results = [70, 80, 80, 60, 40, 90, 20, 80]
for result in results:
    if result < 60:
        break
    print(result)
    total_points += result

print(total_points)
```

The Printed Output is:

```
70
80
80
60
```

290

18.2.4 Automatic Loop Counting

In automatic loop counting, we know beforehand how many times the loop will execute and plan or organize the counter and control variable in advance so they could work better.

Because we know the exact number of times the loop repeats beforehand, a counting loop can be made dependent on the value of a counter.

A common type of program loop is one that is controlled by an integer that counts up from an initial value to an upper limit. Such a loop is called a counting loop. The integer is called a loop control variable. The FOR..NEXT loop in C, C++ and Java plus Do..CONTINUE loops in Fortran are good examples of such loops.

For Loop in Python:

In Python, a `for` loop is used to iterate over sequences such as lists, tuples, strings, etc.

The syntax of a `for` loop is:

```
for val in sequence;

# statement(s)
```

Here, `val` accesses each item of the sequence on each iteration. The loop continues until we reach the last item in the sequence.

Example 8.9:

```
# Python program to show iterating over a range of 0 to n-1

n = 4
```

```
for i in range (0, n) :
        print (i)
```

Output becomes:

```
0
1
2
3
```

In automatic loop counting, we define an integer variable we use to start our counting and THEN define another integer variable to mark the upper bound of the loop, which would be tested every time the loop executes. Another variable is defined to decrement or increment the loop WHILE the statement or statements to be executed are placed within the body of the loop.

18.2.5 Nested Loop

When we place one or more loops inside the body of another loop, THEN we have a nested loop. The loop placed inside is called the inner loop, while the one outside is called the outer loop. In each iteration of the outer loop, the whole inner loop is executed. This happens in the following way:

When nested loops start executing or running, the outer loop
starts first: initialization of its control variable is performed, and after checking for the end of the loop, the code in its body is executed.

After that, the inner loop is executed. The control variable's start position is initialized, a check for the end of the loop is made, and the code in its body is executed.

When the set value for the end of the loop is reached, the program goes back one step up and continues executing the previous outer loop. The control variable of

the outer loop changes with one step; a check is made to see IF the condition for the end of the loop is met, and a new execution of the nested (inner) loop is started.

This is repeated until the variable of the outer loop meets the condition to end the loop. An example of a nested loop is shown in Figure 18.7.

Figure 18.7**:** Example of a nested loop

These loops are particularly useful when evaluating or processing multidimensional data like arrays. When using these loops, the first iteration of the first loop will initialize, followed by the second loop.

Analogue clocks have one gear as the nested loop, and every full rotation knocks the minute gear on by one, etc. We can take this a step further and say that clocks

are just a form of counting system. And IF we think about it like that numbers themselves are a real-world example of nested loops.

Nested Loops in Python:

Outer_loop Expression:

Inner_loop Expression:

Statement inside inner_loop

Statement inside Outer_loop

Example 8.10

```
x = [1, 2]
y = [4, 5]
for i in x:
 for j in y:
  print(i, j)
```

The output will be:

```
1  4
1  5
2  4
```

2 5

18.3 Summary of Chapter

This chapter presented an overall understanding of the basic programming solving concepts for the program structure and their types. We looked at the aspect of making selections or choices in programming, which could be represented with the IF/ELSE and CASE/SWITCH statements and examples shown with the flowchart decision box.

The simple case of making a selection was given with the IF statement followed by multiple cases with IF-ELSE-IF and more with the CASE/SWITH variant.

The other control structure, which involves repetition or looping of some or all aspects of a program, was also treated with examples in the form of WHILE/ END for the pre-selection loop and REPEAT/UNTIL for the post-selection loop structures given. We also showed that loops can also use the FOR----NEXT or DO...CONTINUE structure is available in some languages, and the user or programmer can use any of the above structures to program his code.

The concept of nesting a loop within another loop, which means including a loop within another loop, was also explained with examples of how automatic loop counting in programming could be realized.

We hope your journey into the exciting world of programming has started smoothly. Enjoy the ride.

Exercises

1. **Computers cannot solve problems on their own. We must provide clear, step-by-step directions on how to solve the issue. This solving technique is known as?** a. Problem Solving

b. Problem Addressing c. Problem Analysis d. None of the above

2. What is **the process of identifying a problem, developing an algorithm for the identified problem and finally implementing the algorithm to develop a computer program?** a. Problem Solving

b. Problem Addressing c. Problem Analysis d. None of the above

3. After finalizing the algorithm, we need to convert the algorithm into what? a. Format which can be understood by the computer

b. High-level programming language

c. Both a) and b) d. None of the above

4. To complete each activity in a computer, we follow a sequence of steps. This sequence of steps is known as? a. Problem

b. Algorithm c. Problem Analysis d. None of the above

5. What is the act of locating and fixing problems in software code that could lead to unexpected behaviour or crashes? These errors are sometimes referred to as "bugs." a. Algorithm

b. Problem Solving c. Debugging d. All of the above

6. The purpose of using an algorithm is to?

a. Increase the reliability b. Accuracy of the program

c. Efficiency of obtaining solutions d. All of the above

7. The Characteristics of a good algorithm are?

a. Precision & Uniqueness b. Finiteness c. Input & Output

d. All of the above

8. What is used as a visual representation of an algorithm?

a. Flowchart b. Pseudocode c. Algorithm d. None of the above

9. A flowchart is a diagram made up of? a. Boxes

b. Diamonds c. Shapes d. All of the above

10. A _____ or branching point, usually a yes/no or true/ false question, is asked, and based on the answer, the path gets split into two branches. a. Terminator b. Action Symbol

c. Decision d. Input/ Output

11. What is the connector that shows the order of flow between shapes?

a. Terminator b. Action Symbol c. Decision d. Arrow

12. A _____ is another way of representing an algorithm. It is considered a non-formal language that helps programmers to write algorithms. a. Flowchart b. Pseudocode

c. Algorithm d. None of the above

13. A program written in a high-level language is called what?

a. Source code b. Object c. Machine language

d. None of the above

References

Awati, R. (1999-2023, April 14). Operators. Retrieved from TechTarget.

Taban, J., & Cadorna, E. (2018). Structure sense in algebraic expressions and equations of groups of students. Journal of Educational and Human Resource Development (JEHRD), 6, 140-154.

Blair, A., Duguid, P., Goeing, A.-S., & Grafton, A. (2021). Information: A Historical Companion. Princeton: Princeton University Press.

Bundy, A., Wallen, L. (1984). Means/Ends Analysis. In A. Bundy & L. Wallen (Eds.), Catalogue of Artificial Intelligence Tools. Symbolic Computation. Springer, Berlin, Heidelberg. https://doi.org/10.1007/978-3-642-96868-6_138

Copeland, B. J. (1996). What is computation? Synthese. https://doi.org/10.1007/BF00413693

Grossman, D. A., & Frieder, O. (2004). Information Retrieval: Algorithms and Heuristics (2nd ed.). ISBN 1402030045.

Davis, W. S. (2019). Logic (process) flowcharts. In The Information System Consultant's Handbook (pp. 439-448). CRC Press.

Dierbach, C. (2013). Introduction to Computer Science Using Python: A Computational Problem-Solving Focus. John Wiley & Sons. Retrieved from **http://bedford-computing.co.uk/learning/wp-**content/uploads/2015/10/Introduction-to-Computer-Science-Using-Python.pdf

Dromey, R. G. (2009). How to Solve It by Computer. Pearson Education India.

B. Heine (2015). Handbook of Linguistics, Heine-main Heine-Narrog, pp. 449-472.

Gómez, L. G., & Wren, D. A. (2021). The Art of Solving Problems: Principles and Strategies for Success in Business and Life. Routledge, London.

National Diploma in Computer Technology Introduction to Computing. (2008).

Robinson, L. (1993). Introduction to computing. International Journal of Information Management.

Immerman, Neil (2021). "Computability and Complexity". The Stanford Encyclopedia of Philosophy (Winter 2021 Edition), Edward N. Zalta (ed.). URL https://plato.stanford.edu/archives/win2021/entries/computability/

Jeffrey, C. L. (2021). Build Your Own Programming Language: A Programmer's Guide to Designing Compilers, Interpreters, and DSLs for Solving Modern Computing Problems. United Kingdom: Packt Publishing.

Kabir, S.M.S. (2016). Basic Guidelines for Research: An Introductory Approach for All Disciplines. Book Zone Publication, ISBN: 978-984-33- 9565-8, Chittagong-4203, Bangladesh.

Kamiński, B., Jakubczyk, M., and Szufel, P. (2017). "A framework for sensitivity analysis of decision trees". Central European Journal of Operations Research, 26(1), 135–159. doi:10.1007/s.

Larson, L.C. (2020). Problem-Solving Through Problems. Springer, Cham, Switzerland.

Valiant, L. G. (2003). Three problems in computer science. Journal of the ACM, Volume 50, Issue 1, pp. 96–99. https://doi.org/10.1145/602382.602410

Liedtka, J., King, A., and Bennett, K. (2022). Solving Problems with Design Thinking: Ten Stories of What Works. Columbia University Press, New York.

Merriam-Webster Online Dictionary. (2020). Definition of ALGORITHM. Archived from the original on February 14, 2020. Retrieved April 14, 2023.

National Centre for Computer Education. (2022). File Handling Fundamentals.

Olabiyisi, S. O., Olalere, M., Akande, O. N., Sanjay, M. R., Akashat, A., and Jonathan, O. (2022). Comparison of Selected Algorithms on Breast Cancer Classification. In Advances in Electrical and Computer Technologies (pp. 161–171). Lecture Notes in Electrical Engineering, vol 881. Springer, Singapore.

Patjoshi, R., Petro, S., and Jena, S. (2022). Fundamentals of Problem Solving and Python Programming. Lambert Academic Publishing. ISBN: 9786205515617.

Pólya, G. (2014). How to Solve It: A New Aspect of Mathematical Method. Princeton Science Library.

Posner, E. A., Spier, K. E., & Vermeule, A. (2010). Divide and conquer. Journal of Legal Analysis, 2(2), 417-471.

Pressman, R. S., and Maxim, B. R. (2015). Software Engineering: A Practitioner's Approach. New York: McGraw-Hill.

Quadri, B. A., Olabiyisi, S. O., Folowosele, A. O. (2021). Performance Evaluation of Some Selected Metaheuristic Techniques for Intelligent Path Planning of Mobile Agents. iJournals: International Journal of Software & Hardware Research in Engineering (IJSHRE), Volume 9, Issue 2, pp. 12-24. ISSN-2347-4890.

River, N. J. (1994). Effective Problem Solving, 2nd edition. Marvin Levine, Prentice Hall, Upper Saddle.

Rogers, William T. (2007). "Decision Table Examples: Medical Insurance". Saint Xavier University Systems Analysis and Design. Archived from the original on March 29, 2007.

Rooney, J. J., & Heuvel, L. N. V. (2004). Root cause analysis for beginners. Quality progress, 37(7), 45-56.

Ross, Ronald G. (2005). "Decision Tables, Part 2 ~ The Route to Completeness". Business Rules Journal, 6(8). Retrieved April 14, 2023.

Seacord, R. C. (2020). Effective C: An Introduction to Professional C Programming. United States: No Starch Press.

Şengül, S., & Üner, I. (2010). What is the impact of teaching "Algebraic Expressions and Equations" topic with concept cartoons on the students' logical thinking abilities? Procedia-Social and Behavioral Sciences, 2(2), 5441-5445.

Snow, Paul (2012). "Decision Tables". DTRules: A Java Based Decision Table Rules Engine. Retrieved April 14, 2023.

Taban, J., & Cadorna, E. (2018). Structure sense in algebraic expressions and equations of groups of students. Journal of Educational and Human Resource Development (JEHRD), 6, 140-154.

Tambone, D. (2021). The Problem-Solver's Toolkit: A Surprisingly Simple Guide to Your Lean Six Sigma Journey. Kogan Page, London.

Teichroew, D., & Hull, T. E. (1967). Introduction to Computing. The American Mathematical Monthly.

To, I., & Markets, E. (1999). Chapter One INTRODUCTION TO. 1–4. https://www.easytechjunkie.com/what-is-a-midrange-computer.htm [accessed on April 13, 2023].

Treffinger, D. J., Isaksen, S. G., & Stead-Dorval, K. B. (2006). Creative problem solving: An introduction. Prufrock Press Inc.

Turing, A. (1950). "Computing Machinery and Intelligence." Mind, LIX(236), 433-460.

Varol, O. (2020). Think Like a Rocket Scientist: Simple Strategies You Can Use to Make Giant Leaps in Work and Life. Public Affairs, New York.

Watanabe, K. (2020). Problem-Solving 101: A Simple Book for Smart. Portfolio, New York.

Williams, P. M. (2001, April). Techniques for root cause analysis. In Baylor University Medical Center Proceedings (Vol. 14, No. 2, pp. 154-157). Taylor & Francis. **https://fayllar.org/method-of-focal-objects-is-**technique.html [accessed on April 13, 2023].

Zeitz, P. (2021). The Art and Craft of Problem Solving. John Wiley & Sons, Inc., Hoboken, New Jersey.

Zinoviev, D. (2021). Pythonic Programming. United States: Pragmatic Bookshelf